



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA STROJNÍHO INŽENÝRSTVÍ

FACULTY OF MECHANICAL ENGINEERING

ÚSTAV AUTOMATIZACE A INFORMATIKY

INSTITUTE OF AUTOMATION AND COMPUTER SCIENCE

**NÁVRH VIBRODIAGNOSTICKÉHO SYSTÉMU
S VYUŽITÍM STROJOVÉHO UČENÍ**

DESIGN OF VIBRODIAGNOSTIC SYSTEM ENHANCED BY MACHINE LEARNING

DIPLOMOVÁ PRÁCE

MASTER'S THESIS

AUTOR PRÁCE

AUTHOR

Bc. Matěj Goněk

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. Radek Poliščuk, Ph.D.

BRNO 2026

Zadání diplomové práce

Ústav:	Ústav automatizace a informatiky
Student:	Bc. Matěj Goněc
Studijní program:	Aplikovaná informatika a řízení
Studijní obor:	bez specializace
Vedoucí práce:	Ing. Radek Poliščuk, Ph.D.
Akademický rok:	2025/26

Ředitel ústavu Vám v souladu se zákonem č.111/1998 o vysokých školách a se Studijním a zkušebním řádem VUT v Brně určuje následující téma diplomové práce:

Návrh vibrodiagnostického systému s využitím strojového učení

Stručná charakteristika problematiky úkolu:

Úkolem projektu je tvorba konceptu adaptivního systému pro sledování technického stavu rotačních strojů na základě analýzy vibrací, vylepšeného o techniky strojového učení.

Systém bude využívat průmyslový hardware značky B&R, konkrétně vstupní vibrodiagnostické karty pro sběr dat a Automation PC (APC) pro lokální zpracování.

Nasbíraná data budou předzpracována, extrahují se z nich příznaky a budou uložena do časové databáze.

Student navrhne a vytrénuje vhodné modely strojového učení pro detekci závad, predikci zbývajících životností zařízení (RUL) a identifikaci anomálií.

Systém poběží zcela lokálně, bude kontejnerizovaný na platformě Docker a nabídne uživatelské rozhraní, pro zobrazení dat a záznam servisních poznámek.

Cíle diplomové práce:

1. Vývoj modulu pro sběr dat s použitím vhodné vibrodiagnostické karty B&R X20, předzpracování signálu (časová/frekvenční doména), extrakci příznaků a archivaci dat.
2. Návrh a trénování modelů strojového učení pro detekci poruch z FFT, pro detekci anomálií v časové doméně a odhad zbývajících životností (RUL).
3. Nasazení modelů na APC, pomocí lokálně běžících Docker kontejnerů.
4. Vizualizace a servisní rozhraní – dashboard pro grafickou prezentaci výsledků s uživatelským rozhraním pro operátora.
5. Rozbor použitelnosti navrženého konceptu z hlediska přesnosti, výpočetních nároků a použitelnosti v průmyslové praxi.

Seznam doporučené literatury:

FRIEDMAN, Jerome; TIBSHIRANI, Robert a FRIEDMAN, Jerome. The Elements of Statistical Learning. New York, NY: Springer-Verlag New York, 2009. ISBN 0387848576.

GOODFELLOW, Ian; BENGIO, Yoshua a COURVILLE, Aaron. Deep Learning. MIT Press, 2016. ISBN 9780262035613.

B&R Automation Help – dokumentace k X20 modulům a APC. Online. [cit. 2025-10-17]. Dostupné z: <https://help.br-automation.com/>.

Termín odevzdání diplomové práce je stanoven časovým plánem akademického roku 2025/26

V Brně, dne

L. S.

prof. Ing. Zdeněk Hadaš, Ph.D.
ředitel ústavu

doc. Ing. Jiří Hlinka, Ph.D.
děkan fakulty

ABSTRAKT

Tato práce se zabývá návrhem vibrodiagnostické aplikace s využitím metod strojového učení a vznikla ve spolupráci s firmou B&R. Začátek práce obsahuje teoretický úvod do prediktivní údržby a využití metod strojového učení v tomto oboru. Práce navrhuje kompletní řešení cyklu dat, od jejich sběru a zpracování, přes jejich uložení, vizualizaci až po vyhodnocení pomocí technik strojového učení.

ABSTRACT

The topic of this thesis is design of vibrodiagnostics system enhanced by machine learning techniques and it was done in cooperation with company B&R. The beginning of the thesis contains a theoretical introduction to the predictive maintenance and the use of machine learning techniques in this field of study. The thesis proposes a complete solution for the data cycle, from its collection and processing, through storage, visualization and evaluation by means of machine learning.

KLÍČOVÁ SLOVA

Prediktivní údržba, strojové učení, neuronové sítě, docker, kontejnerizace, React, FastAPI, PyTorch

KEYWORDS

Predictive maintenance, machine learning, neural networks, docker, containerization, React, FastAPI, PyTorch



2026

BIBLIOGRAFICKÁ CITACE

GONĚC, Matěj. *Návrh vibrodiagnostického systému s využitím strojového učení*. Brno: Vysoké učení technické v Brně, Fakulta strojního inženýrství, Ústav automatizace a informatiky, 2026, 43 s. Diplomová práce. Vedoucí práce: Ing. Radek Poliščuk, Ph.D.

PODĚKOVÁNÍ

Tímto bych chtěl poděkovat vedoucímu této práce, kterým je Ing. Radek Poliščuk, Ph.D. za jeho věcné poznámky a svědomité vedení diplomové práce. Zároveň patří díky také Ing. Tomáši Mičulkovi, který práci zaštiťoval za firmu B&R a dodal důležitou podporu týkající se využitého hardwaru.

ČESTNÉ PROHLÁŠENÍ

Prohlašuji, že, že tato práce je mým původním dílem, vypracoval jsem ji samostatně pod vedením vedoucího práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou všechny citovány v práci a uvedeny v seznamu literatury.

Jako autor uvedené práce dále prohlašuji, že v souvislosti s vytvořením této práce jsem neporušil autorská práva třetích osob, zejména jsem nezasáhl nedovoleným způsobem do cizích autorských práv osobnostních a jsem si plně vědom následku porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestně právních důsledků.

Doplňující prohlášení o využití generativní AI: V souladu se zásadami VUT uvádím, že jsem při zpracování práce použil/a nástroje generativní AI; použití je transparentně popsáno v Příloze A.

V Brně dne 20. 5. 2026

.....

Matěj Goněc

OBSAH

1	ÚVOD	11
1.1	Průmysl 4.0 a prediktivní údržba	11
1.2	Cíle diplomové práce	11
1.3	Struktura práce	11
2	VIBRODIAGNOSTIKA A ZPRACOVÁNÍ SIGNÁLU.....	12
2.1	Měření vibrací u rotačních strojů	12
2.2	Typy signálů a jejich zpracování	13
2.2.1	Časová doména	15
2.2.2	Frekvenční doména	16
2.2.3	Časově-frekvenční doména.....	17
2.3	Typické poruchy valivých ložisek a jejich projevy	18
2.3.1	Porucha vnějšího kroužku.....	18
2.3.2	Porucha vnitřního kroužku.....	19
2.3.3	Porucha valivého elementu	20
2.3.4	Další typy poruch	21
3	UMĚLÁ INTELIGENCE V PREDIKTIVNÍ ÚDRŽBĚ	22
3.1	Základy strojového a hlubokého učení	22
3.2	Detekce anomálií bez učitele	23
3.2.1	Isolation Forest.....	23
3.2.2	Autoencoder	24
3.2.3	Auto-Encoder Wasserstein Generative Adversial Network.....	24
3.3	Klasifikace poruch z frekvenčních spekter	25
3.3.1	SVM a Random Forest.....	26
3.3.2	Konvoluční neuronové sítě a model 1DCNNwWGN.....	27
3.4	Odhad zbývajících životností.....	28
3.4.1	Analytické a regresní přístupy	28
3.4.2	Rekurentní sítě a model Bi-LSTM.....	28
4	NÁVRH ARCHITEKTURY SYSTÉMU.....	30
4.1	Koncepce a topologie systému.....	30
4.2	Představení a výběr hardwaru	31
4.2.1	Modul pro měření vibrací X20CM4810	31
4.2.2	Modul pro komunikaci a rozšíření sběrnice X20BC0083	32
4.2.3	Řídicí systém X20CP3586	32
4.2.4	Průmyslový počítač Automation PC 910.....	33
4.3	Software a kontejnerizace	33
4.4	Komunikační prostředky a protokoly	34
4.4.1	Virtuální sítě prostředí Docker.....	35

4.4.2	REST API	35
4.4.3	OPC UA	35
4.4.4	FTP	36
4.4.5	POWERLINK	36
5	SBĚR, ZPRACOVÁNÍ A ARCHIVACE DAT	37
5.1	Sběr dat	37
5.1.1	Metoda kontinuálního sběru charakteristických hodnot	38
5.1.2	Metoda periodického sběru surových dat	39
5.2	Zpracování dat	41
5.3	Archivace dat a návrh databáze	41
5.3.1	Architektura databáze a relační model	42
5.3.2	Automatizace sběru dat a integrace IIoT Connectoru	43
5.3.3	Kontejnerizace a orchestrace úložiště	44
6	IMPLEMENTACE APLIKAČNÍHO ROZHRANÍ – BACKEND	45
6.1	Technologie FastAPI a Uvicorn	45
6.2	Návrh API endpointů	46
6.3	Metoda FTP Pull	48
6.4	Zabezpečení a autorizace	49
6.5	Produkční nasazení – kontejnerizace	50
7	UŽIVATELSKÉ ROZHRANÍ A VIZUALIZACE – FRONTEND	51
7.1	Vývoj webové aplikace – React	51
7.2	Představení HMI	52
7.3	Produkční nasazení – kontejnerizace	60
7.3.1	Proces kompilace a sestavení (Build)	60
7.3.2	Role webového serveru Nginx	60
7.3.3	Kontejnerizace frontendu	61
8	IMPLEMENTACE MODULU PRO STROJOVÉ UČENÍ	62
8.1	Architektura mikroslužby a systémová integrace	62
8.2	Principy a trénink diagnostických modelů	63
8.2.1	Detekce anomálií (AE-AnoWGAN)	63
8.2.2	Klasifikace poruch (1DCNN)	66
8.2.3	Predikce RUL (Bi-LSTM)	69
8.3	Fine-tuning modul	71
8.3.1	Fine-tuning modelu pro detekci anomálií	72
8.3.2	Fine-Tuning modelu pro klasifikaci poruch	73
8.3.3	Fine-tuning modelu modelu pro predikci životnosti	74
8.4	Produkční nasazení – kontejnerizace	75
9	EVALUACE SYSTÉMU	77
9.1	Evaluace modelů strojového učení	77

9.1.1	Model AE-AnoWGAN	77
9.1.2	Model 1DCNN	78
9.1.3	Model Bi-LSTM	80
9.1.4	Celkové zhodnocení modulu strojového učení	81
9.2	Nasazení aplikace na APC	82
9.3	Průmyslová využitelnost	83
10	ZÁVĚR.....	85
	SEZNAM POUŽITÉ LITERATURY	86
	SEZNAM PŘÍLOH	90

1 ÚVOD

1.1 Průmysl 4.0 a prediktivní údržba

V současné době je horkým tématem Průmysl 4.0, což je určité paradigma označující čtvrtou průmyslovou revoluci, spojenou s využitím digitálních technologií ve svůj prospěch. Nejčastějšími tématy jsou například průmyslový internet věcí, využití umělé inteligence a rozšířené reality k dosažení větší efektivity výrobního procesu. Velkým trendem je také prediktivní údržba, která umožňuje šetřit náklady za neplánované odstávky strojů.

Charakteristickým znakem této doby je velké množství dat, které je třeba zpracovat, analyzovat a vyhodnotit. Tato data můžeme následně využít v prediktivní údržbě. Prediktivní údržba spočívá v předvídání poruch a plánování odstávek a oprav strojů předem. Pomocí procesu vibrodiagnostiky získáváme data mechanických vibrací běžících strojů a ty následně vyhodnocujeme. Využitím technik strojového učení se potom snažíme na základě těchto dat detekovat poruchy, klasifikovat je a předpovídat zbývající životnost součástí stroje.

1.2 Cíle diplomové práce

Cílem této diplomové práce je vytvoření komplexního vibrodiagnostického systému prediktivní údržby ve spolupráci se společností B&R automatizace. Tento systém bude sbírat data z průmyslových vibračních senzorů, následně je bude archivovat a zpracovávat. Pomocí metod strojového učení potom bude na základě zpracovaných dat detekovat poruchy u ložisek rotačních strojů, klasifikovat je a predikovat zbývající životnost ložisek. Součástí systému také bude vizualizace celého procesu s využitím moderních technologií jako je například *kontejnerizace* či framework *React*. V závěru budou jednotlivé metody umělé inteligence vyhodnoceny a bude posouzena využitelnost celé aplikace v rámci průmyslové praxe.

1.3 Struktura práce

Práce je strukturována následovně: Nejprve je v kapitolách 2 a 3 položen teoretický základ do vibrodiagnostiky, zpracování signálu a představení technik umělé inteligence použitých v rámci práce.

Kapitola 4 pojednává o hardwarových a softwarových technologiích objevujících se v práci. V rámci hardwaru se jedná o představení a porovnání využitých produktů firmy B&R. Softwarová část potom představí paradigma internetu věcí jako již zmíněná kontejnerizace, webové frameworky využité pro vizualizaci nebo databázové technologie pro ukládání dat.

V 5 kapitole je samotný návrh architektury systému, pojednání o tréninku modelů umělé inteligence a softwarová implementace aplikace. Poslední dvě kapitoly tedy 6 a 7 obsahují vyhodnocení natrénovaných modelů umělé inteligence a celkové vyhodnocení použitelnosti aplikace v průmyslové praxi.

2 VIBRODIAGNOSTIKA A ZPRACOVÁNÍ SIGNÁLU

2.1 Měření vibrací u rotačních strojů

V této kapitole jsou představeny vybrané pojmy technické diagnostiky a jsou zde zmíněny i typické poruchy u ložisek rotačních strojů, jejichž detekcí se práce zabývá. Dále jsou představeny techniky zpracování signálu v uvedených doménách, které jsou v práci využity. Představení celého komplexního oboru technické diagnostiky a zpracování signálů by přesáhlo rozsah projektu, dále jsou proto představeny jen vybrané techniky a pojmy využitě v práci. Není-li uvedeno jinak, zdrojem informací pro tuto kapitolu je kniha *Vibration-based condition monitoring* [1], zabývající se technickou diagnostikou a zpracováním signálu pro její potřeby.

Účelem technické diagnostiky je schopnost monitorovat a předvídat budoucí stav výrobního stroje během jeho provozu. Monitorování stavu stroje je důležitou součástí údržby podle skutečného stavu stroje (*Condition-Based Maintenance – CBM*). Historicky jsou známé tři strategie pro údržbu strojů:

1. **Provoz do vzniku poruchy** – Nejstarší strategie, při které se stroj jednoduše nechá běžet, dokud se neporouchal. Tato strategie sice nabízí nejdelší funkční běh stroje, ale při vzniku poruchy mohou být postiženy i jiné části výrobního procesu, než je samotný zdroj poruchy a může být ohroženo i zdraví obsluhy. Z těchto důvodů se od této strategie upouští.
2. **Preventivní údržba** – Základem této strategie je určení pravidelných intervalů, ve kterých je prováděna údržba. Problematické součásti jsou pak během těchto pravidelných kontrol vyměňovány. Samotné určování servisních intervalů však mívá konzervativní charakter a u řady takto udržovaných strojů proto často dochází k preventivním výměnám ještě použitelných součástí. Na zmíněný problém proto navazuje další strategie:
3. **Údržba podle skutečného stavu stroje** – Tato strategie navazuje na preventivní údržbu a snaží se za pomoci monitorování skutečného stavu stroje předpovědět jeho zbývající životnost a na základě této informace naplánovat další údržbu.

Z principu technické diagnostiky je zřejmé, že je potřeba nedestructivně získat informaci o vnitřním stavu stroje během jeho fungování. Na první pohled se to může zdát jako těžký úkol, nicméně byly vyvinuty techniky, díky kterým umíme tyto informace získat. Mezi nejznámější patří analýza vibrací a analýza maziv.

Ze zkoumání maziva stroje můžeme zjistit informaci o jeho vnitřním stavu například díky částicím opotřebení obsaženým v mazivu, nebo kontaminací maziva chemickým procesem poruchy uvnitř stroje. Existují i další metody, jako analýza výkonu stroje či využití termografie (měření teploty různých částí stroje z vnějšku pomocí termo kamer), tyto metody však zatím nejsou tolik rozšířené jako analýza vibrací, na kterou se soustředí tato práce.

Právě analýza vibrací je jedním z nejsilnějších nástrojů pro bezdemontážního stavu stroje. Stroje vydávají vibrace i během běžného bezporuchového stavu, přičemž zdrojem většiny provozních vibrací bývají periodické děje stroje, jako jsou například rotace hřídelí, či zabírající

ozubená kola a podobně. Frekvence, ve kterých se tyto vibrace opakují, bývají typicky pevně vázány na jejich zdroj, ať už jde o bezporuchový chod anebo specifické závady. Frekvenční analýza vibračního signálu z rotačních strojů pak bývá velmi silným nástrojem pro zkoumání stavu stroje, bez jeho zastavení či demontáže.

Hlavní výhodou analýzy vibrací je okamžitá a adresná reakce vibrací na vznikající problém. Zatímco z analýzy maziv můžeme zjistit přítomnost specifických materiálů z ložisek, tak v případě více stejných ložisek ve stroji nelze přímo určit které ložisko je poškozené. Naproti tomu analýzou vibrací bývá možné poruchu trasovat až do úrovně elementů konkrétního ložiska. Proto bývá analýza vibrací upřednostňována před dalšími metodami.

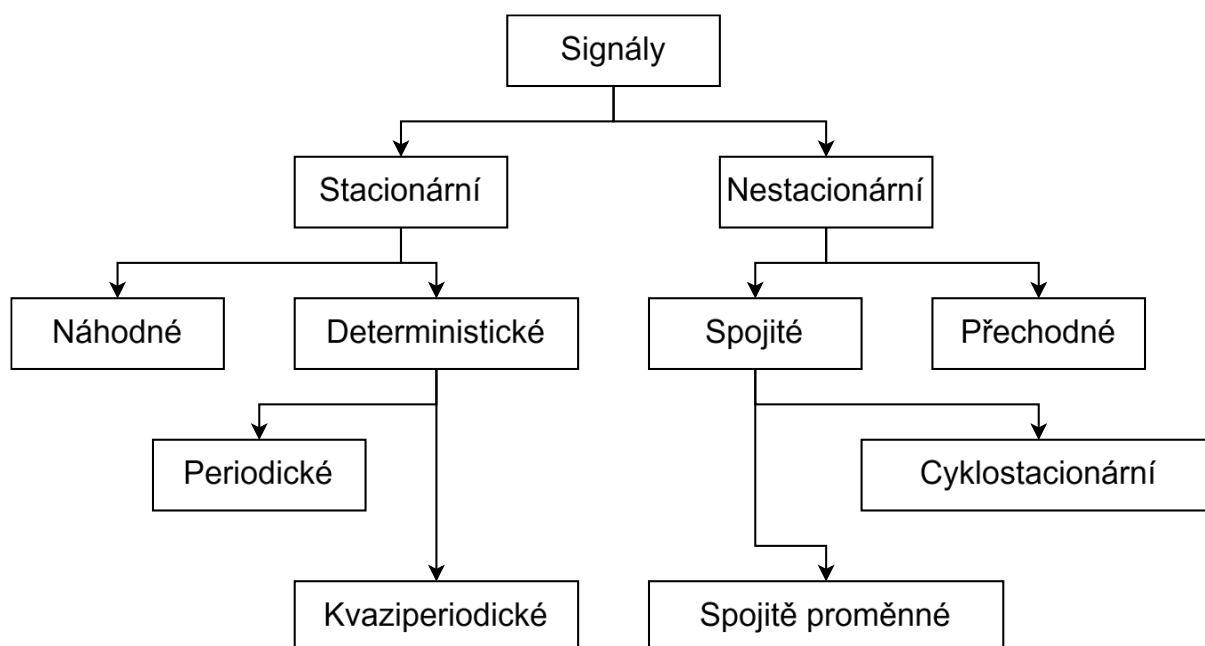
2.2 Typy signálů a jejich zpracování

Většina pohyblivých mechanických součástí produkuje charakteristické vibrační signály, které jsou pro tyto komponenty a jejich závady typické. Rozeznáváme různé druhy signálů, viz obrázek 1. Základní dělení signálů je na stacionární a nestacionární, kde stacionární signály se definují tak, že jejich statistické vlastnosti (např. rozptyl, střední hodnota) se v čase nemění, zatímco nestacionární jsou takové, u kterých jsou v průběhu času dochází ke statistickými metodami detekovatelným změnám.

Stacionární signály se rozdělují mezi náhodné a deterministické, přičemž deterministické se vyznačují tím, že je umíme exaktně matematicky popsat, náhodné jsou pak všechny ostatní. Deterministické signály ještě dělíme na periodické a kvaziperiodické. Periodické signály jsou složené z více harmonických složek a tyto složky se přesně opakují v čase za určité frekvence, která je celočíselným násobkem jedné základní frekvence signálu. U kvaziperiodických nejsou frekvence celočíselným násobkem, takže se signál jako celek nikdy přesně nezopakuje ve stejné podobě.

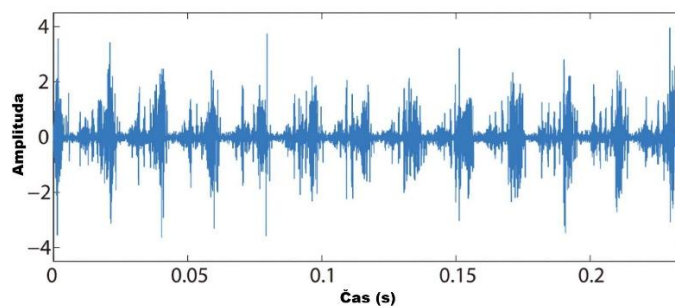
V případě nestacionárních signálů tyto dělíme na spojitě a přechodné. Spojité signály jsou takové, které trvají neomezeně dlouhou dobu a jejich struktura a parametry se neustále vyvíjí. Přechodné signály se vyznačují velmi krátkou a časově omezenou nenulovou částí (typicky jde o izolované rázy). Spojité signály ještě rozdělujeme na cyklostacionární a spojitě proměnné signály. Cyklostacionární signály jsou speciální skupina náhodných signálů, které sice mají náhodnou povahu, ale jejich statistické vlastnosti se v čase periodicky opakují. Spojitě proměnné signály se potom vyvíjí zcela nepravidelně bez jakékoliv periodicity.

V praxi získaný signál bývá vždy směsí několika výše zmíněných kategorií. Pro příklad můžeme uvést, že vyosení hřídele při konstantních otáčkách generuje čistě periodický (deterministický signál), doplněný o náhodný šum. V našem případě se ovšem snažíme o zařazení vibračního signálu do kategorie cyklostacionárních signálů. V čase se totiž periodicky opakují rázové impulsy, vznikající odvalováním elementů po miniaturních nerovnostech ložisek.



Obr. 1: Typy signálů podle [1].

Na obrázku 2 je představen typický vibrační signál (průběh amplitudy vibrací v čase). Je zřejmé, že takovýto signál nám od pohledu mnoho informací neposkytne, proto je signál třeba dále zpracovat, abychom z něj dostali informace potřebné pro další analýzu. V projektu je zpracování vibračních signálů řešeno ve třech kategoriích: zpracování v časové, frekvenční a časově-frekvenční doméně. V následujících podkapitolách budou tyto přístupy přiblíženy a bude představen jejich význam pro další práci.



Obr. 2: Příklad úseku vibračního signálu převzatý z [2].

V průmyslové vibrodiagnostice je běžnou praxí, že neanalyzujeme vysokofrekvenční signál zcela nepřetržitě, protože máme omezenou výpočetní kapacitu hardwaru a datový tok by byl obrovský. Tento problém se řeší periodickým ukládáním diskrétních datových oken (tzv. *time window*), což znamená že se v pravidelných intervalech (například každých 5 minut) zaznamenává definovaný úsek surového signálu. Délka tohoto časového úseku je pevně svázaná se vzorkovací frekvencí (f_s). Nejlépe se to vysvětluje na příkladu: Řekněme, že máme měřicí kartu nastavenou na vzorkovací frekvenci 25 kHz a náš algoritmus pracuje s bloky o velikosti 25 000 bodů. V tomto případě trvá náběr jednoho datového okna přesně 1 sekundu.

Každé takto získané časové okno potom dále zpracováváme následujícími metodami. Tento přístup nám garantuje, že všechny způsoby zpracování signálu pracují s identickým a časově synchronizovaným stavem stroje, což je nezbytný předpoklad pro aplikaci modelů strojového učení.

2.2.1 Časová doména

Zpracování signálu v časové doméně je základní a výpočetně nejlevnější metoda vibrační analýzy. Tato metoda spočívá ve výpočtu charakteristických statistických hodnot přímo z naměřeného datového okna vibrací. Často tyto jednorozměrné parametry nazýváme příznaky (*features*). Příznaky slouží jako identifikátory celkového stavu stroje či výskytu rázových anomálií vzniklých poškozením mechanických částí.

Mezi typické příznaky zařazujeme například střední kvadratickou hodnotu (*RMS – Root Mean Square*), která reflektuje celkovou energii vibrací a je vhodná pro detekci makroskopických poruch. Pro účely této práce a pro následnou extrakci příznaků pro trénink modelů strojového učení byla z velkého množství dostupných příznaků vybrána specifická množina, viz. tabulka 1. Výpočetní vzorce zvolených příznaků vychází z literatury pro zpracování vibračních signálů [1], příslušných průmyslových standardů [3] a manuálu k hardwaru B&R [4], o kterém bude řeč později. V tabulce představuje x_i i -tý vzorek diskrétního signálu, $\bar{x}$ je jeho střední hodnota a N je celkový počet vzorků v datovém okně.

Tab. 1: Vybrané příznaky pro tuto práci.

Číslo příznaku / vzorce	Název příznaku	Vzorec
(1)	RmsAccRaw	$x_{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$
(2)	KurtosisRaw	$K = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^4}{x_{RMS}^4}$
(3)	SkewnessRaw	$S = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^3}{x_{RMS}^3}$
(4)	RmsAccEnvelope	$Env_{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N E_i^2}$
(5)	Vdi3832KtRaw	$K(t) = \frac{x_{RMS}(0) \times x_{Peak}(0)}{x_{RMS}(t) \times x_{Peak}(t)}$
(6)	ActSpeed	Měřená hodnota, nepočítá se ze signálu

Volba těchto příznaků vychází z jejich schopnosti popsat různé fáze degradace valivých ložisek. Parametr efektivní hodnoty surového signálu zrychlení (*RmsAccRaw*) slouží jako indikátor celkové destrukční energie, který začíná výrazně růst až v pozdních fázích poškození. Statické momenty vyšších řádů, kterými jsou špičatost (*KurtosisRaw*) a šikmost (*SkewnessRaw*), jsou pak velmi citlivé na výskyt ojedinělých rázů v signálu, což je důležitá část diagnostiky ložisek. Díky jejich vlastnostem jsou schopny odhalit mikrotrhliny dřív, než se podepíší na celkové amplitudě vibrací. Pro izolaci těchto rázů od běžného šumu se využívá příznak efektivní hodnoty obálky signálu (*RmsAccEnvelope*). Ten hodnotí signál až po provedení filtrace a demodulace (tzv. obávkové analýzy). Bezrozměrný příznak degradace dle směrnice VDI 3832 (*Vdi3832KtRaw*) přispívá k predikci zbývající doby životnosti ložiska. Tento příznak tvoří přirozeně klesající trend s postupným opotřebením na základě poměru zdravých a aktuálních hodnot. Poslední zvolený příznak (*ActSpeed*) obsahuje kontextuální informaci o aktuálních otáčkách stroje a je důležitý pro modely strojového učení pro správné provedení normalizace dat.

2.2.2 Frekvenční doména

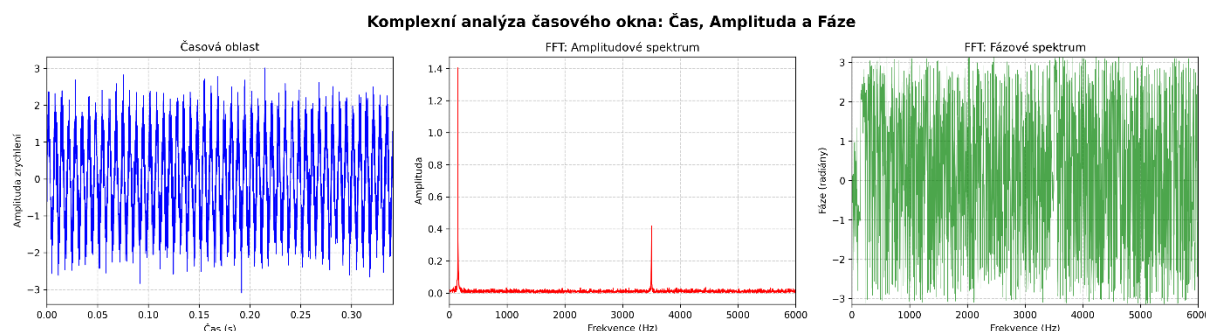
Příznaky získané z časové domény sice poskytují informaci o celkové závažnosti vibrací, avšak je složité z nich určit o jakou konkrétní poruchu ložiska se jedná (toto omezení bylo potvrzeno i přehledovým článkem [5]). Protože se práce zabývá i klasifikací poruch u ložisek, přistupujeme ke druhé metodě zpracování signálu, kterou je *analýza ve frekvenční doméně*. Princip spočívá v transformaci signálu z časového okna do frekvenční oblasti, která umožňuje rozložit složitý vibrační děj na jednotlivé harmonické složky. Bližším zkoumáním těchto složek následně zjistíme typ a původ poruchy. Každý defekt valivých ložisek (např. vada vnějšího kroužku či valivého elementu) totiž generuje vibrace ve specifických kinematických frekvencích.

Základním matematickým nástrojem pro tuto transformaci je algoritmus Fourierovy transformace (*FT*). Podle John G. [6] vyjadřuje spojitá FT v teoretické rovině původní časový signál ve formě integrálu komplexních exponenciál. Zde ovšem pracujeme s vzorkovaným signálem v diskretních časových krocích, proto používáme její diskretní podobu – Diskretní Fourierova transformace (*DFT*). DFT se počítá podle vzorce (viz rovnice 7).

$$X[k] = \sum_{n=0}^{N-1} x[n] \times e^{-j\frac{2\pi}{N}kn}, \quad (7)$$

kde N je celkový počet vzorků v datovém okně, n je index vzorku v časové doméně, k je index vzorku ve frekvenční doméně, $x[n]$ je naměřená hodnota signálu v n -tém časovém vzorku, $X[k]$ je vypočtená komplexní hodnota pro k -tou frekvenční složku a j je imaginární jednotka.

Je však běžnou praxí místo DFT použít algoritmus rychlé Fourierovy transformace (*FFT* – *Fast Fourier transform*), která představuje efektivnější způsob výpočtu DFT. Zatímco přímý výpočet DFT vyžaduje kvadratický počet operací, algoritmus FFT dramaticky snižuje tuto složitost, což umožňuje rychlejší zpracování i u větších datových polí. Reprezentace signálu pomocí FFT umožňuje oddělit poruchové frekvence od provozního šumu a přiřadit špičky dominantních frekvencí ke konkrétním vadám. Na obrázku 3 je porovnán průběh surového signálu v časové doméně a jeho odpovídající rozklad ve frekvenčním spektru.



Obr. 3: Názorná ukázka zpracování časového okna pomocí FFT.

Z DFT dostaneme vektor komplexních čísel. Každá frekvenční složka je tak plně popsána svou reálnou a imaginární částí. V rámci této práce budou ještě tyto komplexní hodnoty rozděleny na amplitudové a frekvenční spektrum. Amplitudové spektrum vyjadřuje množství energie na dané frekvenci a slouží k přímé identifikaci ložiskových vad. Fázové spektrum nese informaci o vzájemném časovém zpoždění jednotlivých harmonických složek. V průmyslové praxi se fázové spektrum často zahazuje, avšak v této práci bude využito pro lepší trénink modelů strojového učení, ale o tom později.

2.2.3 Časově-frekvenční doména

Zásadním omezením standardní frekvenční analýzy je úplná ztráta informace o čase. Z FFT nelze určit, v jakém časovém okamžiku se daná frekvenční složka v signálu objevila, což je pro diagnostiku nestacionárních a přechodových dějů (například rázů od lokálních vad ložisek) silně limitující. Toto omezení překonáváme pomocí třetí metody analýzy vibračního signálu – *zpracování v časově-frekvenční doméně*.

V rámci této práce bude pro zpracování signálu v časově-frekvenční doméně uplatněna kontinuální vlnková transformace (*CWT – Continuous Wavelet Transform*). CWT na rozdíl od FFT nevyužívá nekonečné harmonické funkce, ale lokální tzv. mateřské vlnky, které jsou lokalizované v čase i frekvenci a v průběhu výpočtu se adaptivně škálují (roztahují a smršťují).

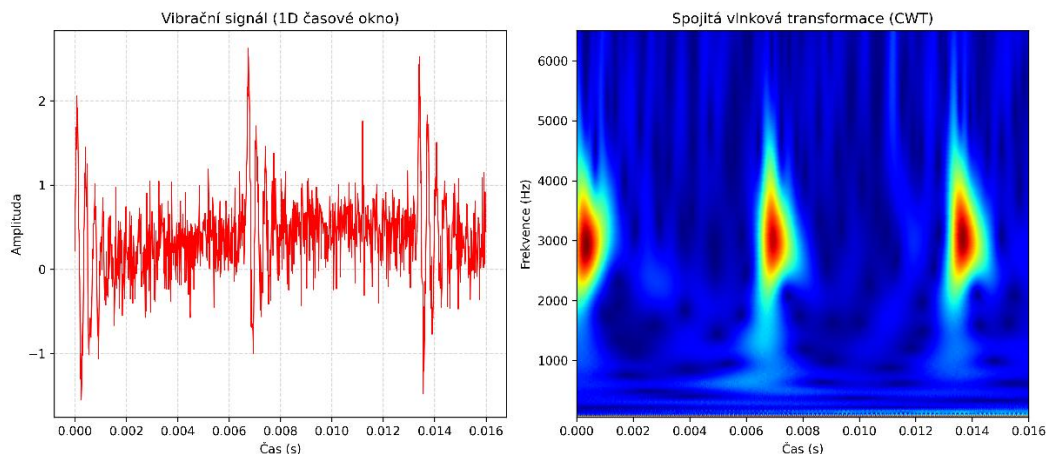
Dle [7] je CWT matematicky definována jako konvoluce analyzovaného signálu $x(t)$ s posunutou a naškálovanou verzí komplexně sdružené mateřské vlnky $\psi(t)$. Z důvodu práce s diskretním signálem je v rovnici 8 uvedena diskretní verze výpočtu CWT.

$$CWT[a, b] \approx \frac{1}{\sqrt{a}} \sum_{n=0}^{N-1} x[n] \times \psi^* \left[\frac{n-b}{a} \right], \quad (8)$$

kde a je parametr měřítka (škálování, $a > 0$). Nižší hodnoty a stlačují vlnku (pro vysoké frekvence), vyšší hodnoty ji natahují (pro nízké frekvence), b je parametr posunutí (určuje přesnou polohu vlnky v čase), $x[n]$ je naměřená hodnota signálu v n -tém časovém vzorku, $\psi[n]$ je hodnota mateřské vlnky (v diagnostice většinou Morletova vlnka) v n -tém vzorku, $\psi^*[n]$ je vzorek komplexně sdružené funkce k mateřské vlnce $\psi(t)$, $\frac{1}{\sqrt{a}}$ je normalizační konstanta, která zajišťuje zachování stejné energie vlnky pro různá měřítka.

Pomocí CWT dostaneme vizuální mapu, tzv. skalogram, která detailně zobrazuje distribuci energie signálu v obou doménách současně. Transformace přirozeně poskytuje

výborné časové rozlišení pro vysokofrekvenční jevy (krátké časové úseky) a naopak dobré frekvenční rozlišení pro nízké frekvence (dlouhé časové úseky). Díky těmto vlastnostem ji autoři Peng a Chu [8] označují jako vysoce efektivní nástroj pro detekci a extrakci velmi slabých mechanických rázů ze silně zašuměného provozního signálu. Vizuální reprezentace skalogramu získaného pomocí CWT je na obrázku 4. Tato metoda bude využita v rámci práce jako vstup modelu strojového učení pro detekci anomálií.



Obr. 4: Ukázka vlnkové transformace.

2.3 Typické poruchy valivých ložisek a jejich projevy

Valivá ložiska patří mezi nejvíce kritické mechanické součásti rotačních strojů, některé zdroje (např. [9]) uvádí, že se jedná o 40 % až 45 % všech závad rotačních strojů. Má tedy smysl se zabývat typickými poruchami ložisek a jejich projevy. Výše uvedené metody zpracování signálu slouží především k oddělení diagnostických informací od provozního šumu.

Když se valivé těleso odvaluje přes defekt v materiálu (např. trhlinu) generuje mechanický ráz. Ačkoliv má jeden takovýto ráz velmi malou energii, série těchto rázů funguje jako budící síla, která rozkmitává celé okolí ložiska. Frekvence, ve kterých k tomuto kmitání nejsou náhodné, ale podléhají fyzikálním parametrům daného ložiska. Tento fakt je všeobecně známý a každý seriózní výrobce valivých ložisek tyto informace poskytuje. Pro příklad budou v této práci uvedeny výpočty z manuálu výrobce SKF [10].

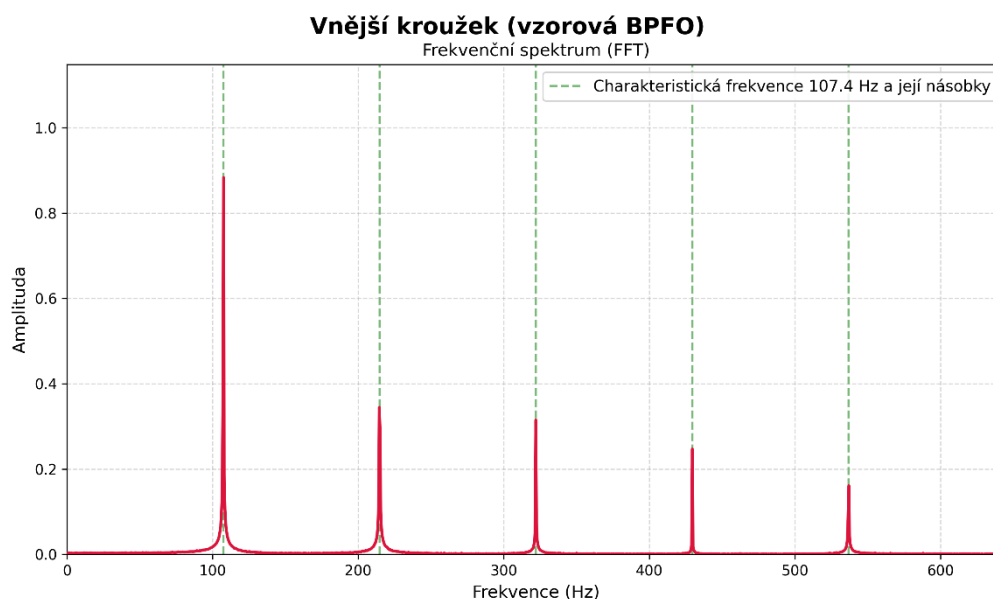
2.3.1 Porucha vnějšího kroužku

Porucha vnějšího kroužku patří mezi nejčastější defekty valivých ložisek. V drtivé většině průmyslových aplikací je vnější kroužek pevně uložen v ložiskovém domku, zatímco vnitřní kroužek rotuje s hřídelí. Z toho důvodu je zóna zatížení na vnějším kroužku stále stejná. Když se valivé elementy odvalují přes trhlinu v této zóně zatížení, generují sérii rázů o konstantní amplitudě. Z hlediska frekvenční domény je tato porucha nejsnáze detekovatelná, protože ve spektru obálky nevykazuje výraznou amplitudovou modulaci. Charakteristickou frekvencí pro tuto poruchu je frekvence přeběhu přes vadu na vnějším kroužku (*BPFO – Ball Pass Frequency of Outer ring*), která vypočítá podle rovnice 9:

$$f_{BPFO} = \frac{Z}{2} \times f_r \times \left(1 - \frac{d}{D} \cos \alpha\right), \quad (9)$$

kde z je počet valivých těles, f_r je rotační frekvence hřídele [Hz], d je průměr valivého tělesa [mm], D je průměr roztečné kružnice ložiska [mm] a α je stykový úhel [°].

Ve frekvenčním spektru se porucha vnějšího kroužku projevuje jako dominantní špička na základní frekvenci BPFO a dalších násobcích této základní harmonické složky ($2 \times BPFO$, $3 \times BPFO$ atd.). Ukázka typického amplitudového spektra obálky pro poruchu vnějšího kroužku je znázorněna na obrázku 5.



Obr. 5: Teoretický model FFT pro vadu vnějšího kroužku (BPFO).

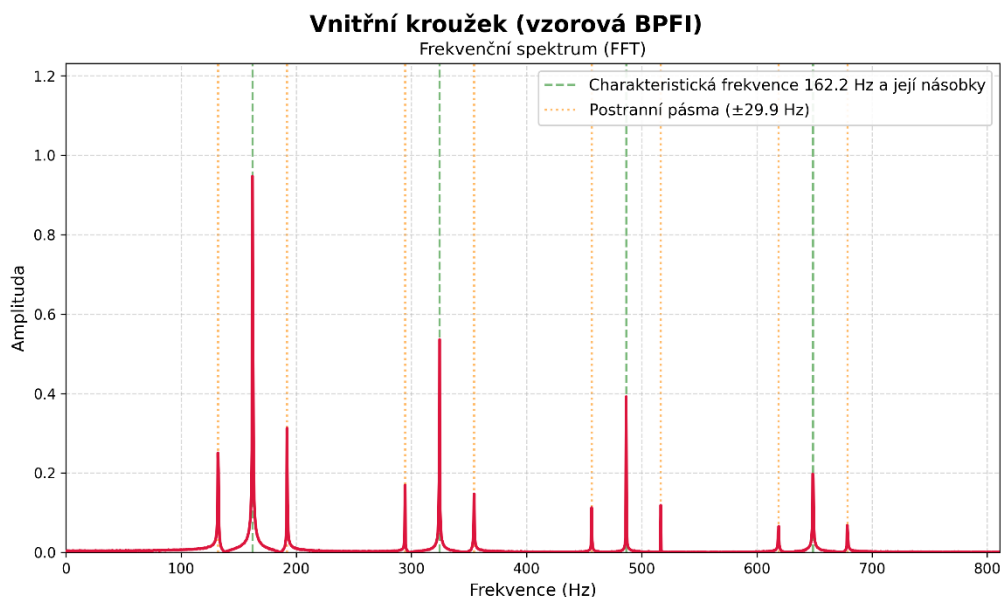
2.3.2 Porucha vnitřního kroužku

Naproti vnějšímu kroužku ten vnitřní zpravidla rotuje společně s hřídelí. Pokud se na něm nachází defekt, tento defekt periodicky vstupuje do zóny maximálního zatížení a následně z ní vystupuje. Výsledkem je série rázů s proměnlivou silou – dochází k amplitudové modulaci signálu. Charakteristickou frekvencí této poruchy je frekvence přeběhu přes vadu na vnitřním kroužku ($BPFI$ – *Ball Pass Frequency of Inner ring*), kterou lze vypočítat dle následujícího vztahu v rovnici 10:

$$f_{BPFI} = \frac{Z}{2} \times f_r \times \left(1 + \frac{d}{D} \cos \alpha\right), \quad (10)$$

kde význam jednotlivých parametrů zůstává stejný jako u rovnice 9.

Ve frekvenčním spektru se tato porucha projevuje dominantními špičkami na harmonických frekvencích BPFI, avšak kvůli zmíněné modulaci jsou tyto špičky obklopeny tzv. postranními pásmy (*Sidebands*). Vzdálenost těchto postranních pásem od centrální frekvence odpovídá přesně frekvenci otáčení hřídele f_r . Typické spektrum s postranními pásmy pro defekt vnitřního kroužku ilustruje obrázek 6.



Obr. 6: Teoretický model FFT pro vadu vnitřního kroužku (BPFI).

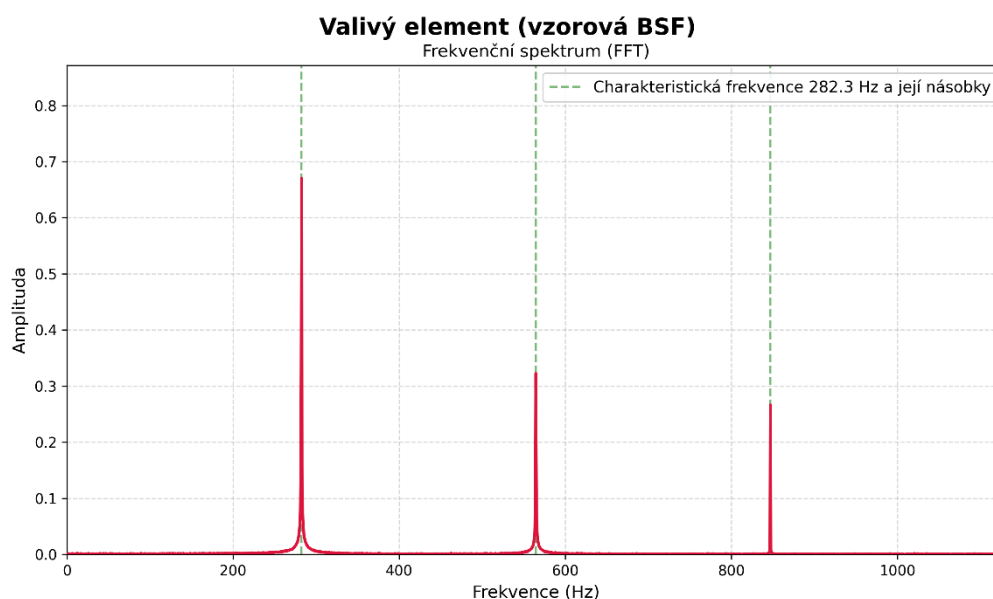
2.3.3 Porucha valivého elementu

Poruchy samotných valivých elementů (kuliček nebo válečků) bývají na diagnostiku nejsložitější. Když má kulička vadu, udeří během jedné své otáčky do vnitřního i vnějšího kroužku. Charakteristická rotační frekvence valivého tělesa (*BSF* – *Ball Spin Frequency*) se vypočte z geometrických parametrů ložiska podle rovnice 11:

$$f_{BSF} = \frac{D}{2} \times f_r \times \left(1 - \left(\frac{d}{D} \right)^2 \cos^2 \alpha \right), \quad (11)$$

kde význam parametrů je opět zachován od rovnice 9.

Diagnostika této poruchy je výrazně komplikovaná skutečností, že valivé tělísko nemusí defektem narazit na kroužek při každé otáčce, protože mění svou orientaci v prostoru. Navíc je signál modulován frekvencí klece ložiska (*FTF* – *Fundamental Train Frequency*), která valivá tělíska unáší. Ve frekvenčním spektru (obrázek 7) se tak porucha projevuje jako špičky na frekvencích $2 \times BSF$ (náráz do obou kroužků) a jejich vyšších harmonických, které jsou modulovány postranními pásmy s odstupem odpovídajícím frekvenci klece (FTF).



Obr. 7: Teoretický model FFT pro vadu valivého tělesa (BSF).

2.3.4 Další typy poruch

Kromě tří hlavních, výše popsaných, typů defektů existuje v průmyslové praxi rozsáhlé množství dalších poruch rotačních soustav. Jedná se například o poškození klece ložiska (*FTF*), problémy s mazáním vedoucí k zadírání, nevyváženost rotoru či nesouosost hřídelí.

Vzhledem ke složitosti vibrační diagnostiky není z časového a rozsahového hlediska možné postihnout v rámci jedné diplomové práce všechny známé typy těchto defektů. Experimentální a klasifikační část této práce (potažmo trénování konvoluční neuronové sítě) se proto primárně a detailně zaměřuje na výše rozebrané tři fundamentální vady valivých ložisek (vnitřní kroužek, vnější kroužek, valivý element), na kterých lze principy strojového učení exaktně demonstrovat a ověřit.

3 UMĚLÁ INTELIGENCE V PREDIKTIVNÍ ÚDRŽBĚ

Jak již bylo zmíněno v části o prediktivní údržbě, historicky se údržba strojů opírala především o dvě metody. Buďto docházelo k opravám a výměně strojních součástí až po vzniklé poruše, nebo se využívala preventivní údržba, která definuje pravidelné intervaly pro opravy strojů a výměnu jejich částí. Oba tyto přístupy jsou ekonomicky neefektivní, neboť vedou buď k neplánovaným prostojům, nebo k předčasné výměně zdravých komponent. Jak potvrzuje přehledový článek [11], nástup Průmyslu 4.0 a masivní nasazení senzoriky pro monitorování stavu (*Condition Monitoring*) umožnily přechod k údržbě založené na aktuálním stavu stroje – CBM. Tato metoda využívá historická i online data k průběžnému hodnocení technického stavu stroje a k optimalizaci termínu údržby stroje. Zlomovým okamžikem v této oblasti se stalo zapojení metod strojového učení a všeobecně algoritmů umělé inteligence, které dokážou z obrovského množství dostupných dat v rámci Průmyslu 4.0 relativně přesně extrahovat vzory degradace a určit stav stroje.

3.1 Základy strojového a hlubokého učení

Strojové učení (*ML – Machine Learning*) je podoblastí umělé inteligence, která umožňuje výpočetním modelům učit se a zlepšovat svůj výkon na základě dostupných dat. V kontextu vibrační diagnostiky se s úspěchem používá dvoufázový přístup. Nejprve se provede extrakce příznaků (*Feature Extraction*) v časové, frekvenční či časově-frekvenční doméně (viz. kapitola o zpracování signálu). Takto extrahované parametry následně slouží jako vstup pro algoritmy strojového učení. Algoritmy strojového učení tyto parametry vyhodnocují a vyvozují z nich závěry. Je tedy zřejmé, že úspěšnost celého tohoto procesu závisí nejen na kvalitě algoritmu, ale i na kvalitě a diskriminačních schopnostech zvolených parametrů.

Hluboké učení (*DL – Deep Learning*) tento limit překonává pomocí vícevrstvých neuronových sítí, které zapojují proces extrakce příznaků přímo do samotného učení (tzv. *End-to-End learning*). Jak uvádí I. Goodfellow a kol. v [12], první vrstvy sítě identifikují nízkourovňové vzory (např. lokální hrany a rázy v signálu), hlubší vrstvy potom tyto informace skládají do komplexních abstraktních reprezentací. Tento přístup je mimořádně efektivní při zpracování vibračních signálů a frekvenčních spekter, kde jsou diagnostické informace často zaneseny znatelným provozním šumem, jenž mají tradiční analytické metody problém spolehlivě odfiltrovat.

V kontextu této diplomové práce jsou výše popsané teoretické principy ML a DL přímo využity v rámci procesu prediktivní údržby rotačních strojů. Tento proces je rozdělen do tří navazujících úloh, které reflektují reálný postup průmyslové diagnostiky. Prvním krokem je detekce anomálií, která slouží k prvotní a včasné identifikaci nezvyklých provozních stavů z časově-frekvenčních dat. Druhým krokem je přesná klasifikace poruchy, jejímž cílem je za pomoci modelů s učitelem identifikovat konkrétní typ a původ vznikajícího defektu. Celý proces je následně završen odhadem zbývajících doby technického života (RUL). V tomto

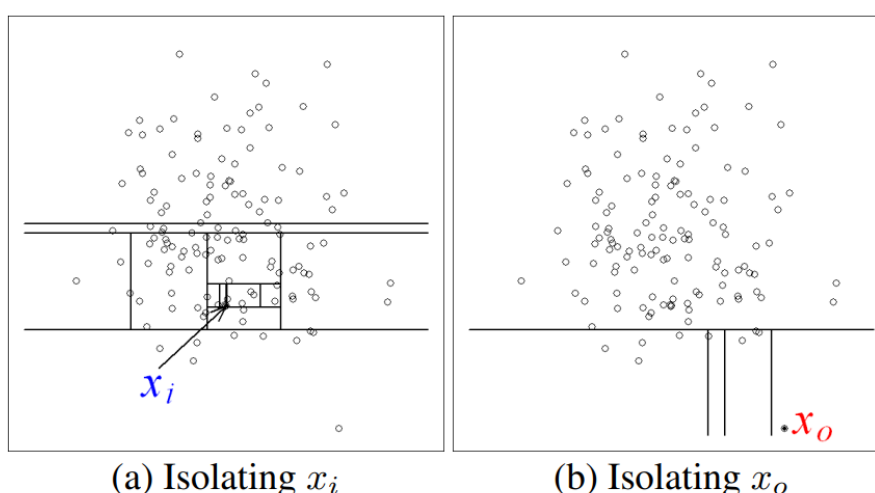
posledním kroku algoritmy umělé inteligence vyhodnocují dlouhodobý vývoj extrahovaných příznaků v čase a snaží se predikovat dobu zbývajících do okamžiku kritického selhání.

3.2 Detekce anomálií bez učitele

Detekcí anomálií se zabývá V. Chandola a kol. ve svém článku [13]. V tomto kroku je úkolem modelů strojového učení identifikovat jakékoliv odchylky od normálního stavu stroje. Nejprve budou představeny tradiční modely, které bude zastupovat *Isolation Forest*, následně složitější rekonstrukční modely, jejichž zástupcem bude *Autoencoder* a na závěr bude představena kombinace *Autoencoderu* a generativní adversiální sítě (*GAN – Generative Adversarial Network*). Tento *State-of-art* model řeší obecný problém nedostatku dat s defekty tím, že se učí pouze na zdravých datech.

3.2.1 Isolation Forest

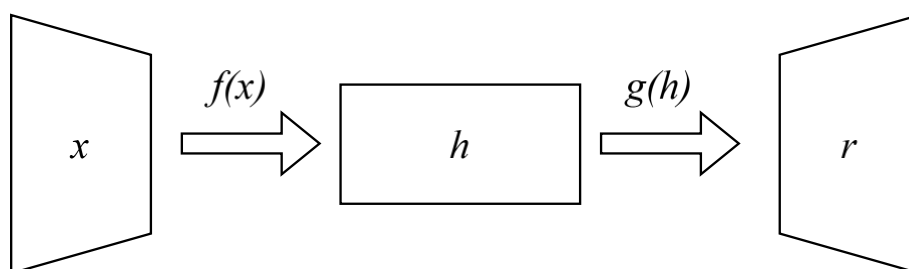
Jako tradiční a výpočtově nenáročný model pro detekci anomálií bude představen algoritmus *Isolation Forest*. Na rozdíl od jiných metod, které primárně modelují profil zdravých dat a anomálie hledají jako odlehlé body, tento algoritmus izoluje anomálie přímo. Princip této metody je graficky znázorněn na obrázku 8. Tento obrázek ilustruje dvě situace. V první (situace a) se snažíme oddělit bod x_i od ostatních dat. Jak můžeme vidět, tak tento proces je relativně náročný a potřebujeme spoustu řezů v prostoru parametrů. Naproti tomu bod anomálie x_o (v situaci b) se nachází mimo hlavní shluk dat a je tedy podstatně jednodušší jej oddělit. Pomocí těchto principů algoritmus přímo určuje anomálie v datech. Ačkoliv je tento algoritmus robustní a rychlý, jeho efektivita klesá při zpracování vysoce dimenzionálních dat, mezi které vibrační data jednoznačně spadají. Pozornost se tedy přesunula k modelům hlubokého učení, které jsou představeny dále.



Obr. 8: Princip algoritmu *Isolation Forest*, převzato z [14].

3.2.2 Autoencoder

Pro překonání limitace spojené s vysokou dimenzionalitou byla v [15] úspěšně použit algoritmus hlubokého učení *Autoencoder* (AE). Tato architektura provádí tzv. učení bez učitele (*non-supervised learning*), konkrétně se učí reprezentovat data pomocí identitní funkce. Základní struktura model se skládá ze dvou hlavních částí: kodér (*Encoder*), jehož úkolem je komprimace vstupních vysokodimenzionálních dat do latentního prostoru o nižší dimenzi a dekodéru (*Decoder*), který se z této komprimované reprezentace snaží zpětně zrekonstruovat původní vstup. V rámci detekce anomálií se AE síť trénuje na datech, reprezentující normální chod stroje, přičemž cílem algoritmu je minimalizovat rekonstrukční chybu mezi vstupem a výstupem. Pokud je do takto natrénovaného modelu následně vložen signál obsahující defekt či nezvyklý ráz, dekodér jej nedokáže z latentního prostoru přesně zrekonstruovat, protože tyto poruchové vzory nebyly součástí tréninkových dat. Velikost rekonstrukční chyby, nejčastěji měřená jako střední kvadratická chyba (*MSE – Mean Squared Error*), tak slouží jako přímý a vysoce citlivý indikátor pro určení míry anomálie. Vizuální reprezentace průchodu signálu touto architekturou je znázorněna na obrázku 9. Na tomto obrázku jsou vysokodimenzionální data x vstupem, následně je na aplikováno funkce $f(x)$ – kodér komprimuje data do latentního prostoru. Data v latentním prostoru jsou označena h . Dekodér se potom snaží pomocí jeho funkce $g(h)$ vytváří rekonstrukci původní dat r .



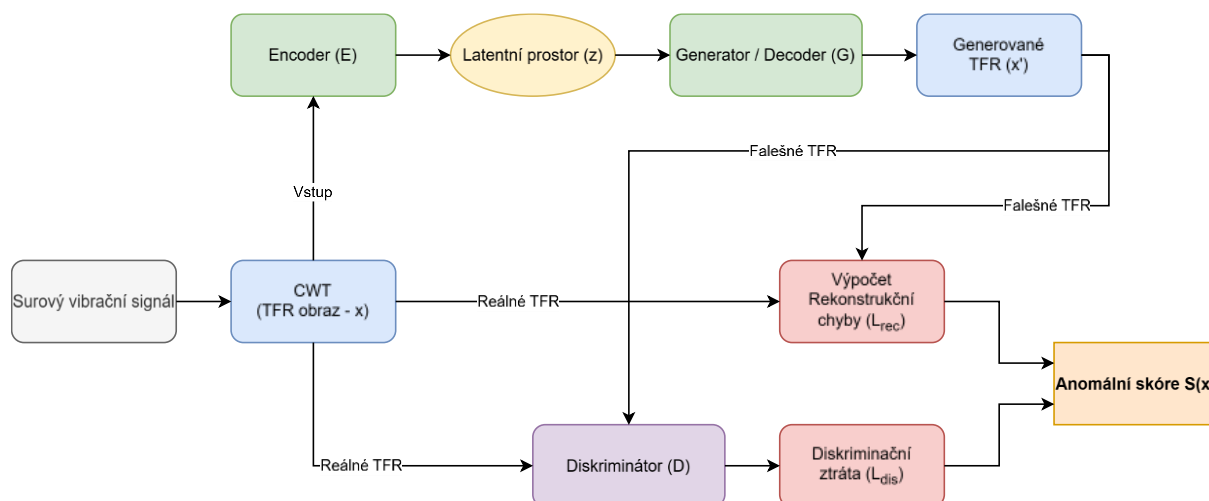
Obr. 9: Princip AE, překresleno z [12].

3.2.3 Auto-Encoder Wasserstein Generative Adversarial Network

Zatímco klasické AE se snaží minimalizovat jednoduchou rekonstrukční chybu, pokročilejší metody se snaží schopnosti AE kombinovat s dalšími metodami pro dosažení vyšší citlivosti. Pro detekci anomálií byl pro potřeby této práce po průzkumu současného stavu poznání zvolen model představený v práci [2] *Auto-Encoder Wasserstein Generative Adversarial Network* (AE-AnoWGAN).

Tento model rozšiřuje klasický AE o principy generativních adversiálních sítí (*Generative Adversarial Network – GAN*) a využívá Wassersteinovu metriku pro dosažení vyšší stability. Princip GAN spočívá v souboji dvou neuronových sítí. Na jedné straně je generátor, který se učí vytvářet co nejrealističtější falešná data. Na straně druhé se nachází diskriminátor, jehož úkolem je falešná data odhalit a rozpoznat od dat skutečných. Wassersteinova metrika tento proces vylepšuje tím, že místo prosté pravděpodobnosti (zda je vzorek pravý či falešný) měří skutečnou „vzdálenost“ mezi distribucí reálných a generovaných dat, což zajišťuje

mnohem plynulejší a stabilnější trénink, bez typických problémů s uváznutím sítě (tzv. mizející gradienty).



Obr. 10: Princip detekce anomálie pomocí AE-AnoWGAN podle [2].

AE-AnoWGAN funguje na principu paralelního tréninku kodéru, generátoru a diskriminátoru na časově-frekvenčních reprezentacích vibračního signálu. Kodér mapuje vstupní data do latentního prostoru, ze kterého se generátor snaží vytvořit co nejvěrnější rekonstrukci. Do procesu však aktivně vstupuje diskriminátor, jehož úkolem je odlišit reálná data od dat rekonstruovaných. Dekodér mapuje vstupní data do latentního prostoru, ze kterého se generátor snaží vytvořit co nejvěrnější rekonstrukci. Do procesu však aktivně vstupuje diskriminátor, jehož úkolem je odlišit reálná data od dat rekonstruovaných. Výsledkem tréninku je skóre anomálie, které kombinuje rozdíl v datovém prostoru a diskriminační hodnocení, které reflektuje, jak moc se testovaný signál odchyluje od naučených rysů normálního stavu. Pro představu je princip detekce anomálie předveden na obrázku 10. Výsledné skóre anomálie je vypočteno pomocí rovnice 12. Pro lepší pochopení viz. kapitola o implementaci a tréninku samotného modelu.

$$S(x) = L_{rec}(x, x') + L_{dis}(x, x'), \quad (12)$$

kde $S(x)$ je celkové výsledné skóre anomálie, $L_{rec}(x, x')$ je rekonstrukční ztráta, $L_{dis}(x, x')$ je diskriminační ztráta.

Hlavní výhodou modelu AE-AnoWGAN oproti běžným autoenkodérům je jeho schopnost provádět vysoce přesnou detekci i lokalizaci anomálií bez nutnosti použití jakýchkoliv označených dat o poruchách, což je v průmyslovém prostředí klíčový požadavek. Dalším významným přínosem je eliminace běžných neduhů architektur AnoGAN, jako je kolaps modu (*mode collapse*) nebo mizející gradienty.

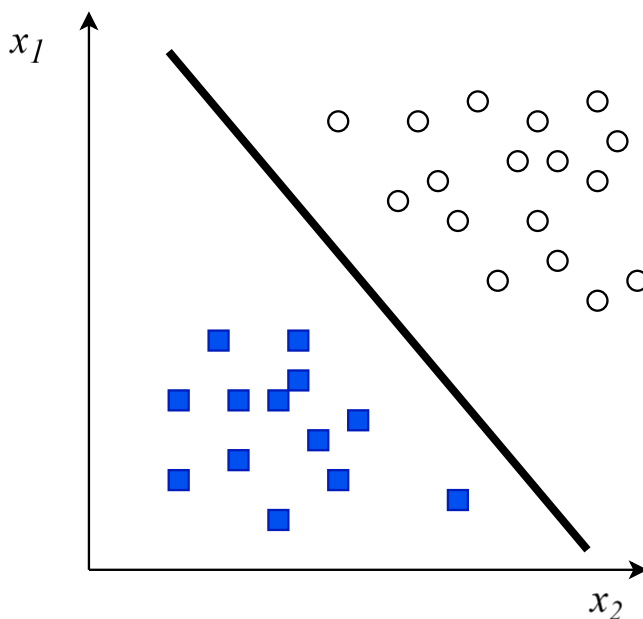
3.3 Klasifikace poruch z frekvenčních spekter

V momentě, kdy byla anomálie systémem úspěšně detekována, nastupuje fáze diagnostiky. Ta má za úkol určit o jaký typ poruchy (např. porucha vnějšího kroužku) se jedná. Tento úkol

je charakterizován jako učení s učitelem (*Supervised Learning*), při kterém jsou k tréninku použity historická data. Data musí být rozděleny do tříd (musí jim být přidělen třída – label). To je smyslem učení s učitelem: dáme algoritmu data a zároveň mu řekneme o jaká data se jedná a algoritmus se na nich učí. Jako vstup do těchto modelů se nejčastěji používají amplitudová spektra získaná FFT nebo časově-frekvenční skalogramy. Jak bylo uvedeno v sekci o zpracování signálu, každá porucha se specificky projevuje v amplitudovém spektru, je tedy možné ji rozeznat a určit její typ.

3.3.1 SVM a Random Forest

Historicky dominovaly klasifikačním úlohám tzv. mělké modely. Mělké modely strojového učení (např. SVM nebo Random Forest) jsou tradiční prediktivní algoritmy, které na rozdíl od hlubokých neuronových sítí postrádají vícenásobné skryté vrstvy. Metoda podpůrných vektorů (*Support Vector Machine – SVM*) hledá v n -rozměrném prostoru takovou hyper-rovinu, která dokáže s maximálním *marginem* (rezerva vzdálenosti) oddělit data reprezentující jednotlivé třídy (typy poruch). Na obrázku 11 je naznačeno schéma fungování SVM, kde je vidět černá hyper-rovina, která odděluje skupinu kroužků od modrých čtverečků. Pro nelineárně separabilní (rozdělitelná) data využívá SVM tzv. *kernolový trik* k transformaci do prostoru vyšší dimenze. Algoritmus SVM byl úspěšně použit ke klasifikaci poruchy ložisek jak dokazuje přehledový článek [16]. Alternativu představuje už dříve představená metoda Random Forest, která byla ke klasifikaci použita v práci [17].



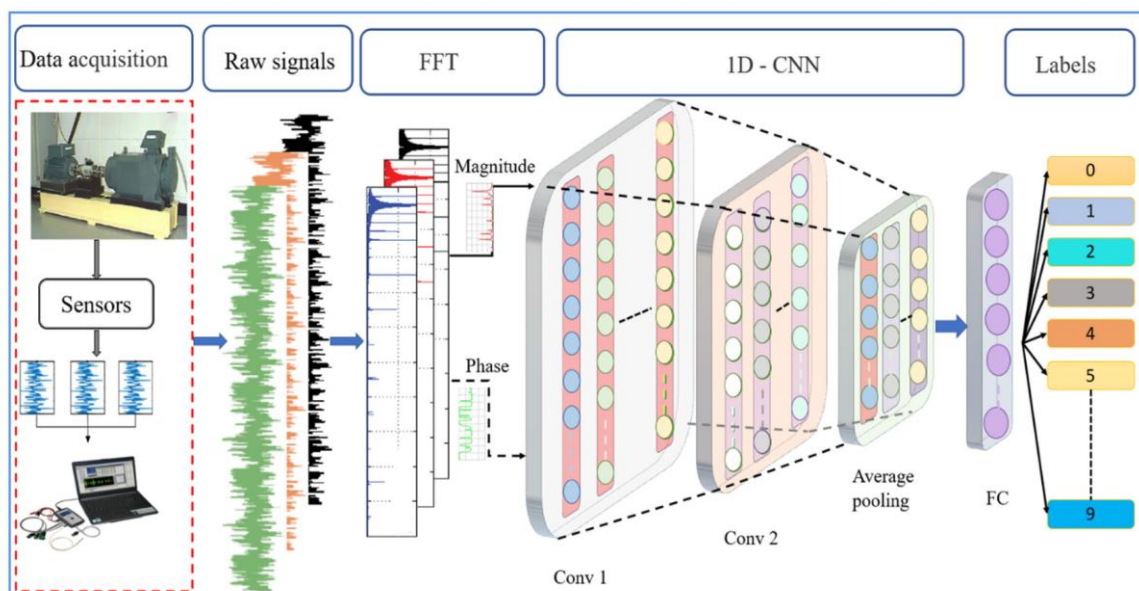
Obr. 11: Princip SVM.

Ačkoliv obě metody dosahují v průmyslu dobrých výsledků, naráží na problém prokletí dimenzionality. Pokud je jim na vstup předloženo kompletní surové FFT spektrum obsahující tisíce bodů, tak těmto metodám začíná dělat problém vyselektovat z dat užitečné informace (charakteristické harmonické špičky poruch), které tvoří pouze malou část dat.

3.3.2 Konvoluční neuronové sítě a model 1DCNNwWGN

V rámci klasifikace poruch je problém dimenzionality řešen pomocí konvolučních neuronových sítí (*Convolutional Neural Network* – CNN). Tento typ sítí byl původně vyvinut pro analýzu obrazu, ale v posledních letech prokázal mimořádnou efektivitu i při zpracování jednorozměrných signálů (1D CNN). Základním principem CNN je využití konvolučních filtrů (jader), které se posouvají nad vstupními daty a automaticky identifikují důležité lokální rysy, jako jsou rázy nebo harmonické špičky. Tento proces tzv. sdílení vah umožňuje síti detekovat diagnosticky významné vzory bez ohledu na jejich přesnou polohu ve frekvenčním spektru. Vrstvy sdružování (Pooling layers) následně snižují dimenzionalitu dat a zajišťují invarianci vůči malým posunům, čímž efektivně řeší problém vysokého počtu vstupních bodů u surového FFT spektra [12].

V této práci je pro úkol klasifikace poruch využit model 1DCNNwWGN (*1D CNN with White Gaussian Noise*), který je podle rešerše v oblasti velmi slibným a aktuálním modelem. Tento model vychází z moderních architektur optimalizovaných pro frekvenční doménu. Klíčovým prvkem tohoto „state-of-the-art“ modelu je integrace mechanismů pro zvýšení robustnosti vůči aditivnímu bílému Gaussově šumu (WGN). Zatímco běžné sítě mohou při vysoké úrovni šumu v průmyslovém prostředí vykazovat značný pokles přesnosti, architektura 1DCNNwWGN využívá techniky, kterou je trénink s injektovaným šumem nebo specifické vrstvy pro normalizaci šarží (*Batch Normalization*), aby se naučila extrahovat čisté diagnostické příznaky i z velmi zašuměného signálu. Tím model simuluje reálné podmínky průmyslového provozu, kde je užitečný signál ložiska vždy doprovázen stochastickým šumem okolních technologií.



Obr. 12: Schéma použití 1D-CNN, převzato z [18].

Hlavní výhodou modelu 1DCNNwWGN je jeho schopnost „end-to-end“ učení, která zcela eliminuje potřebu expertní a manuální extrakce příznaků. První vrstvy sítě fungují jako

adaptivní digitální filtry, které se během tréninku samy naučí odfiltrovat nezajímavá pásma a soustředit se pouze na frekvence relevantní pro konkrétní typy poruch ložisek. Schéma tohoto modelu je na obrázku 12. Díky lehké (*lightweight*) architektuře, popsané v [18], dosahuje model vysoké klasifikační přesnosti při zachování nízké výpočetní náročnosti, což jej činí ideálním pro nasazení na Edge zařízeních s omezeným výkonem. Robustnost modelu vůči různým poměrům signálu k šumu (SNR) pak zajišťuje spolehlivou diagnostiku i v případech, kdy jsou počínající vady ložisek v surovém spektru pro lidské oko nebo tradiční algoritmy neviditelné.

3.4 Odhad zbývajících životnosti

Závěrečnou a nejsložitější fází prediktivní údržby je prognostika, jejímž primárním úkolem je odhad zbývajících doby technického života (*Remaining Useful Life – RUL*). Zatímco detekce a klasifikace popisují aktuální stav stroje, modely pro predikci RUL zpracovávají časové trendy diagnostických parametrů a snaží se předpovědět okamžik, kdy míra degradace překročí kritickou prahovou hodnotu selhání.

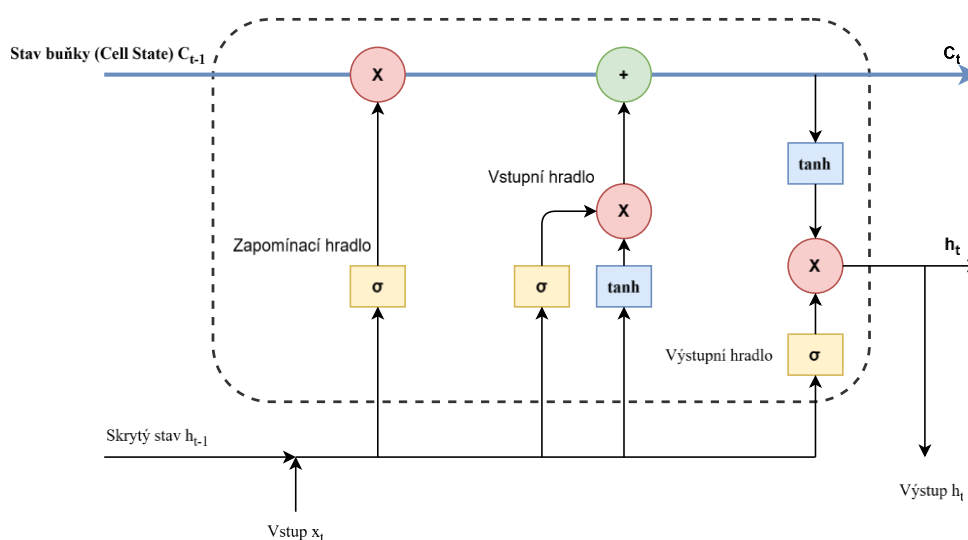
3.4.1 Analytické a regresní přístupy

Historicky základní metody prognostiky spoléhaly na matematické proložení (*curve fitting*) degradujících příznaků v časové doméně. Vybraná charakteristická hodnota (například stoupající trend RMS zrychlení nebo parametr normy VDI 3832) je aproximován lineárním, exponenciálním či polynomiálním regresním modelem, který je následně extrapolován do budoucnosti. Tento přístup vyžaduje hluboké apriorní znalosti o průběhu selhání konkrétní komponenty. Jeho hlavní slabinou je neschopnost zachytit stochastické a silně nelineární vlivy, typické pro pozdní fáze únavy materiálu a šíření trhlin.

3.4.2 Rekurentní sítě a model Bi-LSTM

Pro pokročilé modelování degradace se staly standardem modely založené na rekurentních neuronových sítích (*Recurrent Neural Network – RNN*), které jsou přímo navrženy pro analýzu sekvenčních dat a časových řad. Specificky architektura *Long Short-Term Memory (LSTM)* překonala problémy mizejícího gradientu původních RNN implementací sady řídicích hradel (zapomínací, vstupní a výstupní). Tato hradla umožňují síti naučit se dlouhodobé závislosti v datech a udržet informaci o pomalém vývoji únavy materiálu po dlouhou dobu.

Klíčovým prvkem architektury LSTM je tzv. stav buňky (*cell state*), který prochází celým řetězcem sítě jako pomyslný dopravní pás. Díky němu se informace může předávat lineárně s minimálním zkreslením. Samotný tok informací do stavu buňky je regulován zmíněnými hradly, které používají nelineární aktivační funkce (nejčastěji sigmoid a hyperbolický tangens). Zapomínací hradlo (*forget gate*) nejprve na základě aktuálního vstupu a předchozího skrytého stavu rozhodne, jaké historické informace již nejsou pro další vývoj relevantní a budou ze stavu buňky odstraněny. Následně vstupní hradlo (*input gate*) určí, jaké nové informace z aktuálního časového kroku budou do paměti přidány. Finální výstupní hradlo (*output gate*) pak definuje, jaká část z aktualizovaného stavu buňky bude předána jako skrytý stav do dalšího časového kroku vrstvy. Vnitřní struktura jedné buňky LSTM je vizualizována na obrázku 13.



Obr. 13: Schéma buňky architektury sítě LSTM.

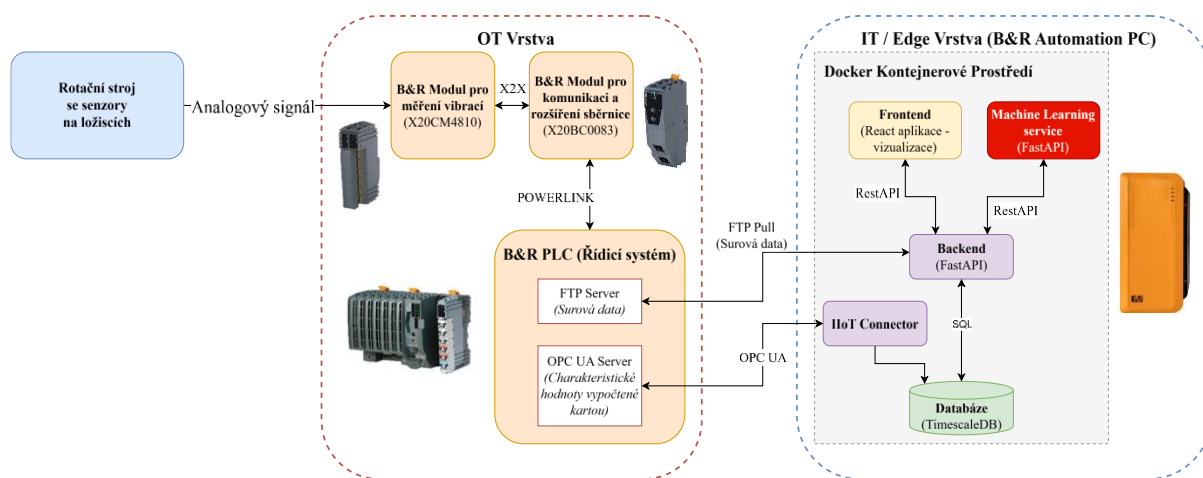
Zatímco standardní LSTM síť zpracovávají časovou řadu pouze v jednom směru (od minulosti k přítomnosti), pro účely přesnější predikce zbývajících životnosti se ukazuje jako výrazně efektivnější architektura obousměrné sítě LSTM (**Bi-LSTM**). Jak popisuje Yao a kol. v [19], Bi-LSTM využívá dvě paralelní skryté vrstvy – jednu dopřednou (*forward*) a druhou zpětnou (*backward*). To znamená, že model během trénování prochází sekvencí diagnostických dat z obou směrů současně. Tento přístup umožňuje síti zachytit mnohem hlubší kontextuální závislosti v degradaci materiálu, neboť stav stroje v daném okamžiku není vyhodnocován pouze na základě předchozího opotřebení, ale je korigován i celkovým budoucím trendem poruchy.

4 NÁVRH ARCHITEKTURY SYSTÉMU

Předchozí kapitoly definovaly teoretický rámec vibrační diagnostiky a představily metody umělé inteligence použité v této práci. Zároveň představují i jejich předchůdce či alternativy. Pro jejich úspěšné nasazení v reálném průmyslovém prostředí je však nutné navrhnout spolehlivou a datově propustnou infrastrukturu. Tato kapitola představuje návrh architektury diagnostického systému, zdůvodňuje výběr konkrétních hardwarových komponent a popisuje použité softwarové technologie a jejich ekosystém. Cílem navržené architektury je zajistit bezpečný a efektivní tok dat od fyzických senzorů na stroji až po interaktivní webovou vizualizaci s integrovanými výstupy algoritmů strojového učení.

4.1 Koncepce a topologie systému

Celková koncepce diagnostického systému je navržena v souladu s moderními principy Průmyslu 4.0 a *Edge computingu*. Aby nedocházelo k zahlcení podnikových sítí vysokofrekvenčními surovými daty, je architektura striktně rozdělena do dvou logických celků: na vrstvu provozních technologií (*Operational Technology – OT*) a vrstvu informačních technologií (*IT / Edge*), což je vidět na schéma v obrázku 14.



Obr. 14: Navržená architektura systému.

První v posloupnosti řešení je *OT* vrstva aplikace. Jejím vstupem je analogový signál, který je přijímán pomocí karty pro měření vibrací od firmy B&R. Zde probíhá prvotní zpracování a digitalizace vstupního signálu. Zmíněná karta je integrována do systému X20 (modulární rodina řídicích a I/O jednotek firmy B&R s kompaktním „plátkovým“ designem, která umožňuje flexibilní skládání rozmanitých hardwarových modulů na společnou sběrnici *X2X Link*, včetně možnosti jejich bezpečné výměny za plného provozu). Ta je připojena k modulu pro komunikaci a rozšíření sběrnice (*Bus Controller*), který slouží jako můstek mezi modulem pro měření vibrací a programovacím logickým automatem (*Programmable Logic Controller – PLC*) prostřednictvím *POWERLINK*. Modul pro měření vibrací X20CM4810 a *Bus Controller* X20BC0083 jsou přímo propojeny sběrníci *X2X Link*. Tato sběrnice je řešení firmy B&R

pro vysokorychlostní a deterministickou komunikaci mezi vstupně-výstupními (*Input/Output – IO*) moduly a jejich řídicím jednotkám. Tento můstek je využit, aby nemuselo být PLC přímo u stroje. K samotnému PLC, které se obvykle nachází v rozváděcí skříni, je potom *Bus Controller* modul připojen pomocí POWERLINK (viz. podkapitola 4.4 Komunikační protokoly).

IT/Edge vrstva následně poskytuje nezbytný výpočetní výkon pro kontejnerizovaný soubor mikroslužeb (*microservices*). Mezi tyto služby patří *backend* (řídicí logika aplikace), *frontend* (webové rozhraní pro vizualizaci), *Machine Learning service* (služba, která obstarává v aplikaci vše, co se týká strojového učení) či kontejner s databází. Tato vrstva bývá fyzicky realizována na průmyslovém PC, umístěném obvykle v kontrolní místnosti či serverovně.

4.2 Představení a výběr hardwaru

V této podkapitole bude blíže představen zvolený hardware, na kterém je aplikace realizována. Samotný výběr hardwarových komponent byl determinován dvěma faktory: technickými požadavky na vysokofrekvenční sběr dat a fyzickou dostupností konkrétních prvků v brněnské laboratoři společnosti B&R. Tato kapitola technicky specifikuje zvolené moduly, které tvoří páteř diagnostického systému.

4.2.1 Modul pro měření vibrací X20CM4810

Jako první v sérii hardwaru pro sběr dat byl zvolen modul pro měření vibrací X20CM4810. Jak je poznat z jeho označení, modul je součástí řady X20, která byla představena dříve. Při návrhu byla zvažována i varianta X20CM4800X, avšak zvolen byl modul X20CM4810. Hlavním důvodem této volby byly jeho pokročilé možnosti v oblasti interního zpracování signálu a větší flexibilita při práci se surovými daty, což je klíčové pro následnou klasifikaci pomocí hlubokého učení. Modul umožňuje připojení až čtyř IEPE senzorů (průmyslový standard pro piezo-elektrické senzory, např. akcelerometry) a disponuje integrovaným digitálním signálovým procesorem (*Digital Signal Processor – DSP*) pro výpočet spekter a statistických ukazatelů přímo v reálném čase. Samotný modul je vyobrazen v rámci schématu na obrázku 14. Výběr jeho klíčových parametrů získaných z manuálu modulu [4] je v tabulce 2 níže.

Tab. 2: Klíčové technické parametry modulu X20CM4810.

Vlastnost	Hodnota
Počet vstupních kanálů	4 (IEPE rozhraní)
Vzorkovací frekvence	až 51,2 kHz na kanál
Rozlišení A/D převodníku	24 bitů
Integrované funkce	RMS, špičková hodnota, činitel výkyvu, FFT, obálková analýza
Datové rozhraní	X2X Link
Interní paměť	Podpora pro bufferování surových dat (raw data)

4.2.2 Modul pro komunikaci a rozšíření sběrnice X20BC0083

Pro zajištění spolehlivého a deterministického přenosu dat z měřicího modulu směrem k nadřazenému řídicímu systému byl do hardwarové sestavy zařazen *Bus Controller* X20BC0083. Tento prvek plní roli klíčového komunikačního uzlu, jenž zprostředkovává obousměrný překlad datového toku z interní propojovací sběrnice X2X Link na real-time průmyslový Ethernet standardu POWERLINK. Zásadní výhodou nasazení tohoto řadiče je možnost realizace distribuované hardwarové topologie. Měřicí moduly mohou být díky němu fyzicky odděleny od hlavního procesoru a umístěny v bezprostřední blízkosti monitorovaného stroje. Tím se minimalizuje nutná délka analogových kabelů od senzorů, což radikálně snižuje riziko elektromagnetického rušení měřeného signálu. Digitalizovaná data jsou pak již bezpečně přenášena po síti do centrálního rozvaděče. Modul je opět ve schématu na obrázku 14.

Tab. 3: Klíčové technické parametry X20BC0083.

Vlastnost	Hodnota
Přenosová rychlost	100 Mbit/s
Konektivita	X2X Link, POWERLINK
Maximální délka segment	100 m mezi dvěma stanicemi
Rozhraní	2x RJ45

4.2.3 Řídicí systém X20CP3586

Pro potřeby řízení procesu sběru dat, hostování komunikačních serverů (OPC UA, FTP) a běhu stavových automatů (budou představeny v části o sběru dat) pro buffering a ukládání surových dat byl využit procesorový modul X20CP3586. Tento model byl vybrán na základě jeho fyzické dostupnosti v brněnské pobočce firmy B&R Automation. Jedná se o výkonné PLC založené na platformě Intel Atom, které poskytuje dostatečnou paměťovou kapacitu a výpočetní výkon pro asynchronní operace se soubory (zápis CSV na flash disk). Vizuální podoba PLC je opět k vidění v rámci schématu na obrázku 14 a v tabulce 3 níže jsou uvedeny klíčové technické parametry získané z manuálu pro PLC [20].

Tab. 4: Klíčové technické parametry PLC X20CP3586.

Vlastnost	Hodnota
Procesor	Intel Atom 1,0 GHz
Operační paměť (RAM)	512 MB
Interní úložiště	2 GB Flash Drive
Rozhraní	1x Ethernet, 2x USB 2.0, 1x X2X Link, 1x CAN
Cyklický čas (min.)	200 μ s

4.2.4 Průmyslový počítač Automation PC 910

Jako klíčové Edge zařízení pro IT/AI vrstvu byl zvolen průmyslový počítač APC 910. Volba tohoto modelu byla opět ovlivněna jeho fyzickou dostupností. Díky svému vysokému výpočetnímu výkonu (vícejádrový procesor) je toto zařízení schopné hostovat Docker prostředí s několika paralelně běžícími kontejnerizovanými mikroslužbami, včetně databáze TimescaleDB a modelů strojového učení, které vyžadují vyšší nároky na CPU a RAM při inferenci. Jako u předchozích kusů hardwaru je vyobrazen na schématu v obrázku 14 a v následující tabulce 4 jsou uvedeny jeho klíčové parametry.

Tab. 5: Konfigurace průmyslového APC 910.

Vlastnost	Hodnota
Typ zařízení	Box PC (průmyslové provedení)
Procesor	Intel Core i7-3615 (2.3 GHz)
Úložiště	Toshiba SSD 512 GB
Paměť RAM	16 GB
Operační systém	Debian GNU/Linux 12
Výhoda	Robustní konstrukce pro průmyslové prostředí

4.3 Software a kontejnerizace

Moderní průmyslová diagnostika vyžaduje nejen pokročilé algoritmy, ale také flexibilní softwarovou infrastrukturu, která je schopna tyto algoritmy efektivně spouštět a provozovat v reálném čase. V kontextu průmyslového internetu věcí (*Industrial Internet of Things – IIoT*) hraje klíčovou roli koncept *Edge computingu*, který představuje přesun výpočetní kapacity z centrálních cloudových úložišť směrem k okraji sítě, tedy do blízkosti zdroje dat. Jak uvádí [21], hlavní motivací pro nasazení Edge zařízení v průmyslu je radikální snížení latence, úspora šířky pásma a schopnost lokálního rozhodování bez nutnosti konektivity k veřejnému internetu. Pro aplikace vibrodiagnostiky je tento přístup nezbytný, neboť surové signály vzorkované na vysokých frekvencích představují obrovské datové objemy, jejichž kontinuální přenos do cloudu by byl neekonomický a technicky neefektivní. Edge zařízení (v této práci B&R APC) funguje jako inteligentní brána, která data lokálně předzpracovává a provádí inferenci modelů strojového učení s okamžitou odezvou.

Pro zajištění konzistentního běhu aplikací napříč vývojovým a produkčním prostředím byla zvolena technologie kontejnerizace realizovaný pomocí platformy Docker. Tato služba umožňuje izolovat aplikace a jejich závislosti do lehkých, přenosných balíčků – kontejnerů. Na rozdíl od tradiční virtualizace, která vyžaduje plnohodnotný hostující operační systém, kontejnery sdílí jádro hostitelského systému, což vede k výrazně nižším nárokům na systémové zdroje. Podle dokumentace [22] a odborných studií [23] je hlavní výhodou Dockeru v průmyslovém nasazení tzv. imutabilita infrastruktury. To znamená, že například modul strojového učení vyvinutý v prostředí Pythonu se všemi specifickými verzemi knihoven

(TensorFlow, PyTorch) poběží na všech průmyslových Edge zařízeních naprosto identicky, bez nutnosti manuální instalace závislostí na cílový hardware.

Softwarový návrh celého systému se odklání od monolitických (vše v jednom kódu) architektur směrem k implementaci pomocí mikroslužeb (*Microservices*). V tomto programovacím paradigmatu je diagnostický systém rozdělen na několik nezávislých jednotek (kontejnerů), jako jsou Backend, Frontend, ML_service a Databáze, které spolu komunikují skrze definovaná rozhraní REST API (viz. další podkapitola o komunikaci).

Tento přístup byl zvolen především kvůli modularitě a škálovatelnosti. Jak zdůrazňuje Newman [24], architektura mikroslužeb umožňuje nezávislé nasazování a aktualizaci jednotlivých částí systému bez ovlivnění běhu zbytku aplikací. V praxi to znamená, že například modul strojového učení může být aktualizován o nově natrénované váhy modelu nebo může proběhnout jeho kompletní rekonfigurace (*fine-tuning*), aniž by došlo k přerušení sběru dat *backendem* anebo výpadku vizualizačního rozhraní pro uživatele. Tato izolace chyb také zvyšuje celkovou robustnost systému, což je velká výhoda v kritických průmyslových aplikacích.

Samotná definice a sestavení každého kontejneru se provádí pomocí konfiguračních souborů nazývaných *Dockerfile*. Tento textový dokument obsahuje přesnou sekvenci instrukcí pro vytvoření obrazu (*image*) dané mikroslužby – od výběru výchozího operačního systému, přes instalaci požadovaných systémových i aplikačních závislostí, až po definici vstupního bodu (*entrypoint*) pro spuštění samotného softwaru. Způsob, jakým jsou tyto soubory strukturovány pro potřeby backendu, frontendu či modulu umělé inteligence, bude detailněji rozebrán v následujících kapitolách při popisu konkrétní implementace jednotlivých částí systému.

Vzhledem k celkovému rozsahu a softwarové komplexnosti navrženého řešení nejsou kompletní zdrojové kódy součástí hlavního textu práce. Nejdůležitější algoritmy a konfigurační úryvky, na které je v textu přímo odkazováno, jsou obsaženy v textových přílohách. Veškerý zdrojový kód celého systému je navíc verzován a v plném znění volně přístupný v online repozitáři na platformě GitHub na adrese: <https://github.com/MatyGono/Vibro-diag-system>. Čtenář si tak může detailně prohlédnout implementaci jednotlivých mikroslužeb, jejich vzájemné propojení i zmíněné konfigurační soubory *Dockerfiles*. Obrazy kontejnerů s jednotlivými mikroslužbami jsou také k dispozici ke stažení z docker hubu: <https://hub.docker.com/repositories/matejgonec>.

4.4 Komunikační prostředky a protokoly

Pro bezproblémový chod distribuovaného diagnostického systému je klíčová spolehlivá a bezpečná výměna dat mezi jeho jednotlivými hardwarovými i softwarovými komponentami. Tato podkapitola definuje čtyři hlavní komunikační standardy, které jsou v rámci navržené architektury využity.

4.4.1 Virtuální síť prostředí Docker

Kontejnerizační platforma Docker disponuje vlastním mechanismem pro správu sítí (tzv. *Software-Defined Networking*). Ve výchozím stavu jsou jednotlivé kontejnery od sebe síťově izolovány. Vytvořením uživatelsky definované virtuální sítě (typicky typu *bridge*) lze kontejnery propojit do jednoho uzavřeného subsystému. Zásadní výhodou tohoto přístupu je integrovaný DNS (*Domain Name System*) server platformy Docker, který umožňuje kontejnerům komunikovat mezi sebou pomocí jejich jmen (např. *database* nebo *backend*) namísto pevných IP adres, což výrazně usnadňuje nasazení a škálovatelnost celého řešení.

V rámci navrhovaného diagnostického systému jsou do jedné sdílené virtuální sítě zapojeny všechny čtyři hlavní mikroslužby (TimescaleDB, Backend, ML_service a Frontend). Toto řešení zaručuje, že databázový a inferenční kontejner nejsou vůbec vystaveny vnější průmyslové síti podniku, čímž je minimalizováno bezpečnostní riziko. Veškerý datový tok mezi databází a modely strojového učení probíhá výhradně uvnitř tohoto virtuálního mostu na úrovni paměti průmyslového PC (B&R APC).

4.4.2 REST API

Architektura REST (*Representational State Transfer*) představuje defacto standard pro návrh aplikačních rozhraní (*Application Programming Interface – API*) v moderním softwarovém inženýrství. Komunikace probíhá bezstavově na bázi protokolu HTTP, přičemž klient odesílá požadavky pomocí standardních metod (*GET* pro čtení, *POST* pro zápis, *PUT* pro aktualizaci) na specifické koncové body (tzv. *endpoints*). Data jsou nejčastěji přenášena v odlehčeném a strojově i lidsky čitelném formátu JSON (*JavaScript Object Notation*).

V této práci slouží REST API jako primární komunikační nástroj mezi izolovanými mikroslužbami. Backendová služba vyvinutá ve frameworku FastAPI (viz. implementace backendu) vystavuje koncové body, na které se dotazuje Frontend (React) pro získání historických trendů a výsledků predikcí k vizualizaci. Stejným způsobem Backend asynchronně odesílá požadavky (např. surová data k analýze) na dedikované REST API kontejneru ML_service, který následně navrací inferenční výsledky z modelů (RUL, klasifikace poruchy).

4.4.3 OPC UA

Protokol OPC UA (*Open Platform Communications Unified Architecture*) je nezávislý průmyslový komunikační standard určený pro spolehlivou a bezpečnou výměnu dat v systémech průmyslové automatizace a je považován za jeden z pilířů Průmyslu 4.0. Oproti starším standardům neposkytuje pouze surové hodnoty s adresami, ale umožňuje strukturovat data do komplexních, objektově orientovaných informačních modelů, které nesou kromě samotné hodnoty i sémantický význam (metadata, časová razítka, datové typy).

V architektuře představeného řešení funguje OPC UA jako hlavní most mezi provozní (OT) a informační (IT) vrstvou. Řídicí systém B&R X20CP3586 provozuje vlastní OPC UA server, do jehož informačního modelu jsou mapovány agregované diagnostické proměnné z vibrační karty (viz. tabulka 1). Zároveň tento server publikuje řídicí proměnné (triggery) pro stavový automat, což umožňuje backendu flexibilně zahajovat a monitorovat proces

ukládání surových dat a z nich vzniklých datových oken pro frekvenční nebo časově-frekvenční analýzu.

4.4.4 FTP

Protokol FTP (*File Transfer Protocol*) je tradiční, robustní síťový standard pracující na architektuře klient-server, který je dedikován výhradně pro přenos počítačových souborů. Zatímco moderní protokoly jako MQTT nebo OPC UA excelují v přenosu krátkých telemetrických zpráv s nízkou latencí, FTP je optimalizováno pro spolehlivý přenos rozsáhlých datových bloků.

Vzhledem k vysoké vzorkovací frekvenci (desítky kHz) vibrační karty jsou ucelená datová okna (tzv. surová data) příliš objemná pro plynulý stream přes OPC UA. Z tohoto důvodu je procesorová jednotka PLC vybavena lokálním FTP serverem. S tímto serverem backend navazuje spojení přes FTP a tento rozsáhlý soubor jednorázově a asynchronně stáhne (metoda *FTP pull*), čímž je zaručeno, že objemné datové přenosy pro potřeby hlubokého strojového učení nijak neovlivní řídicí síť.

4.4.5 POWERLINK

Protokol Ethernet POWERLINK je otevřený standard průmyslového Ethernetu určený pro vysoce deterministický přenos dat v reálném čase. Na rozdíl od běžného kancelářského Ethernetu, který nezaručuje exaktní časování doručení paketů a je náchylný ke kolizím (CSMA/CD), využívá POWERLINK striktně cyklický mechanismus řízení přístupu na síť. Komunikace je centrálně řízena jedním hlavním uzlem (*Managing Node*), který exkluzivně přiděluje vysílací práva uzlům podřízeným (*Controlled Nodes*). Tímto přístupem je dosaženo mikrosekundové synchronizace a garantované doby odezvy, což je nezbytné pro kritické úlohy v průmyslové automatizaci, přičemž fyzická vrstva sítě stále plně využívá standardních a cenově dostupných ethernetových komponent podle normy IEEE 802.3.

V architektuře představeného řešení figuruje POWERLINK jako primární páteřní sběrnice OT vrstvy pro deterministickou komunikaci mezi výpočetním jádrem a distribuovanými periferiemi. PLC zde plní roli hlavního uzlu (*Managing Node*), který v přesně definovaných taktech komunikuje se sběrnicevým řadičem X20BC0083 (*Controlled Node*). Přes tento komunikační kanál je obousměrně přenášen konfigurační datový tok pro měřicí modul X20CM4810 a s vysokou periodicitou jsou stahována čerstvá předzpracovaná data (agregované hodnoty vibrací a stavové bity). Výkonnost a determinismus sítě POWERLINK zaručují, že vysokorychlostní sběr dat z vibrační karty nebude ovlivněn žádným nepředvídatelným síťovým zpožděním a plynule naváže na proces ukládání v paměti PLC.

5 SBĚR, ZPRACOVÁNÍ A ARCHIVACE DAT

Získávání a efektivní správa dat představují jádro každého moderního systému pro prediktivní údržbu. V oblasti vibrodiagnostiky, kde jsou fyzikální děje vzorkovány na frekvencích v řádech desítek kilohertzů, vznikají extrémní nároky na datovou propustnost, synchronizaci a ukládání. Úspěšnost a přesnost následné klasifikace pomocí modelů strojového učení je totiž přímo závislá na kvalitě, čistotě a integritě těchto vstupních dat. Tato kapitola detailně popisuje kompletní životní cyklus dat v této aplikaci – od jejich prvotního nasnímání průmyslovým hardwarem na hraně sítě až po jejich transformaci a trvalou archivaci v databázovém úložišti.

Pro splnění protichůdných požadavků na real-time monitoring stroje a zároveň dostupnost surových signálů pro hlubokou off-line analýzu byla navržena unikátní dvoucestná architektura datových toků.

První část této kapitoly se věnuje právě tomuto rozdělenému sběru: na jedné straně kontinuálnímu přenosu agregovaných diagnostických metrik přes protokol OPC UA a IIoT Connector, na straně druhé asynchronnímu stahování surových dat, které je řízeno stavovými automaty v PLC a asynchronně odbavováno backendem pomocí metody FTP Pull.

Následující podkapitoly přibližují softwarové zpracování těchto zachycených signálů v rámci dedikované mikroslužby, zahrnující programový výpočet statistických ukazatelů, FFT či CWT. Závěr kapitoly je pak věnován struktuře relační časové databáze a návrhu jejího entitně-relačního modelu (*ERD Diagram*), jenž zajišťuje efektivní archivaci pro potřeby klientské vizualizace i budoucího trénování umělé inteligence.

5.1 Sběr dat

Do přímého kontaktu se strojem přichází nejprve senzory. Samotné senzory nejsou předmětem této práce, nicméně jsou nedílnou součástí celé aplikace, tudíž bylo uznáno za vhodné jim věnovat alespoň jeden odstavec v této části práce o sběru dat.

Současným standardem v průmyslu jsou piezoelektrické senzory, které generují kontinuální datový signál. Konkrétně byly v rámci testovacího zapojení využity fyzicky dostupné senzory **0ACS100A.00-1** na pracovišti B&R. Tyto průmyslové akcelerometry se vyznačují nominální citlivostí 100 mV/g a užitečným měřicím rozsahem ± 50 g. Z hlediska frekvenční odezvy jsou schopny spolehlivě snímat mechanické kmity v pásmu od 2 Hz do 10 kHz (s odchylkou ± 5 %), přičemž jejich přirozená rezonanční frekvence v nainstalovaném stavu dosahuje nominální hodnoty 30 kHz. Tento frekvenční rozsah tak striktně definuje fyzikální hranici užitečného signálu, který z monitorovaného objektu do diagnostického systému vstupuje.

Tyto snímače jsou bezprostředně navázány (připojeny) na modul pro měření vibrací X20CM4810, jehož primárním úkolem je převést tento spojitý analogový napěťový signál do digitální podoby. V této fázi je nutné exaktně rozlišovat mezi výše zmíněným frekvenčním rozsahem senzoru a vzorkovací frekvencí měřicí karty. Zatímco senzor je limitován svými mechanickými vlastnostmi, A/D převodník uvnitř modulu definuje, jak často bude přicházet

signál digitálně odečítán. Modul X20CM4810 disponuje maximální vzorkovací frekvencí až 51,2 kHz na kanál. V souladu se Shannon-Nyquistovým teorémem, který udává, že pro věrnou rekonstrukci frekvenčního spektra musí být vzorkovací frekvence minimálně dvojnásobná oproti nejvyšší sledované frekvenci (v našem případě 10 kHz u senzoru), poskytuje tato karta dostatečnou rezervu pro plné využití potenciálu hardwaru bez rizika zkreslení signálu (tzv. *aliasingu*). Zvolená vzorkovací frekvence měřicího modulu je pak tím nejzásadnějším parametrem, od kterého se přímo odvíjí velikost a délka datového okna (*bufferu*) ukládaného do paměti PLC pro následnou analýzu.

Vzhledem k distribuované architektuře systému je měřicí modul X20CM4810 fyzicky propojen se sběrniceovým řadičem X20BC0083 prostřednictvím interní sběrnice X2X Link. Tento *Bus Controller* plní roli komunikační brány, která zajišťuje pomocí POWERLINK deterministický a vysokorychlostní přenos digitalizovaných dat do PLC X20CP3586. Toto hardwarové uspořádání umožňuje umístit moduly pro sběr dat do bezprostřední blízkosti monitorovaného stroje, čímž se minimalizuje délka analogových přívodů od senzorů, zatímco orchestrace datových toků probíhají na straně výkonného PLC.

Z hlediska logiky zpracování byl navržen dvoucestný systém sběru dat, který efektivně balancuje mezi požadavky na včasnou diagnostiku a nároky na síťovou infrastrukturu i kapacitu úložiště. První datová cesta je určena pro kontinuální monitorování trendů a vyhodnocování celkového stavu stroje v reálném čase. V krátkých, typicky pětiminutových intervalech jsou stahovány agregované charakteristické hodnoty (tzv. příznaky), které poskytují okamžitou informaci o stabilitě provozu. Druhá cesta slouží pro hlubokou analýzu a inferenci modelů strojového učení. V delších časových oknech (např. jednou za 4 hodiny) je iniciováno vyčtení určitého časového okna surového signálu, což umožňuje provádět náročné matematické transformace a identifikovat specifické typy poruch, které z pouhých statistických trendů nejsou patrné.

5.1.1 Metoda kontinuálního sběru charakteristických hodnot

První datová cesta je určena pro kontinuální monitorování trendů a vyhodnocování celkového stavu stroje v reálném čase. Přenášet surový vibrační signál neustále po síti by vedlo k jejímu okamžitému zahlcení. Z tohoto důvodu systém plně využívá výpočetní kapacity integrovaného DSP (*Digital Signal Processor*) procesoru, kterým je karta X20CM4810 hardwarově vybavena.

Pro automatizovaný a bezpečný přenos těchto dat do centrální databáze TimescaleDB byl zvolen B&R IIoT Connector. Jedná se o kontejnerizovanou *stand-alone* aplikaci, která slouží jako robustní most mezi průmyslovými protokoly a IT světem, aniž by bylo nutné programovat vlastní komunikační skripty. Nastavení tohoto nástroje sestává ze tří klíčových kroků:

1. **Definice zdroje (Asset):** Namapování proměnných v Automation Studio projektu a jejich zpřístupnění na OPC UA serveru zmíněného PLC. Následuje připojení samotného PLC do aplikace a nastavení přístupu na OPC UA server v rámci samotné aplikace.
2. **Definice cíle (Data Sink):** Navázání spojení s instancí relační databáze TimescaleDB.
3. **Konfigurace toku (Dataflow):** Nastavení plánovače (CRON), který definuje, jak často (v této práci každých 5 minut) se mají hodnoty z PLC vyčíst a odeslat. Samotný *dataflow* v prostředí IIoT Connectoru je na obrázku 15 níže. Na obrázku můžeme na levé straně

vidět menu, které obsahuje již dříve zmíněné položky pro připojení zařízení (*Assets*) a konektory (*Connectors*) pro připojení IT světa jako kupříkladu databáze, MQTT apod.



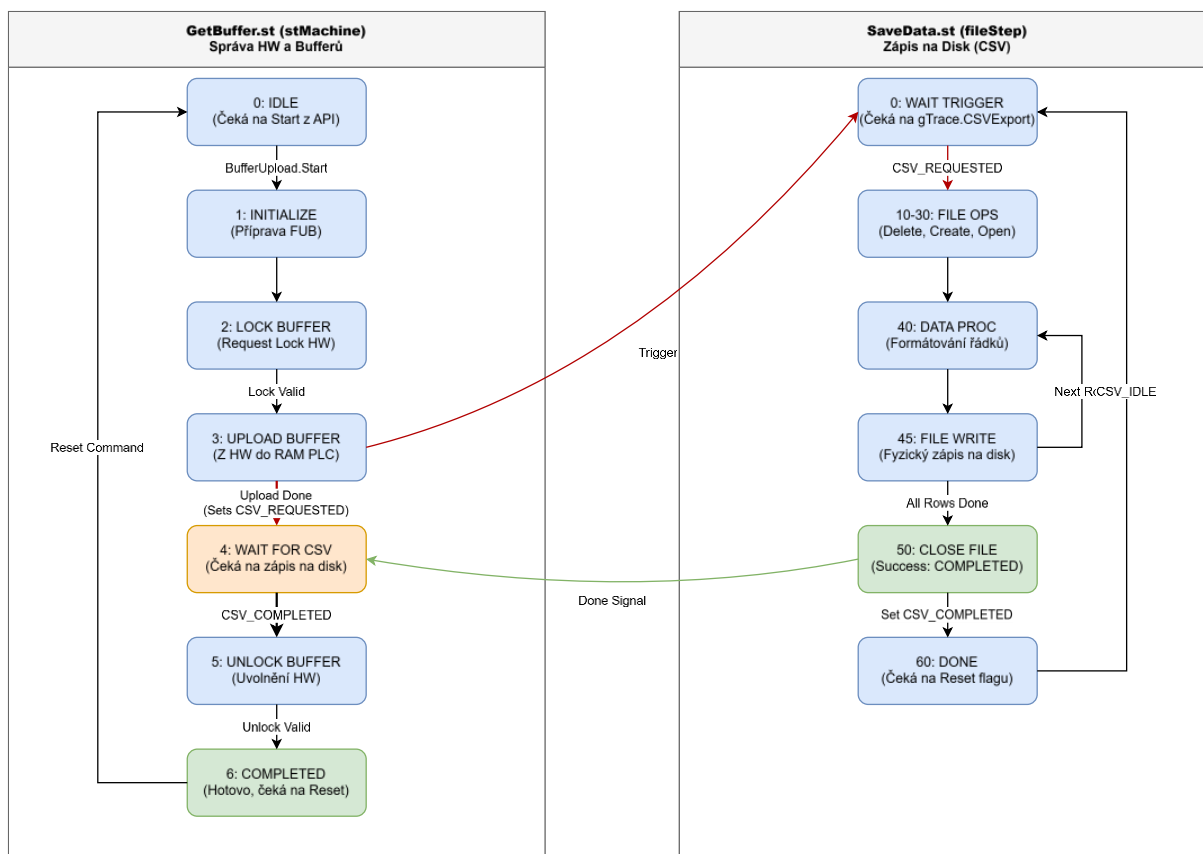
Obr. 15: Konfigurace dataflow v IIoT Connectoru.

Nepříjemným zjištěním při implementaci bylo, že specifikem IIoT Connectoru je odesílání dat do databáze ve strukturovaném JSON formátu. Samotné rozbalení tohoto datového balíčku a přesné rozřídění hodnot k příslušným senzorům je proto řešeno až na straně databáze pomocí přestupní (*buffer*) tabulky a automatického spouštěče (*triggeru*), což je detailněji popsáno v podkapitole 5.3 věnované databázové architektuře.

5.1.2 Metoda periodického sběru surových dat

Zatímco příznaky slouží k monitorování současného stavu stroje a predikci zbývajících životnosti, pro hlubokou analýzu a inferenci modelů strojového učení (např. identifikaci konkrétního poškození ložiska) je nezbytný přístup k surovému signálu. Sběr těchto surových (*RAW*) dat představuje výpočetně i komunikačně nejnáročnější operaci celého systému pro sběr dat.

Sběr surových dat probíhá v delších časových oknech a z hlediska konfigurace karty je definován přesným časovým oknem. Vzorkovací frekvence karty pro sběr dat byla nastavena na 12,8 kHz a velikost bufferu je 8192 diskretních hodnot, z čehož vyplývá, že každý exportovaný záznam představuje časové okno o délce zhruba 0.64 s. Celý tento proces je ovládán dvěma stavovými automaty, které jsou představeny na další straně.



Obr. 16: Diagram funkčnosti stavových automatů na PLC.

Jak ilustruje diagram na obrázku 16, proces probíhá v následujících krocích:

1. **Buffer Lock (Zmrazení dat):** První stavový automat odešle kartě příkaz k uzamčení bufferu. Karta dokončí zápis aktuálního datového okna (0.64 sekund / 8192 hodnot) do své interní paměti a potvrdí platnost dat příznakem. Tím se zajistí, že během vyčítání nebudou data přepsána nově přichozím signálem. Zde je nutno zmínit, že celý tento proces neblokuje funkčnost modulu a ten tak běží paralelně naprosto bez omezení.
2. **Přesun do PLC:** Druhý stavový automat asynchronně vyčte toto pole dat z karty do vyrovnávací paměti (RAM) v PLC. Následně je zámek na kartě uvolněn.
3. **Uložení do souboru:** Vyčtené pole v operační paměti PLC je iterativně zapsáno do standardizovaného formátu CSV, který se uloží na lokální uživatelskou flash paměť procesoru.

Samotné stažení těchto objemných CSV souborů je následně řízeno backendem aplikace, který ovládá stavové automaty skrze OPC UA proměnné (gTrace v projektu Automation studia). Samotné stavové automaty GetBuffer a SaveData jsou naprogramovány pomocí jazyka Structured Text (ST) a jejich kódy jsou k nalazení v příloženém AS projektu. Backend stahuje vytvořené CSV soubory prostřednictvím zabezpečeného FTP spojení (metoda FTP Pull). Vzhledem k omezené kapacitě úložného prostoru na PLC je kriticky důležité, aby backend bezprostředně po úspěšném stažení zajistil smazání zdrojových csv souborů z PLC a zapsal referenční cestu do databáze. Tato logika – řízení, stahování a ukládání je předmětem kapitoly popisující Backend aplikace.

5.2 Zpracování dat

Charakteristické hodnoty z první cesty jsou zapsány do databáze a tím jsou připraveny k použití. Nicméně surová data ze získaných csv souborů je nutno zpracovat před dalším využitím. Samotné zpracování dat není v navržené architektuře součástí backendu, ale je zakomponováno do modulu strojového učení. Tato služba je primárně určena pro běh modelů strojového učení, k čemuž se logicky pojí i zpracování dat. Naopak v textu je představeno zpracování dat už v této části, abychom zachovali chronologickou cestu dat. Proces zpracování je zahájen ve chvíli, kdy uživatel provede ve vizualizaci pokyn k zpracování uložených csv souborů. Backend potom na základě tohoto požadavku odešle do modulu pro strojové učení HTTP požadavek obsahující absolutní cestu k uloženému CSV souboru s vibračním záznamem, který je přečten z databáze.

Modul pro strojové učení pak s touto cestou pracuje. Prvním nezbytným krokem po načtení dat prostřednictvím knihovny Pandas [25] je očištění signálu od nežádoucích složek. Surová data z průmyslových převodníků často obsahují stejnosměrnou složku (DC offset) a lineární trend (drift), které mohou vznikat vlivem zahřívání senzoru nebo nepřesností A/D převodníku. Pro odstranění těchto artefaktů je využita funkce *detrend* z knihovny SciPy [26], která signál matematicky vycentruje kolem nulové osy a odstraní jeho náklon, čímž data připraví pro další zpracování.

Z vyčištěného časového signálu jsou následně extrahovány klíčové statistické parametry z tabulky 1. Tyto výpočty jsou prováděny primárně pomocí vektorizovaných operací knihovny NumPy [27] a modulu *scipy.stats*. Vypočtené hodnoty jsou odeslány zpět hlavnímu backendu ve formátu JSON, který je trvale zapíše k příslušnému záznamu do relační databáze, zatímco původní CSV soubor na disku je přepsán vyčištěným signálem pro budoucí zpracování transformacemi využívanými pro strojové učení.

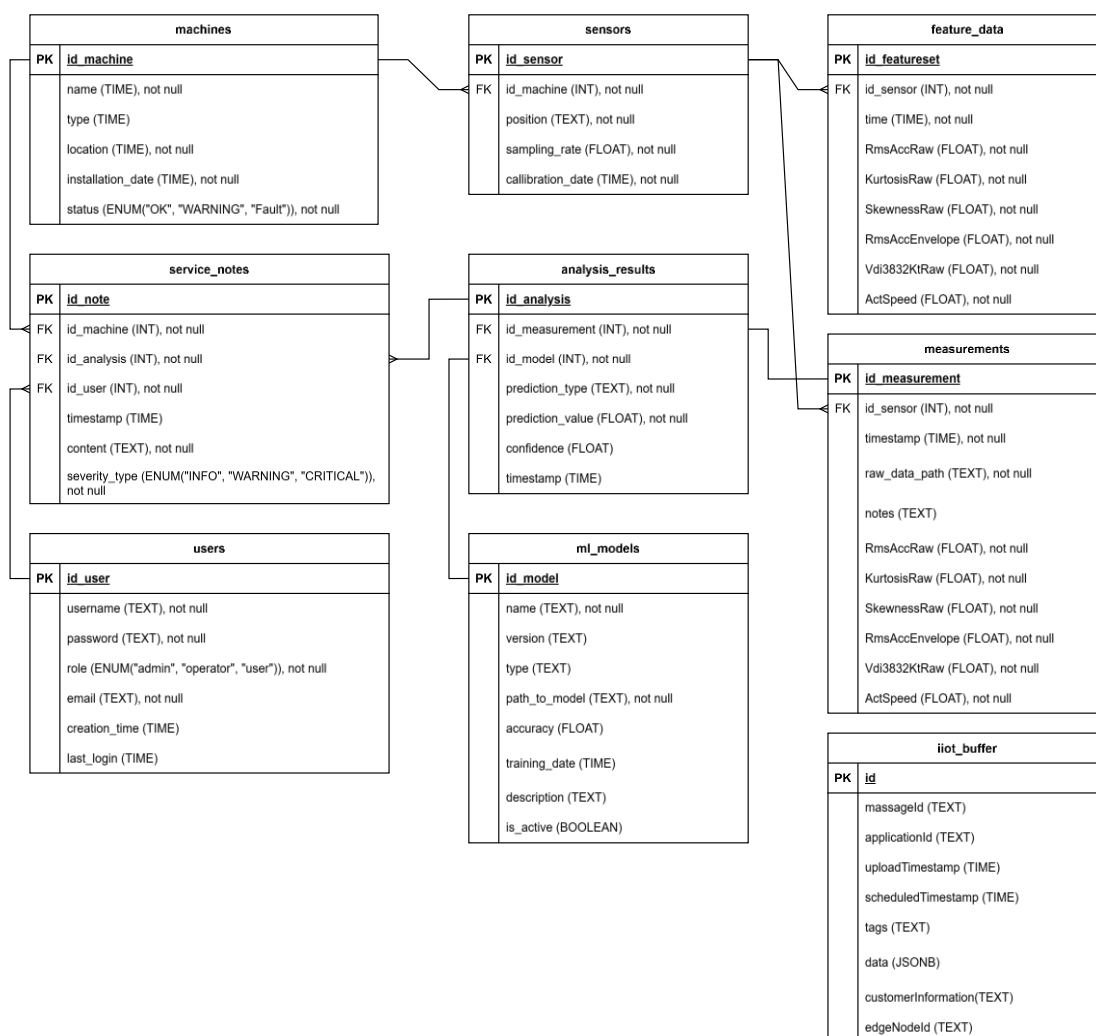
Kromě extrakce časových příznaků zajišťuje modul pro strojové učení také transformace signálu do frekvenční a časově-frekvenční oblasti. Pro spektrální analýzu je využita rychlá Fourierova transformace (FFT) implementovaná v knihovně NumPy, zatímco pro výpočet spojitě vlnkové transformace (CWT) a generování skalogramů slouží knihovna PyWavelets [28]. Z důvodu optimalizace úložného prostoru nejsou výsledky těchto výpočetně náročných transformací ukládány do databáze, ale jsou počítány dynamicky pouze při vyžádání z uživatelského rozhraní, přičemž grafický výstup je pomocí knihovny Matplotlib [29] renderován na serveru a odeslán do webové vizualizace ve formátu Base64.

5.3 Archivace dat a návrh databáze

Posledním, avšak kritickým krokem v řetězci práce s daty je jejich trvalá a efektivní archivace. Vzhledem k průmyslové povaze aplikace, kde data ze senzorů přibývají vysokým tempem, bylo nutné zvolit robustní databázové řešení schopné plynule obsluhovat masivní zápisy (write-heavy workload) a zároveň poskytovat rychlé odezvy pro analytické dotazy vizualizačního rozhraní a potřeby strojového učení.

Z tohoto důvodu byla jako hlavní databázový nástroj zvolena vyspělá open-source relační databáze PostgreSQL, která je provozována jako samostatná izolovaná mikroslužba uvnitř Docker kontejneru. Pro splnění specifických požadavků vibrodiagnostiky byla tato databáze navíc rozšířena o plugin TimescaleDB. Toto rozšíření transformuje standardní PostgreSQL na vysoce výkonnou databázi časových řad (Time-Series Database). Klíčovým mechanismem TimescaleDB je automatické dělení dat (tzv. partitioning) do skrytých časových úseků zvaných hypertabulky (hypertables). Tento přístup radikálně urychluje vyhledávání dat v konkrétních časových oknech a zefektivňuje správu paměti při rostoucím objemu záznamů.

5.3.1 Architektura databáze a relační model



Obr. 17: ERD diagram databáze.

Pro zajištění datové integrity a logického uspořádání informací byl navržen relační model, který lze rozdělit do několika funkčních celků (viz ERD diagram na obrázku 17):

- **Infrastrukturní vrstva** (*machines, sensors*): Tyto tabulky evidují fyzickou hierarchii podniku. Tabulka *machines* uchovává metadata o monitorovaných zařízeních (název, lokace, provozní stav). Na ni je vazbou 1:N navázána tabulka *sensors*, která definuje konkrétní měřicí místa, použitou vzorkovací frekvenci a aktuální stav připojení senzoru.

- **Vrstva naměřených dat** (*measurements, feature_data, iiot_buffer*): Zde se naplno projevuje dvoukolejná architektura sběru dat popsaná v předchozích podkapitolách.
 - Tabulka *measurements* slouží primárně k evidenci surových datových oken a využívá zmíněného pluginu TimescaleDB. Aby relační databáze nebyla neúměrně zatěžována ukládáním milionů vzorků pro každé měření, využívá se sloupec *raw_data_path*. Ten funguje pouze jako textový ukazatel (pointer) na fyzické umístění objemného CSV souboru v lokálním úložišti. Dále tabulka také obsahuje charakteristické hodnoty, které jsou dopočítávány z CSV souborů surových dat.
 - Tabulka *feature_data* je také koncipována jako TimescaleDB hypertabulka. Obsahuje přímo agregované statistické a frekvenční příznaky (viz. Tabulka 1), které jsou indexovány podle přesného časového razítka (*time*). Uložení těchto malých číselných hodnot přímo do databáze umožňuje webové vizualizaci okamžité a plynulé vykreslování historických trendů.
 - Tabulka *iiot_buffer* slouží jako přestupní bod pro charakteristické hodnoty získávané z IIoT Connectoru. Na tuto tabulku je napojen trigger, který rozděljuje jednotlivé hodnoty do tabulky *feature_data*. Bližší informace viz podkapitola 5.3.2 o automatizaci sběru dat.
- **Vrstva strojového učení** (*ml_models, analysis_results*): Pro správu životního cyklu umělé inteligence slouží tabulka *ml_models*, která funguje jako registr (*model registry*). Udrží informace o verzích, přesnosti a obsahuje cesty k uloženým vahám modelů (.pth soubory). Výsledky inferencí (detekce anomálií, klasifikace poruch, predikce zbytkové životnosti) se následně ukládají do tabulky *analysis_results*, která je relací propojena s konkrétním měřením a použitým modelem, což zajišťuje plnou zpětnou sledovatelnost rozhodnutí AI.
- **Administrativní vrstva** (*users, service_notes*): Poslední blok tabulek zajišťuje správu uživatelských účtů (s využitím hashovaných hesel) a evidenci servisních zásahů či poznámek údržby vázaných na konkrétní stroje.

5.3.2 Automatizace sběru dat a integrace IIoT Connectoru

Při integraci softwarového řešení B&R IIoT Connector vyvstal zásadní technický problém: tento nástroj odesílá vyčtená data z OPC UA serveru v komplexním, vnořeném formátu JSON. Přímý zápis těchto dat do relační tabulky *feature_data* by byl neefektivní a vyžadoval by neustálou transformaci na straně aplikačního backendu, což by neúměrně zatěžovalo CPU.

Tento problém byl vyřešen implementací mechanismu automatického třídění přímo na straně databázového nástroje. Byla vytvořena speciální „přestupní“ tabulka *iiot_buffer* typu JSONB. K ní byl vytvořen SQL trigger spojený s *procedurou process_iiot_json()*. Celý proces funguje autonomně:

1. IIoT Connector zapíše surový JSON balík do tabulky *iiot_buffer*.
2. Databáze okamžitě spustí trigger, který pomocí *Common Table Expressions* (CTE) rozbalí JSON pole a extrahuje hodnoty pro jednotlivé kanály.
3. Procedura automaticky dohledá příslušné *id_machine* podle identifikátoru senzoru v OPC UA cestě a provede čistý zápis (*pivot*) do cílové tabulky *feature_data*.

4. Původní JSON záznam je po zpracování smazán, čímž je udržována minimální velikost databáze.

Tento přístup ukazuje efektivitu delegování výpočetní logiky z aplikační vrstvy do databázové, což výrazně zvyšuje propustnost celého diagnostického systému.

5.3.3 Kontejnerizace a orchestrace úložiště

Důležitým aspektem navržené architektury je její plná replikovatelnost. Databáze není instalována přímo do operačního systému Edge zařízení, ale je plně kontejnerizována v Dockeru. To umožňuje snadnou správu verzí a izolaci od okolního prostředí.

Pro správu a ladění databáze v průběhu vývoje byl využit nástroj pgAdmin 4, který byl rovněž součástí Docker stacku. Tento nástroj umožnil efektivní monitorování výkonu TimescaleDB a vizuální kontrolu integrity dat během testování stavových automatů PLC. Klíčovým prvkem pro automatizované nasazení je inicializační skript `init.sql`. Při prvním spuštění kontejneru (tzv. „cold start“) tento skript automaticky:

- povolí rozšíření TimescaleDB,
- definuje kompletní schéma tabulek a vztahy (Foreign Keys),
- vytvoří hypertables a nastaví trigger pro IIoT Connector,
- vytvoří výchozího administrátora.

Díky této orchestraci je systém v duchu moderních DevOps principů *"production-ready"* okamžitě po spuštění příkazu *docker-compose up*, bez nutnosti jakékoliv manuální konfigurace na straně koncového uživatele nebo pracovníků IT údržby podniku.

6 IMPLEMENTACE APLIKAČNÍHO ROZHRAŇÍ – BACKEND

Aplikační rozhraní (*Backend*) představuje hlavní komunikační uzel a „mozek“ celého systému. Jeho úlohou není pouze zprostředkování dat, ale především orchestrace toků mezi databázemi *TimescaleDB*, mikroslužbou pro strojové učení a klientskou aplikací. Backend zajišťuje autentizaci uživatelů, validaci vstupních dat a spouštění asynchronních úloh, jako je stahování surových dat z PLC přes protokol FTP (*FTP Pull*).

6.1 Technologie FastAPI a Uvicorn

Pro vývoj aplikačního rozhraní (*backend*) byl zvolen moderní webový framework *FastAPI* [30]. V kontextu IIoT aplikací, kde server musí paralelně obsluhovat desítky nesouvisejících požadavků – od neustálých dotazů do databáze přes asynchronní stahování objemných souborů přes FTP až po komunikaci s modely strojového učení – je naprosto klíčová schopnost efektivně řídit síťový provoz. *FastAPI* je od základu navrženo pro nativní využití asynchronního programování (pomocí klíčových slov *async/await*) v jazyce Python. Na rozdíl od tradičních synchronních modelů, kde procesorové vlákno nečinně čeká na dokončení pomalé síťové či diskové operace (tzv. blokující I/O operace), asynchronní model umožňuje serveru přepnout kontext a během tohoto čekání obsluhovat další klienty [31].

Důležitým architektonickým rozhodnutím v rámci implementace backendu je využití takzvaného hybridního přístupu ke zpracování požadavků. Zatímco I/O náročné úlohy s potenciálně dlouhou latencí, jako je automatizovaný sběr dat z PLC přes protokoly OPC UA a FTP, jsou implementovány plně asynchronně, pro standardní databázové operace (vyhledávání a zapisování) je využito klasické synchronní rozhraní knihovny *SQLAlchemy* [32]. Framework *FastAPI* je totiž navržen tak, že koncové body definované standardním klíčovým slovem *def* automaticky deleguje a spouští v odděleném fondu vláken (*external threadpool*). Hlavní asynchronní smyčka událostí (*Event Loop*) tak zůstává neblokovaná. Toto řešení kombinuje maximální propustnost sítě pro dlouhotrvající operace se stabilitou a robustností tradičního relačního mapování bez nutnosti zavádět složité asynchronní databázové adaptéry.

Významným prvkem frameworku *FastAPI* je jeho úzká integrace s knihovnou *Pydantic* [33], která využívá standardní typování jazyka Python (*Type Hints*) k automatické validaci dat. Jak lze vidět ve zdrojovém kódu implementace backendu u tříd jako *TrainingSegmentFrontend* nebo *WebhookPayload*, backend zpracovává striktně definované datové objekty (*Data Transfer Objects*). Pokud klientská aplikace pošle požadavek s chybným datovým typem (například textový řetězec místo očekávaného celého čísla u ID stroje), systém požadavek automaticky zamítne a vrátí chybovou zprávu podle HTTP. To podle oficiální dokumentace [30] zásadně snižuje riziko pádu aplikace v důsledku neošetřených výjimek a zvyšuje celkovou robustnost systému pro průmyslové využití.

Při návrhu architektury pomocí mikroslužeb byly brány v potaz i tradiční alternativy, které dlouhodobě dominují webovému vývoji v Pythonu, především frameworky *Django* [34] a *Flask* [35]. Django představuje robustní, avšak pro účely této práce příliš těžkopádné a monolitické řešení. Mikroframework *Flask* je sice lehký, ale jeho návrh je historicky synchronní. Přestože moderní verze těchto frameworků začínají asynchronní chování dodatečně podporovat, *FastAPI* tuto funkcionalitu nabízí nativně s prokazatelně vyšším výkonem, který se blíží kompilovaným programovacím jazykům jako *Go* nebo *NodeJS*.

Aby bylo možné tento asynchronní potenciál plně využít, je nutné nasadit odpovídající webový server. Běžné webové aplikace v Pythonu dříve využívaly standard *WSGI* (*Web Server Gateway Interface*) a servery jako *Gunicorn*, které však kvůli své synchronní architektuře představovaly úzké hrdlo (*bottleneck*) pro asynchronní kód. Z toho důvodu je backendová aplikace hostována na serveru *Uvicorn* [36], který implementuje modernější specifikaci *ASGI* (*Asynchronous Server Gateway Interface*). *Uvicorn* funguje jako vysoce výkonný prostředník mezi sítí a samotným aplikačním kódem *FastAPI*, čímž zajišťuje maximální propustnost a minimální latenci celého systému.

6.2 Návrh API endpointů

Samotné aplikační rozhraní je navrženo jako *RESTful API*, kde jsou jednotlivé zdroje dostupné přes unikátní cesty (koncové body – endpoints). Velkou výhodou zvoleného frameworku je automatické generování interaktivní dokumentace podle standardu *OpenAPI* (dříve *Swagger*). Tato dokumentace je dostupná přímo na běžícím serveru (adresa služby – v tomto případě *IP_adresa_APC:8005/docs*) a umožňuje otestovat jednotlivé endpointy v reálném čase bez nutnosti externích nástrojů.

V rámci práce je implementováno velké množství endpointů, které jsou v kódu rozděleny do sekcí podle jejich účelu, třeba správa uživatelů, monitoring dat a řízení modelů strojového učení. Jako reprezentativní příklady implementace endpointů jsou níže uvedeny dva reprezentativní příklady:

- **Login endpoint** (POST */login*): Tento koncový bod zajišťuje autentizaci uživatelů a generování přístupových JWT (JSON Web Token) tokenů. Přihlašovací údaje z klientské aplikace jsou zpracovány pomocí vestavěné třídy *OAuth2PasswordRequestForm*. Backend se následně připojí k databázi a ověří existenci uživatele. Shoda zadaného hesla s uloženým hashem je kontrolována pomocí pomocné kryptografické funkce *verify_password*. Při selhání validace je proces okamžitě přerušen a klientovi je vrácena výjimka s HTTP kódem 401 (*Unauthorized*). Po úspěšném ověření identity systém aktualizuje v databázi čas posledního přihlášení (*last_login*) pro účely auditu a monitoringu. Následně je vygenerován podepsaný JWT token, do jehož datové části (payloadu) se ukládá uživatelské jméno a úroveň oprávnění (tzv. role). Klientské aplikaci je nakonec vrácen JSON objekt s přístupovým tokenem, specifikací jeho typu (*Bearer*) a přidělenou rolí, kterou webová vizualizace využívá k řízení přístupu k jednotlivým částem uživatelského rozhraní. Kód tohoto endpointu je k vidění na obrázku 18.


```

174 @app.post("/login")
175 def login(form_data: OAuth2PasswordRequestForm = Depends()):
176     """
177     Ověří uživatelské jméno a heslo oproti DB a vygeneruje přístupový JWT token.
178     Aktualizuje čas posledního přihlášení uživatele v DB.
179     """
180     with engine.connect() as conn:
181         query = text("SELECT id_user, username, hashed_password, role FROM users WHERE username = :user")
182         user_record = conn.execute(query, {"user": form_data.username}).fetchone()
183
184         if not user_record:
185             raise HTTPException(status_code=401, detail="Uživatel neexistuje")
186
187         if not verify_password(form_data.password, user_record[2]):
188             raise HTTPException(status_code=401, detail="Nesprávné heslo")
189
190         # Aktualizace timestampu přihlášení (id_user je index 0)
191         update_query = text("UPDATE users SET last_login = :now WHERE id_user = :uid")
192         conn.execute(update_query, {
193             "now": datetime.now(timezone.utc),
194             "uid": user_record[0]
195         })
196         conn.commit()
197
198         # Generování JWT (sub = username na indexu 1, role na indexu 3)
199         access_token = create_access_token(
200             data={"sub": user_record[1], "role": user_record[3]}
201         )
202
203         return {"access_token": access_token, "token_type": "bearer", "role": user_record[3]}

```

Obr. 18: Ukázka kódu koncového bodu pro přihlášení uživatele.

- Endpoint aktivace ML modelu (PUT `/models/{model_id}/activate`):** Tento chráněný koncový bod slouží k bezpečnému přepnutí aktivní verze modelu pro diagnostickou úlohu a zajišťuje její okamžitou synchronizaci s běžící mikroslužbou strojového učení. Po úspěšné autorizaci uživatele a přijetí identifikátoru `model_id` z URL cesty se backend připojí k databázi, kde ověří existenci modelu a zjistí jeho logický název. Následně v rámci transakce provede dvě sekvenční operace: hromadně deaktivuje (`is_active = False`) všechny dosud aktivní modely se stejným názvem a vzápětí aktivuje (`is_active = True`) pouze ten nově zvolený. Jakmile jsou změny úspěšně zapsány do databáze, backend odešle synchronní HTTP POST požadavek na koncový bod a reload mikroslužby strojového učení. Tím jí přikáže uvolnit staré modely z operační paměti a načíst aktuálně platné váhy. Kód endpointu obsahuje ošetření chyb (blok `try-except`). Pokud mikroslužba na požadavek o přenačtení z důvodu dočasného výpadku neodpoví, databázová změna zůstává platná, avšak uživatel je v návratové zprávě varován, že se změna v reálné inferenci projeví až po restartu příslušné služby.


```

1298 @app.put("/models/{model_id}/activate")
1299 def activate_model_version(model_id: int, token: str = Depends(oauth2_scheme)):
1300     """
1301     Aktivuje vybranou verzi modelu v DB, hromadně deaktivuje předchozí verze
1302     se stejným logickým názvem a nařídí běžící ML mikroslužbě okamžitý reload vah do RAM.
1303     """
1304     with engine.connect() as conn:
1305         res = conn.execute(text("SELECT name FROM ml_models WHERE id_model = :mid"), {"mid": model_id}).fetchone()
1306         if not res: raise HTTPException(status_code=404, detail="Model nenalezen.")
1307         model_name = res[0]
1308
1309         conn.execute(text("UPDATE ml_models SET is_active = False WHERE name = :name"), {"name": model_name})
1310         conn.execute(text("UPDATE ml_models SET is_active = True WHERE id_model = :mid"), {"mid": model_id})
1311         conn.commit()
1312
1313     try:
1314         reload_res = requests.post(f"{ML_SERVICE_URL}/reload", timeout=10)
1315         reload_res.raise_for_status()
1316     except Exception as e:
1317         return {
1318             "status": "partial_success",
1319             "message": "Model aktivován v DB, ale ML servisu neodpovídá. Změna se projeví po jejím restartu."
1320         }
1321     return {"status": "success", "message": "Model byl úspěšně nasazen do produkce."}

```

Obr. 19: Ukázka kódu endpointu pro aktivaci ML modelu.

Kompletní výčet všech dostupných metod a jejich parametrů je součástí automaticky generované dokumentace Swagger. Dále jsou všechny dostupné ve zdrojovém kódu aplikace, který tvoří technickou přílohu této práce, či z již výše zmíněné github stránky.

6.3 Metoda FTP Pull

Klíčovou funkcí backendové aplikace, která ji odlišuje od běžných webových rozhraní, je schopnost aktivní a plně automatizované komunikace s PLC. Architektura sběru surových vibračních dat nevyužívá pasivní naslouchání (kdy by PLC data samo odesílalo), ale implementuje metodu tzv. "FTP Pull". V tomto modelu přebírá backend roli nadřazeného systému (Master), který v přesně definovaných intervalech aktivně dotazuje a řídí chování PLC (Slave).

Proces je plně automatizován pomocí knihovny *APScheduler* (konkrétně třídy *AsyncIOScheduler*), která je inicializována přímo při startu serveru v rámci správce životního cyklu aplikace (lifespan). Plánovač je nastaven tak, aby spouštěl asynchronní sběrnou rutinu v pravidelných časových oknech (v produkčním režimu každé čtyři hodiny). Aby se předešlo zablokování hlavního vlákna serveru a výpadku dostupnosti API pro ostatní uživatele, je celá komunikační sekvence implementována jako neblokující asynchronní úloha (*async/await*). Samotná sekvence sběru dat u každého aktivního stroje probíhá v následujících logických krocích:

1. **Ověření stavu stroje:** Před zahájením komunikace systém provede bezpečnostní kontrolu v databázi. Vypočítá průměrnou hodnotu RMS (Root Mean Square) za poslední hodinu, a pokud je hodnota nižší než stanovená mez (např. 0.1), systém vyhodnotí, že stroj aktuálně nepracuje. Sběr dat se pro daný cyklus přeskočí, čímž se efektivně zamezuje stahování a ukládání prázdných datových sad (tzv. "bílého šumu").

2. **Řízení stavového automatu (OPC UA):** Backend naváže asynchronní spojení s PLC pomocí protokolu OPC UA (využitím knihovny *asynqua*). Skript dynamicky iteruje přes mapované datové kanály a zapisuje řídicí parametry do specifických uzlů (především *BufferNumber*, pro určení konkrétního senzory a *WorkID*, které udává pracovní název pro celý proces, potažmo název koncového souboru). Následně backend přepnutím uzlu *Start* na logickou jedničku inicializuje na straně PLC proces překlopení vyrovnávací paměti (bufferu) do CSV souboru, jak je popsáno v sekci o sběru surových dat. Během tohoto procesu server asynchronně vyčkává na potvrzovací signál v uzlu *Done*.
3. **Zabezpečený přenos souboru (FTP_TLS):** Jakmile PLC potvrdí úspěšné vytvoření CSV souboru a předá jeho název, backend ukončí proces přes uzly *Start* a *Reset*. Následně v odděleném vlákně naváže zabezpečené spojení FTPS (File Transfer Protocol přes TLS). Při implementaci tohoto kroku bylo nutné vyřešit specifikum průmyslového hardwaru B&R. Vestavěný FTP server PLC totiž podporuje pouze starší kryptografické standardy. Z toho důvodu bylo nezbytné v rámci SSL kontextu na straně backendu explicitně povolit starší verze protokolu (např. TLSv1) a snížit výchozí bezpečnostní úroveň šifer (nastavením parametru *SECLEVEL=0*). Bez této striktní úpravy by moderní prostředí serveru komunikaci z bezpečnostních důvodů automaticky zablokovalo. Předpokladem pro toto snížení úrovně zabezpečení je fakt, že se celá komunikace odehrává v lokální podnikové síti a je tedy zaručena její bezpečnost. Po úspěšném navázání spojení je soubor stažen na lokální úložiště a vzápětí smazán z paměti PLC, čímž se předchází jejímu přeplnění.
4. **Zápis do databáze:** Po úspěšném stažení a ověření souboru systém vyhledá odpovídající identifikátor senzoru a zapíše absolutní cestu k novému CSV souboru společně s časovým razítkem do databázové tabulky *measurements*. Tím je cyklus uzavřen a data jsou připravena pro následné zpracování modely strojového učení.

Tímto postupem je získán CSV soubor jednoho záznamu. Tento postup byl v rámci testování uzavřen do cyklu *FOR*, pomocí kterého takto procházel data pro všechny 4 senzory, dostupné v testovacím zapojení.

Díky této robustní architektuře doplněné o mechanismy proti selhání (např. automatické odpojení při překročení časového limitu – *timeout*) je zajištěna vysoká spolehlivost sběru dat i v prostředí s možnými sítovými výpadky.

6.4 Zabezpečení a autorizace

Vzhledem k tomu, že systém pracuje s průmyslovými daty, bylo bráno v potaz i zabezpečení celé aplikace. Implementované řešení využívá standard *JWT*. Proces zabezpečení, realizovaný v modulu *auth.py*, probíhá v několika krocích:

- **Hashování hesel:** Hesla uživatelů nejsou v databázi nikdy uložena v čistém textu. Používá se algoritmus *BCrypt* (z knihovny *passlib*), který ke každému heslu přidává náhodnou sůl (*salt*) a vytváří bezpečný *hash*.

- **Generování tokenů:** Po úspěšném ověření identity vytvoří server podepsaný token s definovanou dobou platnosti (např. 30 minut). Tento token obsahuje identifikaci uživatele a je šifrován pomocí tajného klíče (*SECRET_KEY*).
- **Ověřování přístupu:** Každý další požadavek z frontendu na chráněný endpoint musí v hlavičce obsahovat tento token. Backend jej pomocí algoritmu HS256 dekóduje a ověří jeho integritu a platnost.

Tento bezstavový přístup k autentizaci je ideální pro architekturu mikroslužeb, protože umožňuje snadné škálování bez nutnosti sdílení relací (*sessions*) mezi servery.

6.5 Produkční nasazení – kontejnerizace

Pro zajištění bezproblémové přenositelnosti aplikačního rozhraní mezi vývojovým a produkčním prostředím a pro izolaci softwarových závislostí je využita technologie kontejnerizace platformy *Docker*. Nasazení backendu v kontejneru garantuje, že aplikace bude mít k dispozici identické běhové prostředí bez ohledu na hostitelský operační systém průmyslového serveru.

Základem kontejnerizace backendové služby je konfigurační soubor *Dockerfile*, který definuje přesný postup sestavení obrazu (*image*) a je dostupný v přílohách nebo na github stránce projektu. Proces začíná výběrem optimálního základního obrazu (*Base Image*), zde byl zvolen `python:3.11-slim`. Odlehčená verze „*slim*“ byla zvolena záměrně, neboť obsahuje pouze nezbytné systémové knihovny pro běh jazyka Python, čímž se radikálně snižuje celková velikost výsledného kontejneru a minimalizuje se zranitelnost díky nadbytečným knihovnám.

Následně je nastavena proměnná prostředí `PYTHONUNBUFFERED=1`. Tento parametr byl klíčový pro odstranění chyb v produkčním nasazení, protože zakazuje ukládání standardního výstupu do mezipaměti (*bufferu*). Výpisy z aplikace a chybová hlášení jsou tak okamžitě propisovány do logů kontejneru, což je vhodné pro zmíněné odstranění produkčních chyb či pro monitorování asynchronního sběru dat z PLC v reálném čase.

Po definování pracovního adresáře `/app` uvnitř kontejneru jsou zkopírovány požadavky na externí knihovny (soubor `requirements.txt`) a je spuštěna jejich instalace. Následuje přesun samotného zdrojového kódu a dedikovaný příkaz `RUN mkdir -p /app/data`. Vytvoření této složky je kritickým krokem pro správnou funkčnost rutiny stahování dat přes FTP – do tohoto adresáře se totiž ukládají CSV soubory z řídicího systému.

V závěru konfiguračního souboru je vystaven síťový port 8000 a definován spouštěcí příkaz (CMD), který inicializuje webový server Uvicorn na adrese 0.0.0.0. Tímto nastavením je zajištěno, že server naslouchá požadavkům přicházejícím zvenčí kontejneru.

Z hlediska distribuce a nasazení je uplatňován standardní postup. Vytvořený obraz backendu je zkompileován a nahrán do centrálního veřejného repozitáře Docker Hub. Na produkčním průmyslovém hardwaru je pak obraz stažen a spuštěn prostřednictvím nástroje Docker Compose. Ten funguje jako orchestrátor, který zajišťuje logické propojení backendu s dalšími izolovanými mikroslužbami v systému (například databází TimescaleDB a dedikovaným kontejnerem pro strojové učení) do jedné virtuální sítě. Detailní popis této komplexní orchestrace celého systému je pak předmětem závěrečného shrnutí této práce.

7 UŽIVATELSKÉ ROZHRAŇÍ A VIZUALIZACE – FRONTEND

Důležitou součástí jakéhokoliv pokročilého diagnostického systému je také způsob prezentace získaných výsledků koncovému uživateli. V průmyslovém prostředí je nezbytné, aby operátoři a pracovníci údržby měli k dispozici přehledné rozhraní člověk-stroj (*Human-Machine Interface – HMI*), které dokáže přetvořit rozsáhlé objemy surových dat a komplexní výsledky inferencí do srozumitelné a vizuálně uchopitelné podoby.

S nástupem Průmyslu 4.0 a distribuovaných IIoT architektur se standardem pro tvorbu těchto rozhraní stávají moderní webové technologie. Oproti tradičním lokálním vizualizacím, které jsou fixně vázány na konkrétní hardwarový panel u stroje, umožňuje webový přístup bezpečný dohled nad systémem z jakéhokoliv zařízení (PC, tablet či průmyslové dotykové panely) v rámci podnikové sítě. Cílem navržené prezentační vrstvy v této práci je poskytnout uživateli nejen pasivní náhled na historické trendy vibrací, ale také interaktivní ovládací prvky pro obousměrnou komunikaci se systémem – od správy uživatelských účtů až po vzdálené řízení životního cyklu modelů strojového učení.

V počáteční fázi návrhu architektury se pro vizualizaci časových řad zvažovalo využití populární *open-source* platformy Grafana [37], která disponuje nativní podporou pro použitou databázi TimescaleDB. Přestože Grafana představuje průmyslový standard pro rychlé sestavování analytických dashboardů, pro potřeby tohoto komplexního projektu se ukázala jako nevhodná, pro její omezené možnosti týkající se celkové obsluhy aplikace. Její primární zaměření na pasivní zobrazování metrik činí implementaci složitějších uživatelských interakcí (jako je například manuální anotace datových segmentů či odesílání parametrů pro dodatečné učení modelů přes REST API) velmi složitou až neuskutečnitelnou. Z důvodu zajištění naprosté kontroly nad celistvostí aplikace, maximální flexibility a vytvoření jednotného uživatelského zážitku (*User Experience – UX*) bylo nakonec přistoupeno k vývoji vlastního webového rozhraní na míru (tzv. *Single Page Application*), které je postaveno na moderní javascriptové knihovně *React* [38].

7.1 Vývoj webové aplikace – React

Pro realizaci klientské části systému byla zvolena knihovna React, která v současnosti představuje jeden z nejrozšířenějších nástrojů pro tvorbu dynamických uživatelských rozhraní. React, vyvíjený společností Meta, je založen na deklarativním paradigmatu, což znamená, že vývojář definuje, jak má rozhraní vypadat v určitém stavu, a knihovna se postará o efektivní aktualizaci a vykreslení komponent při změně dat. Hlavní výhody nasazení Reactu v diagnostickém systému jsou podle její dokumentace [38]:

- **Komponentová architektura:** Celé HMI je rozděleno na znovupoužitelné stavební bloky (komponenty). Například graf pro zobrazení časových řad, indikátor stavu stroje nebo formulář pro přihlášení jsou vyvíjeny jako izolované entity. To výrazně usnadňuje

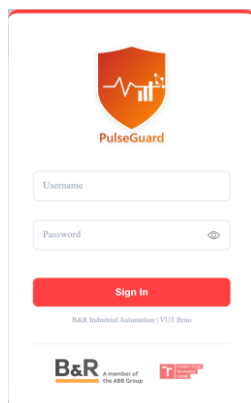
údržbu kódu a umožňuje snadné rozšiřování systému o nové senzory či analytické nástroje.

- **Virtual DOM:** React využívá mechanismus tzv. virtuálního DOMu (Document Object Model). Při změně dat (např. příchod nového RMS vzorku z TimescaleDB) React nejprve provede změnu v paměti a následně aktualizuje pouze ty části reálné webové stránky, které se skutečně změnily. V aplikaci, která zobrazuje tisíce datových bodů v reálném čase, je tato efektivita klíčová pro plynulost uživatelského zážitku bez zbytečného přetěžování procesoru klientského zařízení.
- **Bohatý ekosystém knihoven:** Pro účely vibrodiagnostiky byla využita integrace s knihovnou Recharts, která umožňuje vysoce výkonné vykreslování technických grafů, a Axios pro asynchronní komunikaci s backendovým REST API prostřednictvím protokolu HTTP.

Architektura aplikace využívá moderní přístupy *Hooks* (např. *useState*, *useEffect*), které umožňují efektivní správu životního cyklu komponent a synchronizaci dat s databází bez nutnosti častého manuálního obnovování stránky.

7.2 Představení HMI

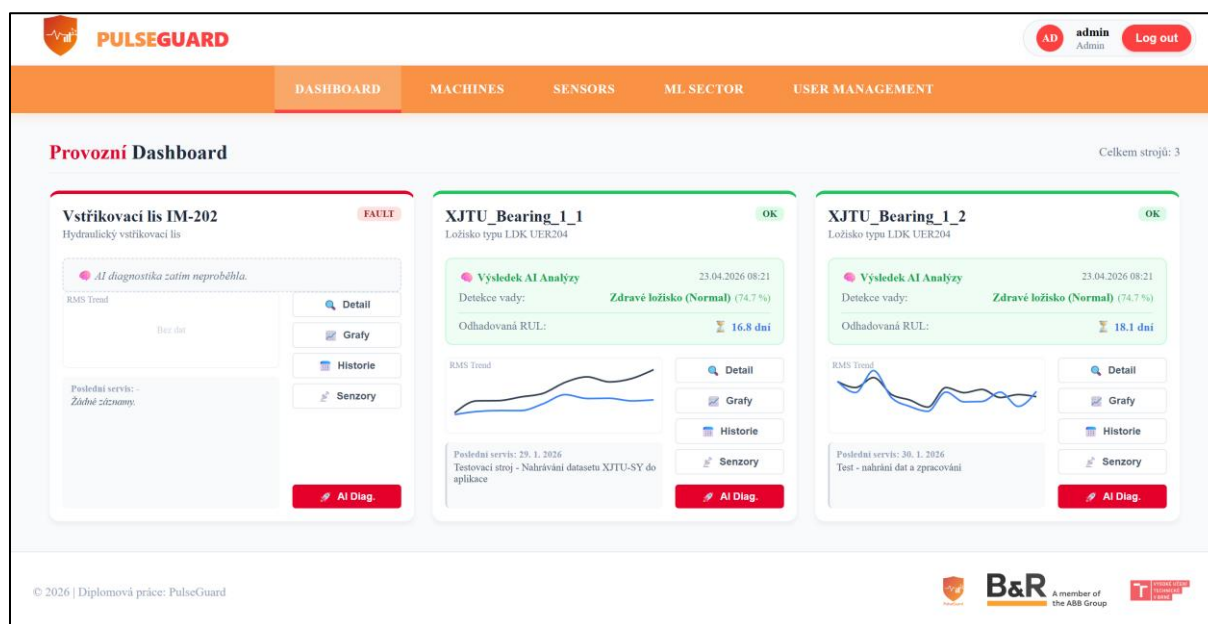
První situace, která nastane při spuštění vizualizace ve webovém prohlížeči je ta, že vyskočí komponenta pro přihlášení do aplikace. V čele panelu je pracovní logo a název celé aplikace PulseGuard. Zde je vhodné zmínit, že aplikace má implementován běžný uživatelský systém. Při prvním spuštění je vytvořen v databázi pouze jediný základní uživatel – admin (přihlašovací jméno: admin, heslo: .admin). Samotná přihlašovací komponenta je na obrázku 20 níže.



Obr. 20: Úvodní obrazovka vizualizace – přihlášení.

Po přihlášení do aplikace se nám zobrazí domovská stránka aplikace *Dashboard* (obrázek 21). Hlavní komponentou této stránky jsou karty s jednotlivými monitorovanými stroji. Každá karta je koncipována tak, aby ukazovala ty nejdůležitější informace a odkazy týkající se stroje. V horní části každé karty můžeme vidět název stroje s jeho popisem a kontrolkou stavu stroje (zelené OK – vše je v pořádku, červené FAULT – poruchový stav, další stavy jsou WARNING – nebezpečí a STOPPED – stroj je zastaven). V pravé straně karty jsou odkazy na jednotlivé

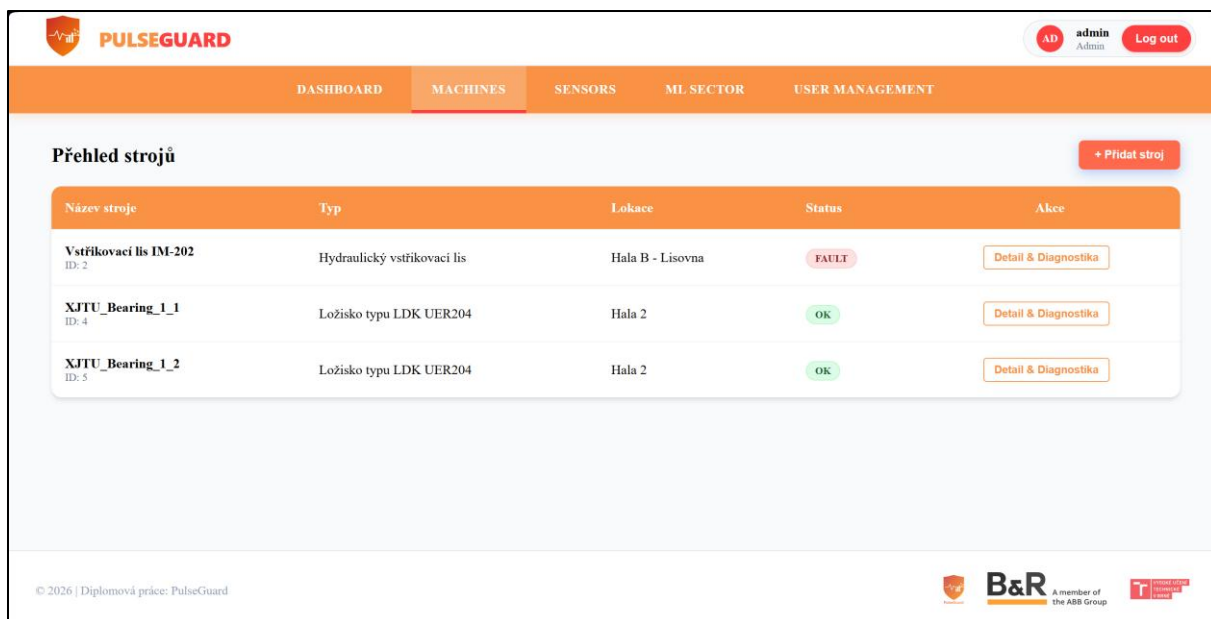
specifikace stroje (ať už detail stroje, grafy charakteristických hodnot, historie měření, detail senzorů připojených ke stroji, a hlavně červené tlačítko odkazující do sekce s diagnostikou pomocí strojového učení). Hlavní část karty stroje zabírají tři informační panely. Když půjdeme odshora, první panel ukazuje poslední dostupné výsledky analýzy stroje, prostřední panel obsahuje trend charakteristické hodnoty RMS stroje z jednotlivých senzorů. Tento parametr byl zvolen jako nejvíce vypovídající o stavu stroje. Spodní panel potom obsahuje poslední poznámku uloženou u stroje – k samotným servisním poznámkám se dostaneme později.



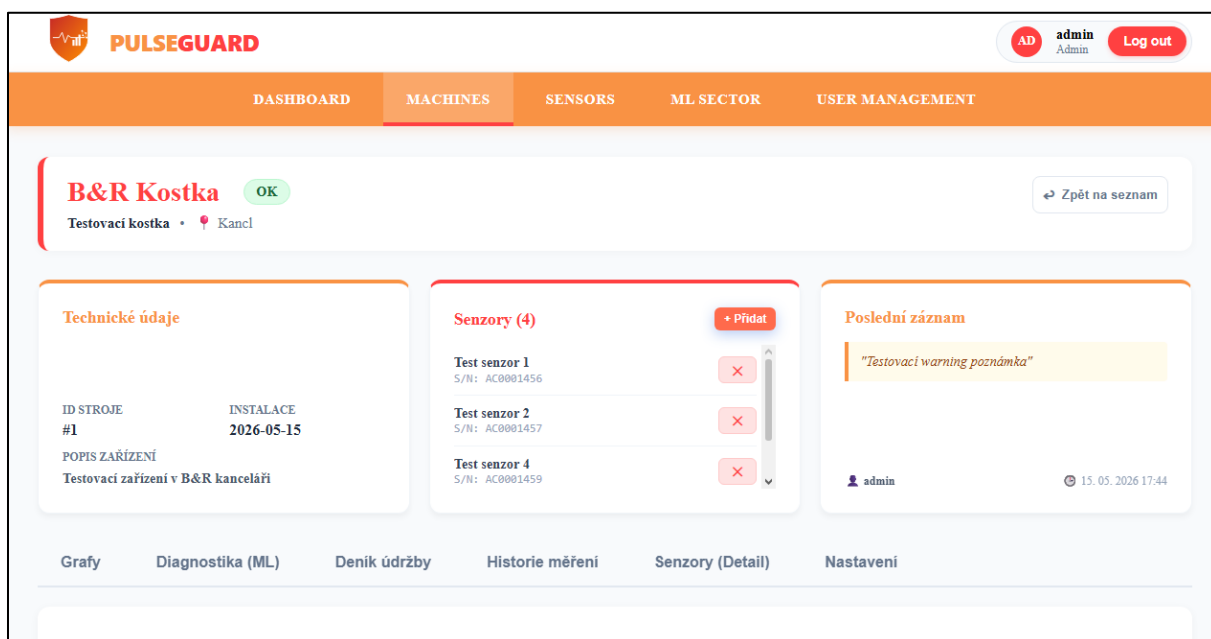
Obr. 21: Provozní dashboard aplikace.

Když se budeme dále posouvat hlavním menu tak narazíme na sekci strojů (obrázek 22). Zde můžeme vidět tabulku se stroji a tlačítko pro spuštění komponenty s přidáním nového stroje.

Z této stránky se pomocí akce „Detail a Diagnostika“ dostaneme na přehledovou stránku stroje (Obrázek 23). Této stránce opět dominují čtyři informační panely (název stroje, technické údaje stroje, poslední servisní záznam a panel se senzory, pomocí kterého můžeme registrované senzory připojovat ke stroji – viz dále).

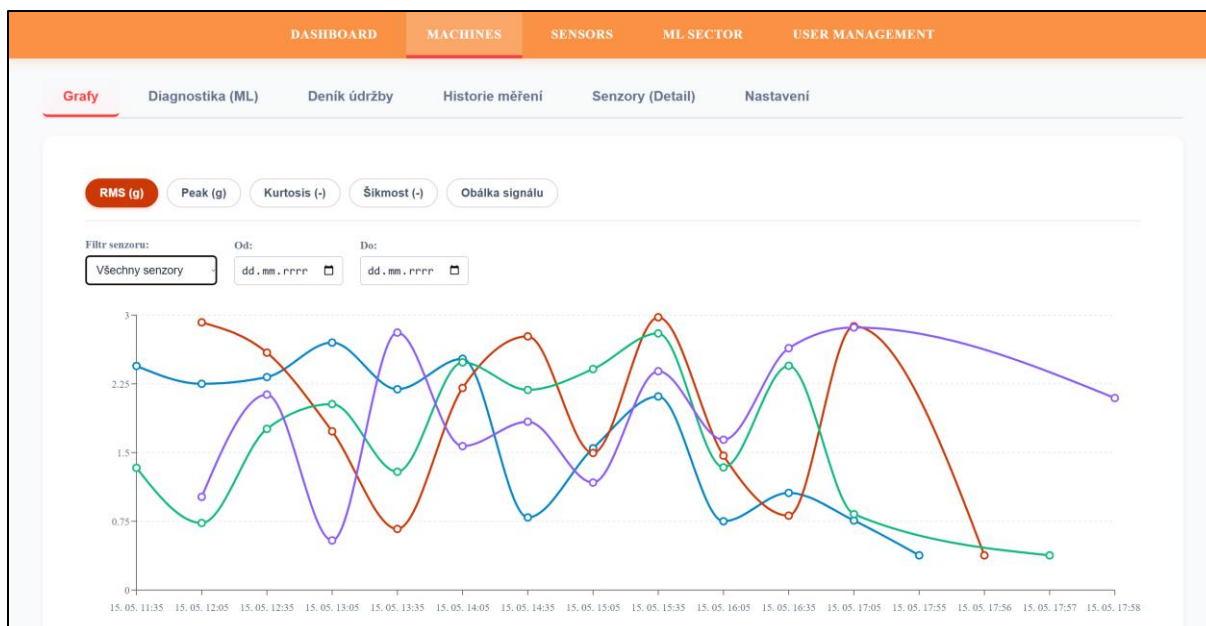


Obr. 22: Stránka se seznamem strojů aplikace.



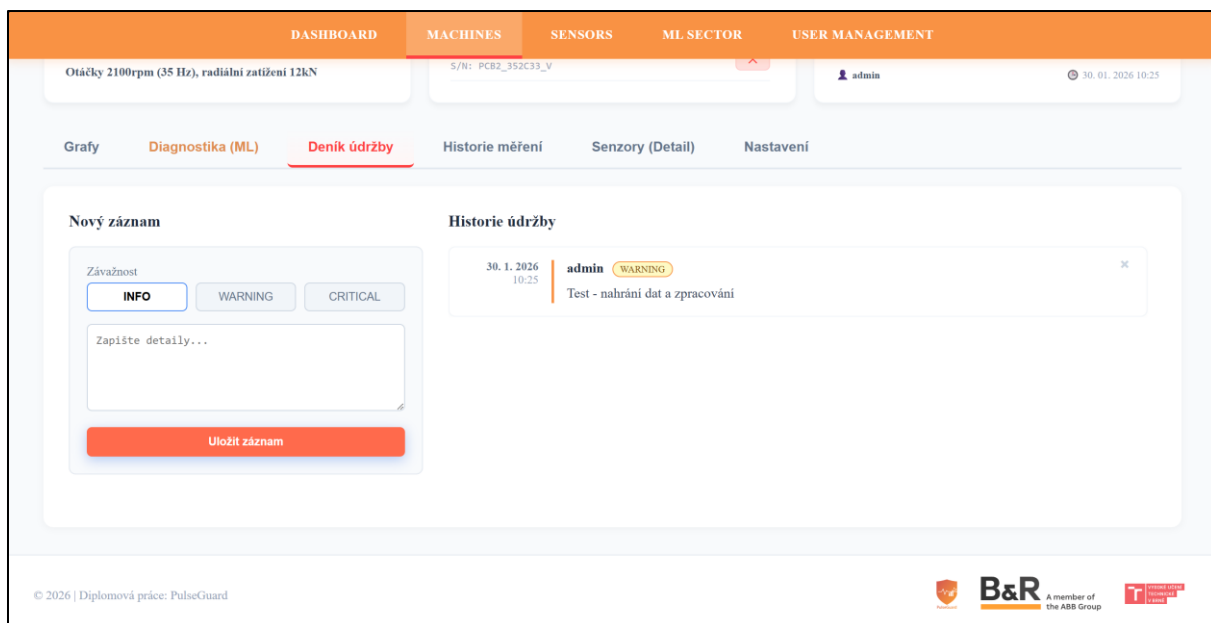
Obr. 23: Stránka s detailem stroje.

Pod těmito informačními panely se nachází stěžejní nabídka aplikačních funkcí pro daný stroj. První položkou v menu jsou grafy. Záložka obsahuje grafy průběhu jednotlivých charakteristických hodnot. Můžeme si vybrat hodnotu pro zobrazení a následně tuto hodnotu filtrovat dle jednotlivých senzorů a časového okna (obrázek 24). Další záložkou je Diagnostika (obrázek 25). Záložka s diagnostikou je vypovídající – spouští se zde analýzy stroje za pomoci jednotlivých modelů strojového učení. Následuje záložka „Deník údržby“, která slouží k přidávání servisních poznámek o stroji (obrázek 26). Implementovány jsou 3 úrovně závažnosti – INFO, WARNING (varování) a CRITICAL (kritická).



Obr. 24: Grafy charakteristických hodnot stroje.

Obr. 25: Sekce diagnostiky stroje.



Obr. 26: Deník údržby stroje.

Historie měření obsahuje tabulku se záznamy o jednotlivých měřeních (obrázek 27). Tato tabulka byla opět vybavena možností filtrace pro snadnější hledání záznamů. Jak můžeme vidět, tak tabulka rozlišuje zdroj dat. Prvním druhem dat jsou charakteristické hodnoty získané přímo z IIoT Connectoru, tyto data už mají svou finální podobu. Naopak záznamy surových dat můžeme dále zpracovávat. Po kliknutí na lupu v řádku měření se dostaneme do detailu měření. V tomto detailu musíme nejprve zpracovat data pomocí tlačítka „Spustit analýzu“, viz obrázek 28. Po zpracování dat vidíme v pravé části dopočítané charakteristické hodnoty a zároveň můžeme přepínat mezi grafy – buďto časový průběh amplitudy vibrací, frekvenční spektrum či časově-frekvenční spektrum (obrázek 29). Předposlední záložkou je tabulka senzorů (obrázek 30), funkčnost je obdobná jako u informačního panelu se senzory, můžeme zde otevřít detail senzorů a připojovat/odpojovat jednotlivé senzory. Poslední záložkou je nastavení (obrázek 31), které obsahuje konfiguraci připojení PLC ke stroji. Zapíná se zde automatický sběr surových dat ze stroje a také je zde možnost otestování FTP a OPC UA připojení na PLC.

DASHBOARD

MACHINES

SENSORS

ML SECTOR

USER MANAGEMENT

Grafy

Diagnostika (ML)

Deník údržby

Historie měření

Senzory (Detail)

Nastavení

Senzor:

Všechny senzory

Stav:

Všechny stavy

Datum:

dd.mm.rrrr

Zpracovat vybrané (0)

	Čas měření	Senzor	Zdroj dat	RMS [g]	Kurtosis	Stav	Akce
-	15. 05. 2026 17:58	AC0001459 4	Surový záznam	2.0952	-1.08	✓ Analyzováno	
-	15. 05. 2026 17:57	AC0001458 3	Surový záznam	0.3779	-0.11	✓ Analyzováno	
-	15. 05. 2026 17:56	AC0001457 2	Surový záznam	0.3779	-0.11	✓ Analyzováno	
-	15. 05. 2026 17:55	AC0001456 1	Surový záznam	0.3779	-0.11	✓ Analyzováno	
-	15. 05. 2026 17:05	AC0001459 4	IoT Connector	2.8650	2.53	✓ Analyzováno	
-	15. 05. 2026 17:05	AC0001456					

Obr. 27: Tabulka historie měření.

Detail měření #4 Spustit analýzu ×

Časový průběh (0.64 s) **Frekvenční spektrum (FFT)** **Časově-frekvenční (CWT)**

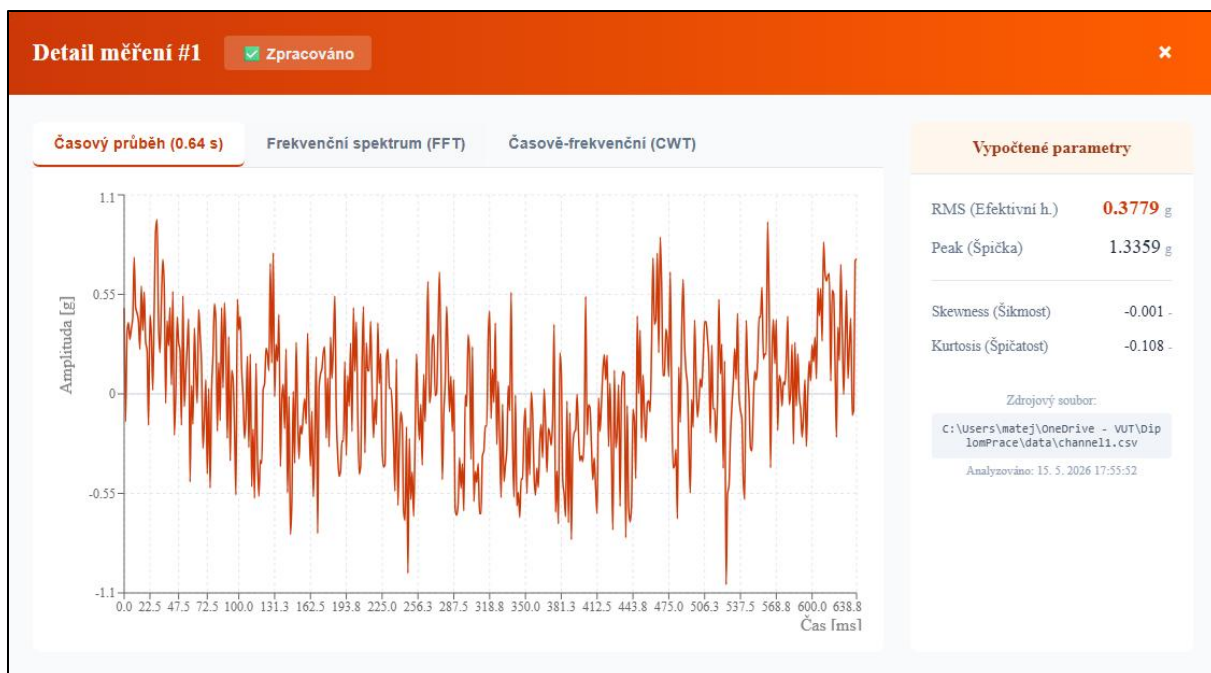


Nejprve data zpracujte
Použijte tlačítko 'Spustit analýzu' vpravo nahoře.

Vypočtené parametry

Data nebyla zpracována.
Hodnoty se zobrazí po spuštění analýzy.

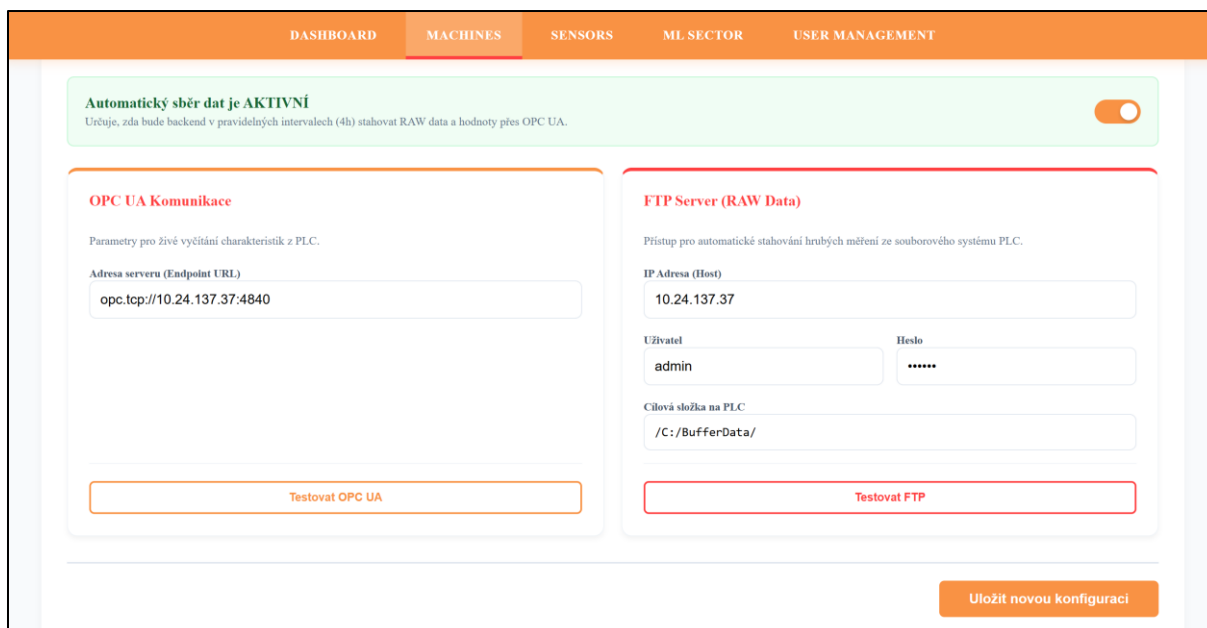
Obr. 28: Detail měření.



Obr. 29: Detail měření zpracovaných dat.

DASHBOARD	MACHINES	SENSORS	ML SECTOR	USER MANAGEMENT															
XJTU_Bearing_1_2 OK Zpět na seznam																			
Ložisko typu LDK UER204 • Hala 2																			
Technické údaje	Senzory (2) + Přidat	Poslední záznam																	
ID STROJE #5 POPIS ZARÍZENÍ Otáčky 2100rpm (35 Hz), radiální zatížení 12kN	Na ložisku v horizontálním směru S/N: PCB2_352C33_H × Na ložisku ve vertikálním směru S/N: PCB2_352C33_V ×	"Test - nahrání dat a zpracování" admin 30. 01. 2026 10:25																	
Grafy	Diagnostika (ML)	Deník údržby	Historie měření	Senzory (Detail)															
<table> <tr> <th>Sériové číslo</th><th>Model / Popis</th><th>Pozice na stroji</th><th>Status</th><th>Akce</th></tr> <tr> <td>PCB2_352C33_H</td><td>Na ložisku v horizontálním směru</td><td>Horizontal</td><td>AKTIVNI</td><td>Detail Odpojit</td></tr> <tr> <td>PCB2_352C33_V</td><td>Na ložisku ve vertikálním směru</td><td>Vertical</td><td>AKTIVNI</td><td>Detail Odpojit</td></tr> </table>					Sériové číslo	Model / Popis	Pozice na stroji	Status	Akce	PCB2_352C33_H	Na ložisku v horizontálním směru	Horizontal	AKTIVNI	Detail Odpojit	PCB2_352C33_V	Na ložisku ve vertikálním směru	Vertical	AKTIVNI	Detail Odpojit
Sériové číslo	Model / Popis	Pozice na stroji	Status	Akce															
PCB2_352C33_H	Na ložisku v horizontálním směru	Horizontal	AKTIVNI	Detail Odpojit															
PCB2_352C33_V	Na ložisku ve vertikálním směru	Vertical	AKTIVNI	Detail Odpojit															

Obr. 30: Tabulka se senzory.

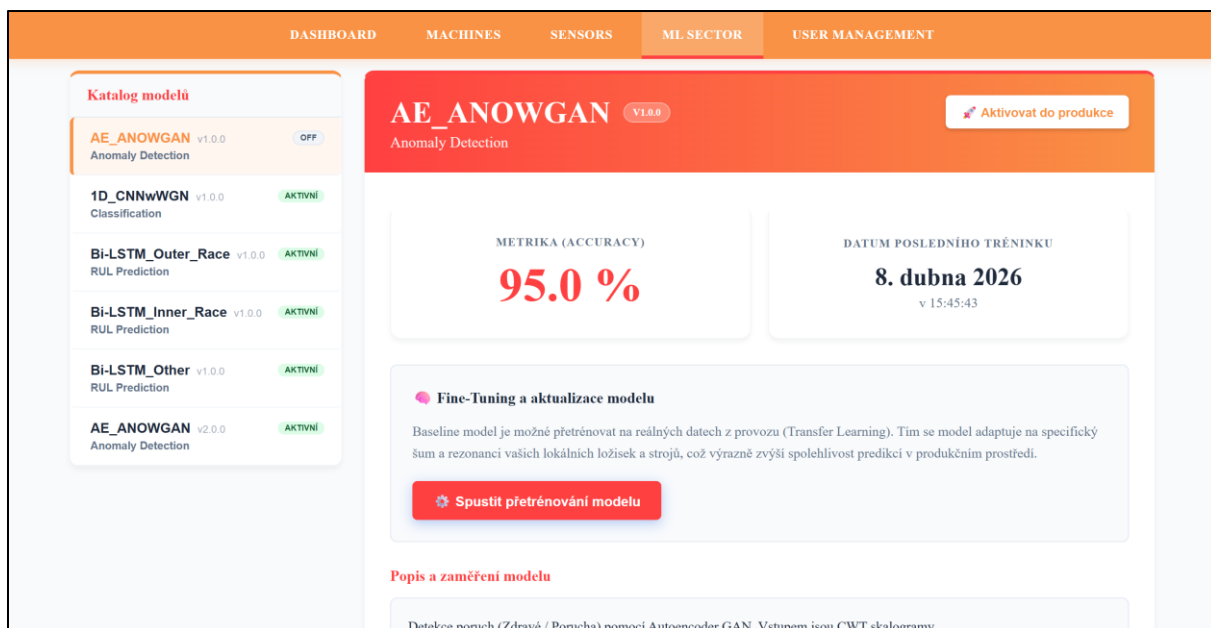


Obr. 31: Stránka s konfigurací připojení OPC UA a FTP na PLC server.

Další položkou v hlavní nabídce je záložka se senzory. Ta obsahuje, obdobně jako stránka se stroji, tabulku se všemi registrovanými senzory v aplikaci. Opět je zde tlačítko pro registraci nového senzoru.

Položka „USER MANAGEMENT“ pak slouží pro správu uživatelů, která je implementována jako tabulka se seznamem uživatelů s možností přidání uživatelů, kteří mohou mít 3 různé role – „admin“, „operator“ a „user“. Uživatelská role rozhoduje o možnostech uživatele v rámci aplikace. Admin má všechny pravomoci (přidávání nových uživatelů, strojů atd..), zatímco běžný uživatel může aplikaci pouze proklikávat a zobrazovat si informace.

Poslední a pro funkci aplikace zcela zásadní záložkou je „ML SECTOR“ (Machine Learning Sector – sektor strojového učení). Zde (obrázek 29) vidíme seznam různých verzí jednotlivých modelů s možností jejich aktivace do produkce. Důležitá je také sekce pro dotrénování modelů, bližší informace o tomto modulu viz kapitola o strojovém učení).



Obr. 32: Sekce strojového učení.

7.3 Produkční nasazení – kontejnerizace

Finální fáze nasazení webové aplikace vyžaduje přechod z vývojového prostředí (development) do optimalizované produkční formy. Na rozdíl od backendu, který běží jako aktivní proces v Pythonu, je React aplikace po dokončení vývoje zkompileována do sady statických souborů (HTML, CSS a JavaScript).

7.3.1 Proces kompilace a sestavení (Build)

Během procesu sestavení, který je v projektu iniciován nástrojem Vite (nebo Webpack), dochází k tzv. „minifikaci“ a „obfuskaci“ kódu. Tento proces odstraní nepotřebné znaky, komentáře a optimalizuje strukturu skriptů tak, aby výsledný balíček (bundle) byl co nejmenší a načítal se v prohlížeči operátora co nejrychleji. V této fázi jsou také do aplikace "zapečetěny" produkční proměnné prostředí (environment variables), jako je statická IP adresa backendu (např. <http://10.24.137.6:8005>), definovaná dříve v souboru `.env.production`.

7.3.2 Role webového serveru Nginx

Jelikož výsledkem buildu jsou pouze statické soubory, které samy o sobě nedisponují schopností síťové komunikace na portu HTTP, je do architektury začleněn webový server Nginx [39]. Tento server plní roli vysoce výkonného statického souborového serveru uvnitř Docker kontejneru.

Nginx přijímá požadavky z klientských prohlížečů (standardně na portu 80, resp. v našem případě na portu 8080 pro zamezení kolizí s B&R serverem) a servíruje jim příslušné části aplikace. Výhodou tohoto řešení je minimální náročnost na systémové prostředky a vysoká bezpečnost, kdy je klientský kód striktně oddělen od aplikační logiky backendu.

7.3.3 Kontejnerizace frontendu

Celá frontendová aplikace je zabalena do samostatného Docker obrazu (image). Proces sestavení v Dockerfile je řešen jako vícefázový (multi-stage build):

- Fáze sestavení: V dočasném kontejneru (Node.js) proběhne instalace závislostí a kompilace zdrojového kódu.
- Fáze produkce: Výsledné zkompilevané soubory jsou překopírovány do čistého, lehkého obrazu Nginx.

Tento přístup zajišťuje, že výsledný kontejner neobsahuje zbytečný balast (např. zdrojové kódy, vývojové nástroje), má minimální velikost a je snadno přenositelný na jakékoliv Edge zařízení v rámci průmyslové sítě.

8 IMPLEMENTACE MODULU PRO STROJOVÉ UČENÍ

Poslední mikroslužbou vyvinutou v rámci této práce je specializovaný modul pro strojové učení, v softwarové architektuře identifikovaný jako *ml_service*. Tento modul funguje jako analytické jádro celého vibrodiagnostického systému, které rozšiřuje předchozí systémové vrstvy o schopnost pokročilého zpracování dat a automatizovaného rozhodování.

Architektura použitých algoritmů přímo vychází z řešerše aktuální oborové literatury [40]. Pro účely této práce byly replikovány a naimplementovány modely, které v současnosti reprezentují stav techniky (state-of-the-art) v oblasti průmyslové diagnostiky rotačních strojů.

Celý diagnostický proces je koncipován jako sekvence tří na sebe navazujících úloh. Prvním krokem je včasná detekce anomálií, která má za úkol kontinuálně monitorovat stav stroje. Tuto fázi zajišťuje hybridní generativní model AE-AnoWGAN [2], který na základě naučené distribuce bezporuchových dat dokáže rozpoznat jakoukoliv odchylku od normálního provozu.

Pokud je detekováno nestandardní chování, proces přechází do druhé fáze, kterou je klasifikace poruchy. Zde je využita jednorozměrná konvoluční neuronová síť (1D-CNN) [18]. Její rolí je detailně analyzovat vibrační signály a přesně určit konkrétní typ vznikající závady, jako je například poškození vnitřního či vnějšího kroužku ložiska.

Diagnostický řetězec je následně završen predikcí zbývajících užitečné životnosti (RUL – Remaining Useful Life). Tuto třetí fázi obsluhuje obousměrná rekurentní síť typu Bi-LSTM [19]. Na základě vývoje charakteristických statistických parametrů v čase dokáže tento model odhadnout časový horizont, ve kterém dojde ke kritickému selhání monitorované komponenty.

8.1 Architektura mikroslužby a systémová integrace

Stejně jako u aplikačního rozhraní aplikace je i architektura mikroslužby pro strojové učení postavena na webovém frameworku FastAPI, v kombinaci s asynchronním serverem Uvicorn. V kontextu modulu pro strojové učení to znamená, že server může asynchronně přijímat a zařazovat nové požadavky na diagnostiku od hlavního backendu, zatímco na pozadí probíhá výpočetně náročnější zpracování a inference dříve přijatých dat.

Pro zajištění spolehlivé komunikace mezi oběma mikroslužbami je opět využita úzká integrace FastAPI s knihovnou Pydantic. Veškerá výměna dat, ať už jde o požadavky na detekci anomálií nebo odesílání objemných polí se senzorickými daty, probíhá výhradně přes definované datové objekty (Data Transfer Objects). Pydantic zajišťuje striktní typovou validaci těchto příchozích struktur. Pokud by hlavní backend odeslal nekompletní nebo chybně formátovaná data, mikroslužba požadavek zamítne ještě předtím, než by tato nekonzistentní data mohla způsobit selhání matematických operací uvnitř neuronových sítí.

Zásadním specifikem a rozšířením této mikroslužby je implementace prostředí pro hluboké učení, k čemuž byl jako hlavní nástroj zvolen framework PyTorch. Při návrhu

analytického modulu byly zvažovány i tradiční alternativy v podobě knihoven TensorFlow a Keras. Ekosystém PyTorch byl však upřednostněn především pro svou vyšší flexibilitu. Vzhledem k tomu, že implementované diagnostické modely – zejména pak generativní síť typu WGAN – vycházejí z komplexních akademických architektur, PyTorch nabízí podstatně intuitivnější rozhraní pro replikaci, úpravy a ladění těchto algoritmů než konkurenční řešení. Dalším podstatnou výhodou byla jeho nativní podpora masivně paralelních výpočtů. Podpora hardwarové akcelerace prostřednictvím technologie CUDA na grafické kartě NVIDIA byla využita při tréninku dále popsaných sítí.

Přístup k samotné inferenci neuronových sítí a dalším analytickým operacím je realizován pomocí aplikačních koncových bodů (endpointů), které jsou navrženy ve shodném konceptu jako v případě hlavního backendu. Toto řešení zachovává konzistentní strukturu rozhraní API napříč celým distribuovaným systémem. Konkrétní definice těchto tras, obsluha příchozích požadavků a jejich logické provázání s analytickými skripty jsou detailně implementovány v příloženém zdrojovém kódu této mikroslužby.

8.2 Principy a trénink diagnostických modelů

Všechny navržené modely byly kompletně implementovány v prostředí frameworku PyTorch, který poskytuje nezbytné programové zázemí pro definici vrstev, optimalizačních funkcí i pro samotné řízení trénovacího cyklu.

Vlastní fáze prvotního trénování a validace výchozích (baseline) modelů probíhala na hardwarové konfiguraci osazené grafickou kartou NVIDIA GeForce RTX 4060 Laptop GPU. Využití tohoto dedikovaného grafického akcelerátoru bylo klíčové pro zvládnutí vysoké výpočetní náročnosti, kterou vyžadují zejména generativní síť a hluboké konvoluční či rekurentní architektury, pracující s rozsáhlými poli senzorických dat.

S ohledem na budoucí praktické nasazení a plánovanou integraci modulu pro adaptivní dotrénování (fine-tuning) přímo v reálném provozu jsou však všechny zdrojové skripty navrženy jako hardwarově nezávislé. Na samém počátku každého výpočetního cyklu proto program automaticky ověřuje dostupnost akcelerace CUDA. Pokud je kompatibilní grafická karta v cílovém systému přítomna, jsou výpočty alokovány na ni, v opačném případě se provádění kódu plynule přesměruje na centrální procesor (CPU), což zajišťuje plnou přenositelnost softwaru na libovolnou průmyslovou platformu.

8.2.1 Detekce anomálií (AE-AnoWGAN)

Prvním a zcela zásadním krokem navrženého diagnostického řetězce je spolehlivá detekce anomálií. V oblasti průmyslové vibrodiagnostiky představuje dlouhodobě největší výzvu pro metody strojového učení kritický nedostatek dat reprezentujících různé chybové stavy, jelikož rotační stroje tráví drtivou většinu svého životního cyklu v nominálním, bezporuchovém provozu. Z tohoto důvodu byl pro tuto úvodní fázi zvolen a naimplementován model AE-AnoWGAN (Autoencoder-based Anomaly Wasserstein Generative Adversarial Network) [2]. Jeho stěžejní výhodou je skutečnost, že k efektivnímu tréninku vyžaduje výhradně vzorky zdravých dat. Model se učí detailně mapovat a rekonstruovat distribuci tohoto bezporuchového

stavu, díky čemuž je následně schopen identifikovat jakoukoliv odchylku jako anomálii, aniž by daný typ poruchy musel v trénovací sadě předtím vůbec „vidět“.

Dataset

Pro trénování a následné ověření úspěšnosti modelu AE-AnoWGAN byl zvolen celosvětově uznávaný dataset Paderbornské univerzity (PU dataset), který je v současné komunitě průmyslové diagnostiky považován za jeden z nejmodernějších benchmarků. Tento experimentální soubor se specificky zaměřuje na monitorování stavu valivých ložisek typu 6203. Jeho hlavní předností je, že vedle uměle indukovaných poruch obsahuje také ložiska s přirozeným mechanickým poškozením, kterého bylo dosaženo v rámci zrychlených životnostních zkoušek.

V rámci fáze předzpracování dat byla replikována metodika z referenčního článku. Surová vibrační data z akcelerometrů, snímaná na původní frekvenci 64 kHz, byla rozřezána na diskrétní segmenty o konstantní délce 1024 vzorků. Na tyto signálové úseky, z nichž každý zachycuje časový úsek o délce 0,016 sekundy, byla následně aplikována spojitá waveletová transformace (CWT) s využitím Morletovy vlnky. Tímto procesem byla jednorozměrná data transformována na dvourozměrné časově-frekvenční reprezentace (TFR) ve formě obrazových matic o rozlišení 256×256 pixelů. Pro zajištění konzistence s původní studií byly využity záznamy z bezporuchového ložiska K001 a ložiska KA01, které reprezentuje poškození vnějšího kroužku.

Tímto procesem segmentace a transformace bylo celkově vygenerováno 3920 obrazových datových oken pro optimální zdravý stav a 3871 oken pro stav s uměle indukovanou poruchou. V souladu s principem bezpříznakového (*unsupervised*) učení byla zdravá data rozdělena do trénovací a testovací množiny v poměru 70:30. Trénovací množina tak obsahuje 2744 výhradně čistých vzorků zdravého provozu, na kterých se model učí mapovat běžnou distribuci signálu. Zbývajících 1176 zdravých oken je alokováno do testovací sady, kde jsou pro účely vyhodnocení úspěšnosti detekce a stanovení optimálního rozhodovacího prahu smíchána se všemi 3871 anomálními vzorky.

Trénink

Základní stavební kámen detekční fáze představuje model AE-AnoWGAN. Tradiční síť typu GAN (Generative Adversarial Network) často trpí nestabilitou během tréninku a kolapsem módů, a proto byla zvolena robustnější varianta WGAN-GP (Wasserstein GAN s gradientní penalizací), která pro měření vzdálenosti mezi distribucí reálných a generovaných dat využívá Wassersteinovu metriku. Vnitřní topologie generátoru i diskriminátoru je tvořena pěti vrstvami hluboké konvoluční sítě (CNN), jež jsou optimalizovány pro zpracování dvourozměrných časově-frekvenčních reprezentací o rozlišení 256x256 pixelů. Vstupem pro generativní část je latentní prostor o pevně definovaném rozměru vektorů ($latent_dim = 100$), ze kterého se síť učí rekonstruovat komplexní příznakové mapy.

Pro zvýšení detekční spolehlivosti a eliminaci vlivu nevhodné inicializace vah nevyužívá systém pouze jeden izolovaný model, ale celý soubor (ensemble) navzájem se ovlivňujících podsítí. Architektura je tvořena maticí tří nezávislých generátorů a tří nezávislých

diskriminátorů. V první fázi učení probíhá takzvaný křížový trénink. Každý generátor se snaží oklamat všechny tři diskriminátory současně, zatímco každý diskriminátor se učí rozeznávat padělky pocházející od všech tří generátorů. Tento komplexní kompetitivní přístup zajišťuje, že si generátory osvojí vysoce přesnou a stabilní mapu latentního prostoru, jež odpovídá výhradně optimálnímu bezporuchovému chodu stroje.

Samotné nastavení trénovacího procesu bylo parametrizováno s ohledem na zajištění stabilní konvergence. Optimalizační cyklus probíhal po dobu 50 epoch, přičemž velikost mini-batche byla pro efektivní distribuci gradientů v paměti grafického akceleratoru stanovena na 64 vzorků. Jako optimalizátor byl použit algoritmus Adam s konstantní rychlostí učení 0.0002. Matematickou stabilizaci diskriminátoru (kritika) zajišťuje gradientní penalizace s váhovým koeficientem $\lambda_{gp} = 10$, přičemž pro zachování správného poměru rychlosti učení mezi soupeřícími sítěmi je diskriminátor aktualizován pětikrát častěji než generátor, což definuje parametr $n_{critic} = 5$.

Jakmile je latentní prostor stabilizován, přechází výpočetní proces do druhé, sdílené fáze tréninku (Joint Training). V tomto kroku je do systému integrován enkodér, jehož úkolem je transformovat reálné vibrační snímky zpět do komprimovaného latentního prostoru, přičemž předtrénovaný generátor v této fázi přebírá roli dekodéru. V každém kroku optimalizace je náhodně vybrán jeden pár enkodér-dekodér a jeden diskriminátor. Enkodér komprimuje reálný vstup, dekodér se jej pokusí zrekonstruovat a diskriminátor následně posuzuje míru shody mezi originálem a touto rekonstrukcí.

Klíčovým aspektem této finální fáze je definice kombinované ztrátové funkce, na jejímž základě dochází k úpravě vah sítě. Tato celková penalizace se skládá ze tří nezávislých složek. První složkou je adversariální ztráta, která nutí model generovat vizuálně přesvědčivé výstupy. Druhou složku tvoří standardní rekonstrukční chyba definovaná střední kvadratickou odchylkou (MSE) mezi vstupním a výstupním snímkem. Třetí a nejsofistikovanější složkou je tzv. feature matching loss, která neporovnává snímky po pixelech, ale zkoumá shodu ve vnitřních příznakových mapách extrahovaných z hlubokých vrstev diskriminátoru, čímž zajišťuje shodu na úrovni vysokofrekvenčních strukturálních detailů signálu.

Výpočet anomálního skóre a inference v aplikaci

Fáze inference neboli proces vyhodnocování nových a neznámých dat ze snímačů, využívá již pouze plně natrénovaný enkodér, dekodér (původní generátor) a diskriminátor. Pro zvýšení predikční robustnosti a eliminaci falešných poplachů je v produkčním prostředí *ml_service* nasazen soubor (tzv. ensemble) tří nezávisle natrénovaných modelů AE-AnoWGAN.

Samotný proces diagnostiky v rámci softwarové architektury je inicializován uživatelským požadavkem v klientském grafickém rozhraní. Po stisknutí diagnostického tlačítka systém vyhledá poslední dostupný datový záznam pro daný stroj v databázi a prostřednictvím backendu odešle cestu k surovým vibračním datům do mikroslužby pro strojové učení. Zde je signál načten a následně transformován do podoby časově-frekvenční reprezentace pomocí spojitě vlnkové transformace (CWT). Tento 2D obraz je následně paralelně zpracován všemi třemi enkodéry, které jej zkomprimují do latentních vektorů, jež jsou dekodéry zrekonstruovány zpět do původní dimenze.

Na základě míry shody mezi vstupním a zrekonstruovaným snímkem je vypočítáno takzvané anomální skóre. Tato výsledná skalární metrika je definována jako vážený součet dvou komponent: primární rekonstrukční chyby (hodnotící střední kvadratickou odchylku na úrovni jednotlivých pixelů) a sekundární diskriminační chyby (vyhodnocující rozdíl v hlubokých příznakových mapách diskriminátoru). Výsledné anomální skóre celého systému je stanoveno jako aritmetický průměr skóre ze všech tří sub-modelů. Kombinace těchto přístupů zajišťuje vysokou citlivost modelu i na zcela minoritní strukturální odchylky v signálu.

Pro zajištění automatizovaného chodu diagnostického systému bez nutnosti expertního posuzování výsledků je nezbytné stanovit exaktní rozhodovací práh (*decision threshold*). K jeho počáteční kalibraci byla využita vyčleněná testovací množina obsahující výhradně zdravá data, se kterými se model v průběhu tréninku nesetkal. Na základě hranice tohoto referenčního rozložení byl pro produkční nasazení v aplikaci rozhodovací práh fixně stanoven na hodnotu 0,75.

Při každé nové analýze je nově vypočítané anomální skóre porovnáváno právě s tímto nakalibrovaným prahem. Pokud je hodnota skóre nižší než stanovená hranice, algoritmus klasifikuje stav monitorovaného stroje jako normální. Vyhodnocený výsledek je odeslán zpět do hlavní backendové služby, která jej trvale zapíše do databáze a přepošle do uživatelského rozhraní k vizualizaci. V momentě, kdy anomální skóre tento práh překročí, tedy model spolehlivě identifikuje cizí distribuci v datech. V takovém případě analytický uzel zaznamená výskyt anomálie a diagnostický proces se v rámci aplikace automaticky přesouvá do své druhé fáze, kterou je klasifikace konkrétního typu závady pomocí 1D-CNN sítě.

8.2.2 Klasifikace poruch (1DCNN)

Jakmile úvodní článek diagnostického řetězce (AE-AnoWGAN) úspěšně detekuje nestandardní chování stroje, přechází proces plynule do své druhé fáze, kterou je určení konkrétního typu závady. Pro tuto klasifikační úlohu byl zvolen a naimplementován model lehké jednorozměrné konvoluční neuronové sítě (1D-CNN), který vychází z vnitřní architektury navržené ve studii Hakim et al. [18]. Jeho stěžejní výhodou pro nasazení v reálných aplikacích je výjimečná schopnost extrakce hlubokých příznaků přímo z frekvenční domény. Namísto konvenčního zpracování časových řad model přijímá na svém vstupu dvoukanálová frekvenční spektra (amplitudu a fázi) transformovaná rychlou Fourierovou transformací (FFT) ze surového vibračního signálu. Tento přístup, podpořený robustní augmentací dat pomocí posuvného okna s vysokým překryvem, umožňuje síti identifikovat zcela izolované a dominantní poruchové harmonické frekvence, čímž eliminuje nutnost složitého manuálního předzpracování. Díky záměrně nízkému počtu vnitřních parametrů tak architektura zaručuje nejen vysokou přesnost, ale i krátkou dobu odezvy při diagnostice v reálném čase.

Dataset

Pro trénování a následné ověření úspěšnosti klasifikačního modelu 1D-CNN byl zvolen celosvětově uznávaný dataset Case Western Reserve University (CWRU dataset), který je v oblasti vibrodiagnostiky považován za klasický zlatý standard a nejrozšířenější benchmark pro testování algoritmů řízeného učení. Tento experimentální soubor obsahuje záznamy

z testovacího stavu, který se skládal z elektromotoru o výkonu dvou koňských sil, momentového snímače, výkonové elektroniky a zátěžového dynamometru. Surová vibrační data byla snímána pomocí akcelerometrů umístěných na domku ložiska na straně pohonu (*drive end*) za definovaných provozních podmínek při různých úrovních mechanického zatížení.

V rámci fáze předzpracování dat byla implementována metodika odpovídající principům referenčního článku, která spočívá v přímé transformaci jednorozměrných časových řad do frekvenční domény. Kontinuální vibrační signály, snímané se vzorkovací frekvencí 12 kHz, byly rozřezány pomocí metody posuvného okna o konstantní délce 1024 vzorků se zvoleným překryvem 75 %, což slouží jako efektivní augmentace dat. Na každý takto získaný časový rámec byla následně aplikována rychlá Fourierova transformace (FFT), s jejímž využitím bylo získáno amplitudové a fázové spektrum signálu. Tyto dvě spektrální složky byly uspořádány do podoby dvoukanálového jednorozměrného vektoru o rozměru 2x512 prvků (odpovídající polovině frekvenčního spektra po Nyquistovu frekvenci), který vstupuje přímo do první konvoluční vrstvy klasifikátoru.

Významnou modifikací oproti původní studii, kde byla data rozřazována do devíti specifických skupin v závislosti na exaktním geometrickém průměru defektů, je účelové sloučení dat do čtyř základních diagnostických tříd při konstantním středním zatížení elektromotoru o velikosti 1 HP. Pro zvýšení schopnosti generalizace modelu byly v rámci každé defektní třídy sloučeny tři různé velikosti mechanických vad (0,007, 0,014 a 0,021 palce). Tímto procesem segmentace signálu s vysokým překryvem bylo celkově vygenerováno 1887 oken pro zdravý stav (třída Normal), 1416 oken pro poruchu vnitřního kroužku (třída IR), 1417 oken pro poruchu valivého elementu (třída Ball) a 1422 oken pro poruchu vnějšího kroužku (třída OR). V souladu s nastaveným experimentálním schématem byla tato data rozdělena na trénovací a testovací množinu v poměru 70:30. Trénovací sada tak obsahuje 1320 zdravých vzorků a shodně 991 (pro IR a Ball), respektive 995 (pro OR) vzorků poškození, zatímco zbývající část vyčleněných oken (567 pro Normal, 425 pro IR, 426 pro Ball a 427 pro OR) slouží jako nezávislá testovací sada pro validaci celkové přesnosti klasifikátoru.

Trénink

Vnitřní topologie navržené sítě je striktně optimalizována pro minimalizaci výpočetní náročnosti při zachování vysoké klasifikační přesnosti. Vstupem do modelu je dvoukanálový tenzor reprezentující normalizovanou amplitudu a fázi frekvenčního spektra. První konvoluční vrstva (C1) využívá jádro o velikosti 16 vzorků s délkou kroku 4 k extrakci dvaceti nezávislých příznakových map, které jsou vzápětí nelineárně transformovány pomocí aktivační funkce ReLU. Druhá konvoluční vrstva (C2), navazující přímo na výstup první vrstvy, využívá zmenšené jádro o velikosti 8 a totožný krok 4. Tato vrstva zahušťuje prvotní frekvenční rysy do padesáti komplexnějších abstrakcí, čímž model získává schopnost identifikovat specifické harmonické složky charakteristické pro jednotlivé typy poškození.

Aby byla síť chráněna před přeučením a došlo k radikální redukci trénovatelných parametrů, je za konvolučními bloky zařazena adaptivní průměrovací poolingová vrstva (Adaptive Average Pooling). Tato výpočetní operace redukuje každou z padesáti příznakových

map na jedinou průměrnou hodnotu, čímž vzniká vysoce kompaktní jednorozměrný vektor o délce 50 prvků. Tato redukce dimenzionality navíc zajišťuje teoretickou nezávislost sítě na přesné délce vstupního signálu. Klasifikační řetězec je následně završen jedinou plně propojenou (Dense) vrstvou, která tento kompaktní vektor lineárně transformuje na čtyři výstupní logity, jež odpovídají definovaným diagnostickým třídám (Normal, IR, Ball, OR).

Během fáze trénování byl pro optimalizaci vah v síti využit algoritmus stochastického gradientního sestupu (SGD) s momentem nastaveným na hodnotu 0,9 a počáteční rychlostí učení 0,01. Pro vyčíslení celkové odchylky modelu od skutečných štítků sloužila funkce křížové entropie (Cross-Entropy Loss), jež v sobě efektivně integruje výpočet pravděpodobností pomocí funkce Softmax. Optimalizační cyklus byl nastaven na 100 epoch s velikostí dávky (batch size) 64 vzorků. Jak ukázaly experimentální výsledky, zvolená strategie sloučení různých geometrií vad do čtyř hlavních kategorií v kombinaci se 75% překryvem oken vedla k extrémně rychlé konvergenci. Model překročil hranici 99% úspěšnosti na testovací sadě již kolem 30. epochy a v dalších fázích tréninku se jeho chybovost limitně blížila nule bez jakýchkoliv známek přeučení (overfittingu), což prokazuje vysokou robustnost a vhodnost navržené architektury pro nasazení v reálných průmyslových podmínkách.

Výstup modelu a inference v aplikaci

V okamžiku, kdy je proces spuštěn manuálně z uživatelského rozhraní, se přesouvá diagnostika do své druhé fáze. Příslušný datový záznam je odeslán do analytického uzlu mikroslužby (ml_service), kde probíhá jeho klasifikace pomocí 1D-CNN. Surový vibrační signál je zde nejprve délkově standardizován na exaktní rozměr 4096 vzorků. Takto připravený signál je následně transformován z časové do frekvenční oblasti prostřednictvím rychlé Fourierovy transformace (FFT). Vzniklé komplexní spektrum je algoritmicky rozděleno na amplitudovou a fázovou složku. Amplitudová část je statisticky normalizována (tj. je odečten průměr a výsledek je vydělen směrodatnou odchylkou) a fázová složka je škálována konstantou π . Tyto dvě upravené reprezentace signálu jsou nakonec sloučeny do dvoukanálového jednorozměrného tenzoru, jenž tvoří přímý vstup do hluboké konvoluční sítě.

Finální fáze klasifikačního procesu spočívá v transformaci surových výstupních hodnot sítě (tzv. logitů) do formátu, který je snadno interpretovatelný pro koncového uživatele či řídicí systém. Výstupní vektor z poslední plně propojené vrstvy je podroben normalizační aktivační funkci Softmax, která tyto nelimitované reálné hodnoty matematicky převede na diskrétní rozdělení pravděpodobností. Výsledkem je čtveřice procentuálních hodnot, jejichž součet je roven přesně 100 %, přičemž každá hodnota reprezentuje míru jistoty modelu o příslušnosti analyzovaného signálu k jedné ze čtyř definovaných tříd (Normal, IR, Ball, OR).

Samotné diagnostické rozhodnutí je následně provedeno výběrem té kategorie, která vykazuje maximální predikovanou pravděpodobnost. Zjištěná třída závady je společně s její mírou jistoty (tzv. *confidence score*) formátována do standardizované struktury a odeslána zpět backendu. Tento mechanismus nejenže poskytuje exaktní kategorizaci zjištěné závady, ale obratem informuje operátora údržby v grafickém rozhraní o jistotě dané predikce, což je z hlediska reálné průmyslové praxe klíčový parametr zabraňující falešným poplachům a zbytečným servisním zásahům.

8.2.3 Predikce RUL (Bi-LSTM)

Třetí a finální článek navrženého diagnostického řetězce se zaměřuje na oblast prognostiky, konkrétně na predikci zbývající užitečné životnosti (Remaining Useful Life – RUL) valivých ložisek. Zatímco referenční studie Yao et al. [19] využívá pro prvotní zpracování surových vibračních signálů hluboký konvoluční odšumovací autoenkodér, v této práci byl implementován inženýrsky optimalizovaný přístup založený na přímé doménové extrakci klíčových příznaků. Jako vstupní reprezentace pro predikční model byly záměrně zvoleny časové řady vektorů charakteristických statistických a diagnostických hodnot, konkrétně efektivní hodnota zrychlení vibrací (RmsAccRaw), špičatost (KurtosisRaw), šikmost (SkewnessRaw), efektivní hodnota obálky signálu (RmsAccEnvelope), diagnostický faktor Vdi3832KtRaw a aktuální provozní otáčky (ActSpeed). Tento specificky navržený příznakový vektor šesti veličin de facto plní analogickou roli jako enkodér v původním článku, avšak s tou výhodou, že pracuje s jasně definovanými, fyzikálně interpretovatelnými parametry získanými přímo z IIoT infrastruktury. Skládáním těchto vektorů do sekvenčních časových oken získává obousměrná síť LSTM (Bi-LSTM) komplexní kontextuální přehled o dosavadním trendu degradace ložiska, což z této datové struktury činí vysoce vhodný a stabilní základ pro exaktní modelování vývoje stárnutí mechanické komponenty.

Dataset

Pro trénování a verifikaci predikčního modelu Bi-LSTM byl zvolen experimentální dataset Xi'an Jiaotong University (XJTU-SY), který je v komunitě strojového učení považován za mezinárodní standard pro úlohy predikce zbývající životnosti (run-to-failure). Tento soubor obsahuje data ze zrychlených životnostních zkoušek valivých ložisek, jež byla za definovaných provozních podmínek (variabilní otáčky a radiální zatížení) kontinuálně monitorována od počátečního zdravého stavu až po totální mechanické selhání. Surové vibrační signály byly zaznamenávány v pravidelných časových intervalech, což umožňuje podrobně sledovat evoluci a šíření strukturních defektů v čase.

V rámci fáze předzpracování dat byla implementována metodika transformace surových signálů na kompaktní vektorový tok, který simuluje reálný provozní výstup v rámci navržené IIoT infrastruktury. Z každého časového snímku vibračního signálu byla extrahována šestice diagnosticky významných příznaků: efektivní hodnota (RmsAccRaw), špičatost (KurtosisRaw), šikmost (SkewnessRaw), energie obálky (RmsAccEnvelope), diagnostický faktor Vdi3832KtRaw a aktuální otáčky (ActSpeed). Tento proces redukce dat eliminuje redundanci surových signálů a poskytuje modelu Bi-LSTM vysoce informativní vstupní vektor, který v čase transparentně mapuje degradační proces ložiska.

Pro potřeby tréninku a validace byly tyto extrahované příznakové vektory seskupeny do sekvenčních oken o fixní délce 10 kroků, přičemž každé okno reprezentuje krátkodobou dynamiku vývoje poruchy. V rámci experimentu byla data rozdělena do tří hlavních kategorií podle typu vady: poškození vnějšího kroužku (OR), vnitřního kroužku (IR) a ostatní defekty valivých elementů (O). Celkový objem takto připravených trénovacích dat čítá tisíce sekvenčních vzorků, přičemž pro každé okno je vypočtena normalizovaná hodnota RUL v intervalu $\langle 0,1 \rangle$, kde 1 značí stav nového ložiska a 0 reprezentuje kritické selhání. Tento

sekvenční datový soubor byl pro každý model (kategorii) rozdělen v poměru 80:20 na trénovací a testovací množinu, přičemž rozdělení bylo provedeno striktně podle časové osy, aby model nebyl vystaven datům z budoucnosti a mohl se reálně naučit predikovat trend degradace.

Trénink

Pro vlastní predikční úlohu byla zvolena architektura obousměrné sítě s dlouhou krátkodobou pamětí (Bi-LSTM), která je optimálně uzpůsobena pro analýzu sekvenčních závislostí v čase. Jádrem modelu jsou dvě vrstvy Bi-LSTM buněk, každá s 64 vnitřními stavy, které umožňují síti efektivně zaznamenávat dynamiku vývoje poruchy v obou směrech časové osy – tedy nejen z minulosti do budoucnosti, ale i v kontextu celkového trendu degradace. Výstupní sekvence z poslední vrstvy je dále transformována pomocí plně propojených vrstev (Dense), které finální příznakový stav mapují na skalární hodnotu RUL v rozsahu $\langle 0,1 \rangle$. Proces trénování modelu byl definován následujícím postupem:

1. **Strukturování dat:** Vstupní data o šesti parametrech byla seskupena do časových oken o délce 10 kroků (`WINDOW_SIZE=10`), což tvoří základní jednotku (matice 10×6), ze které síť vyvozuje predikci.
2. **Časové rozdělení:** Dataset byl rozdělen v poměru 80 % pro trénovací fázi a 20 % pro validaci, přičemž toto rozdělení bylo provedeno sekvenčně podle časové osy. Tímto přístupem bylo zajištěno, že model při tréninku nikdy nepracuje s informacemi z budoucích stavů, což je pro korektní modelování predikce životnosti naprosto zásadní.
3. **Optimalizace a ztrátová funkce:** K minimalizaci chyby mezi predikovanou a skutečnou hodnotou RUL byla využita funkce střední kvadratické chyby (MSE Loss), která silně penalizuje i menší odchylky v odhadech. Optimalizaci vah v síti zajišťoval adaptivní algoritmus *Adam* s rychlostí učení nastavenou na hodnotu 0,005, což zajistilo plynulou a stabilní konvergenci.
4. **Trénovací cyklus:** Celý proces probíhal v 100 epochách s velikostí dávky (batch size) 32 vzorků. V každé epoše se model snažil adaptovat své vnitřní parametry tak, aby minimalizoval rozdíl mezi predikovanou a reálnou zbývajícím životností napříč celou testovací množinou, čímž postupně vytvořil robustní pravidla pro odhad trendu degradace ložiska.

Tato architektura, v kombinaci s cíleným výběrem vstupních vektorů, umožňuje modelu dosahovat vysoké míry predikční přesnosti, přičemž grafické výstupy pro jednotlivá ložiska potvrzují, že model věrně kopíruje fyzikální proces stárnutí komponenty a včas indikuje blízký se konec její technické životnosti.

Výstup modelu a inference v aplikaci

Třetí, závěrečná fáze diagnostického řetězce nastává v momentě, kdy uživatel provede ve vizualizaci požadavek na odhadnutí zbývajícím životnosti. Poněvadž predikční model Bi-LSTM vyžaduje pro zachycení časových závislostí a trendu degradace sekvenční kontext, hlavní backend agreguje z databáze časovou řadu deseti po sobě jdoucích historických měření dané komponenty. Každé z těchto měření je reprezentováno šestiprvkovým příznakovým vektorem. Vzniklá vstupní matice o exaktních rozměrech 10×6 je následně společně s identifikovanou

kategorií poškození odeslána prostřednictvím POST požadavku na endpoint /predict-rul mikroslužby pro strojové učení.

V rámci tohoto inferenčního uzlu dochází k okamžitému lokálnímu předzpracování přijatého časového okna. Hodnoty v matici jsou normalizovány pomocí algoritmu *MinMaxScaler*, což zajišťuje, že dynamické rozsahy jednotlivých diagnostických veličin striktně odpovídají distribuci dat použité při trénování sítě. Škálovaná sekvence je následně převedena na PyTorch tenzor a doplněna o dimenzi dávky (batch size), čímž vzniká trojrozměrný tenzor o tvaru (1, 10, 6), reprezentující strukturu (Dávka, Délka sekvence, Počet příznaků). Podle přijaté kategorie je z paměti vyvolán příslušný specializovaný model Bi-LSTM. Výstupní skalární hodnota je z důvodu procesní bezpečnosti softwarově omezena (clamping) do striktního intervalu $\langle 0,0; 1,0 \rangle$ což eliminuje riziko vzniku nereálných mimorozsahových hodnot v okrajových fázích životního cyklu ložiska.

Výstupem každého ze tří specializovaných modelů je tak spojitá hodnota, která přímo reprezentuje normalizovanou zbývajících užitečnou životnost ložiska. Hodnota 1,0 odpovídá stavu nového, bezvadného ložiska, zatímco hodnota 0,0 indikuje stav kritického poškození, při kterém je nutné komponentu neprodleně vyřadit z provozu. Tato škála poskytuje inženýrům intuitivní přehled o progresi degradačních procesů specifických pro daný typ poruchy, čímž umožňuje plánování údržby na základě aktuálního technického stavu namísto pevných časových intervalů.

Pro usnadnění interpretace a nasazení v reálném průmyslovém provozu je tato bezrozměrná výstupní hodnota z intervalu $\langle 0,0; 1,0 \rangle$ v rámci hlavní backendové služby transformována na absolutní časový údaj. Získaná normalizovaná frakce je algoritmicky přepočtena na predikovaný počet zbývajících dní do selhání. V rámci aktuální softwarové implementace se využívá lineární transformace, při níž je výstup sítě násoben referenční konstantou 30 dní, jež představuje stanovený maximální diagnostický horizont od počátku detekovatelné degradace po kritické selhání. Výsledná predikce, zaokrouhlená na jedno desetinné místo, je následně provázána s identifikačním číslem posledního měření, trvale uložena do primární databáze (analysis_results) a poskytnuta klientskému rozhraní. Na základě tohoto exaktního časového údaje je následně v uživatelském prostředí aplikována prahová logika pro vizuální varování operátora. Pokud systém predikuje vysokou zbývajících životnost (např. nad ekvivalent 75 %, tedy více než 22 dní), stav je klasifikován jako nominální. Při rychlém poklesu pod kritickou mez (např. pod 25 %, resp. 7,5 dne) systém automaticky generuje požadavek na urgentní servisní prohlídku či výměnu komponenty, čímž efektivně předchází neplánovaným haváriím a minimalizuje finanční ztráty způsobené odstávkou výrobního procesu.

8.3 Fine-tuning modul

Během fází experimentálního tréninku a následné evaluace diagnostických modelů bylo zjištěno, že spolehlivost a celková přesnost výsledné vibrodiagnostické analýzy jsou kriticky závislé na parametrech konkrétní průmyslové implementace. Přenositelnost modelů natrénovaných výhradně na standardizovaných laboratorních datasetech (jako je XJTU-SY)

do reálného provozu bývá často limitována technologickými a fyzikálními odlišnostmi dané aplikace. Charakter a dynamika snímaného vibračního signálu jsou totiž nevyhnutelně ovlivněny celou řadou provozních faktorů. Mezi ty nejzásadnější patří specifický konstrukční typ použitého ložiska, nastavená vzorkovací frekvence měřicího řetězce, zvolená velikost analyzovaného datového okna a především reálné podmínky, jako jsou proměnlivé otáčky či dynamické mechanické zatížení konkrétního stroje.

Z těchto důvodů se jako optimální a nejrobustnější řešení pro dlouhodobé nasazení ukazuje průběžná adaptace předtrénovaných modelů přímo na specifika monitorovaného zařízení. K tomuto účelu slouží technika dodatečného učení, tzv. fine-tuning, která využívá reálná data kontinuálně získávaná z produkčního prostředí.

Jako přímá reakce na tuto inženýrskou výzvu byl do architektury vyvíjeného diagnostického systému implementován specializovaný MLOps (Machine Learning Operations) modul pro fine-tuning. Tento nástroj poskytuje aplikační rozhraní, pomocí kterého může systém proaktivně reagovat na odlišnosti v provozních datech. Modul umožňuje operátorům interaktivně selektovat relevantní historická měření a automatizovaně spouštět proces dotrénování jednotlivých sítí, čímž aplikace získává klíčovou schopnost průběžně se učit a neustále zpřesňovat svou diagnostiku na základě reálných pracovních podmínek.

8.3.1 Fine-tuning modelu pro detekci anomálií

Do sekce fine-tuningu se dostaneme přes seznam modelů v záložce „ML SECTOR“ hlavního menu – viz obrázek 32 v kapitole a webové vizualizaci. Pokud u příslušného modelu klikneme na tlačítko „Spustit přetrénování modelu“, spustí se nám karta pro dotrénování modelu. Pro modely type AE-AnoWGAN pro detekci anomálií zde stačí pouze vybrat jednotlivé měření surových dat, ve kterých bylo ložisko ve zdravém stavu a spustit dotrénování modelu. Obrázek 33 představuje diskutovanou kartu. Pro snadnější vyhledávání datových oken je zde implementována filtrace dat podle stroje, senzoru a data.


Konfigurace tréninku
×

Model:
AE_ANOWGAN_Production v1

Režim Unsupervised (Baseline)
Vybírejte pouze úseky bezporuchového stavu stroje.

Vyhledat data v databázi ← Zpět

Stroj: **Všechny stroje**

Senzor: **Všechny senzory**

Od: dd.mm.rrrr --:--

Do: dd.mm.rrrr --:--

Hledat

Stroj / Senzor	Časový úsek	Vzorků	Akce
B&R Kostka 4	2026-05-15 15:58 2026-05-15 15:58	1	Přidáno ✓
B&R Kostka 3	2026-05-15 15:57 2026-05-15 15:57	1	+ Přidat
B&R Kostka 2	2026-05-15 15:56 2026-05-15 15:56	1	+ Přidat

Zrušit
Spustit Fine-Tuning

Obr. 33: Karta konfigurace dotrénování modelu pro detekci anomálií.

Samotné trénovací skripty jsou součástí modulu strojového učení. Po spuštění Fine-Tuningu jsou cesty k souborům vybraných datových oken odeslány backendem do modulu strojového učení, kde jsou zpracovány a následně je na nich zvolený model dotrénován a jsou upraveny jeho váhy.

8.3.2 Fine-Tuning modelu pro klasifikaci poruch

Podobná funkčnost je implementována i pro dotrénování modelu pro klasifikaci poruch. Jeho karta je na obrázku 34. Z obrázku je patrný rozdíl od předchozího modelu, kdy se nyní jedná o učení s učitelem (Supervised Learning). To spočívá v tom, že jednotlivá časová okna surového signálu je potřeba označit konkrétní poruchou. Ve sloupci „Štítek (Label)“ vybereme nabízenou poruchu.

Konfigurace tréninku
×

Model:

1D_CNN_Production v1

Režim Supervised (Labelování)
 Přiřaďte úsekům typy poruch pro učení klasifikátoru.

Vybrané datové úseky

+ Přidat data

Stroj / Senzor	Období měření	Vzorků	Štítek (Label)	Odstranit
B&R Kostka 4	15. 5. 2026 15:58:00	1	Zdravé ložisko (Normal)	×
B&R Kostka 3	15. 5. 2026 15:57:00	1	Zdravé ložisko (Normal) Porucha vnitřního kroužku (IR) Porucha vnějšího kroužku (OR) Porucha valivého elementu / klece - other (O)	×

Zrušit

Spustit Fine-Tuning

Obr. 34: Konfigurace fine-tuningu modelu pro klasifikaci poruch.

Po spuštění Fine-Tuningu jsou opět backendem vyhledány v databázi cesty k csv souborům. K cestám jsou přidány zvolené štítky a jako balíček jsou odeslány do modulu strojového učení, který znovu data zpracuje a předá skriptu pro dotrénování modelu.

8.3.3 Fine-tuning modelu modelu pro predikci životnosti

Specifikem tohoto modelu je potřeba znalosti životního cyklu hlídaného ložiska. Jako je vidět na obrázku 35 je potřeba vybrat datum instalace ložiska a datum jeho výměny. Poslední údaj, tedy v jakém čase z toho intervalu bylo časové okno surového signálu pořízeno, je automaticky vyčteno z databáze.


Konfigurace tréninku
×

Model:
Bi_LSTM_RUL_Production vv1


Parametry životního cyklu ložiska

Datum instalace (Start RUL = 1.0)



Datum výměny/selhání (Konec RUL = 0.0)



Kdy bylo nové ložisko nasazeno do provozu.

Okamžik, kdy ložisko dosáhlo konce životnosti.

Poznámka: Datum pořízení datového záznamu bude automaticky spárováno s tímto intervalem pro výpočet zbývající životnosti.

Vybrané datové úseky
+ Přidat data

Stroj / Senzor	Období měření	Vzorků	Odstranit
B&R Kostka 3	15. 5. 2026 15:57:00	1	

Zrušit
Spustit Fine-Tuning

Obr. 35: Konfigurace fine-tuningu modelu pro predikci zbývající životnosti.

Tyto informace jsou posílány backendem do modulu pro strojového učení, kde jsou vypočteny příznaky ze zvolených časových oken a je k nim dopočteno v jaké fázi degradace se ložisko nacházelo. Takto zpracovaná data jsou potom vstupem pro dotrénování modelů pro predikci zbývající životnosti.

8.4 Produkční nasazení – kontejnerizace

Analogicky k hlavní backendové aplikaci (viz podkapitola 6.5) je i mikroslužba pro strojové učení (ml_service) plně kontejnerizována s využitím technologie Docker. Tento přístup izoluje rozsáhlé matematické a hluboké učící knihovny (jako jsou PyTorch, SciPy či PyWavelets) od hostitelského operačního systému průmyslového serveru a garantuje absolutní přenositelnost běhového prostředí. Jako základní obraz byl opětovně zvolen odlehčený python:3.11-slim. Postup definice pracovního adresáře, správa a cachování závislostí z requirements.txt, jakož i distribuce výsledného obrazu přes repozitář Docker Hub, sdílí identické principy a přínosy detailně popsané v sekci věnované kontejnerizaci backendu.

Hlavní specifika a odlišnosti v konfiguračním souboru Dockerfile pro mikroslužbu strojového učení vyplývají z hlubokých matematických a vizualizačních nároků použitých algoritmů. Jelikož minimalistický základní obraz neobsahuje žádné nadbytečné systémové balíčky, bylo nutné do sestavovacího procesu explicitně integrovat příkaz `RUN apt-get update && apt-get install -y libgl1 libglib2.0-0`. Tyto nativní systémové knihovny jsou kriticky vyžadovány nízkourovňovými moduly pro zpracování signálů a grafické vykreslování

(např. knihovnou matplotlib při asynchronním generování vlnkových spektrogramů CWT do klientského rozhraní).

Na rozdíl od backendu kontejner pro strojové učení nevyžaduje vlastní rutinu pro zakládání FTP složek, neboť k surovým CSV souborům přistupuje přímo prostřednictvím sdílených svazků (volumes) v rámci orchestrace. V závěru konfiguračního souboru je pak spouštěcí příkaz pro server Uvicorn směřován na dedikovaný síťový port 8001. Tato izolace portů umožňuje orchestrátorskému pluginu Docker Compose jednoznačně oddělit inferenční uzly od zbytku systému a bezpečně je propojit do společné virtuální sítě.

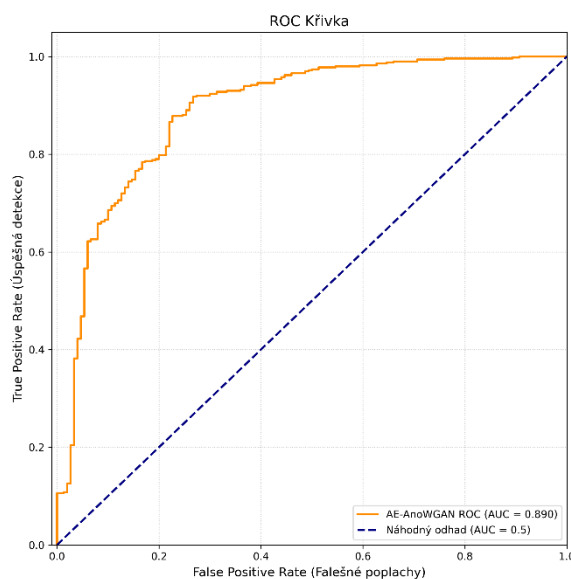
9 EVALUACE SYSTÉMU

9.1 Evaluace modelů strojového učení

V této kapitole budou postupně zhodnoceny použité modely strojového učení. Každý model bude ohodnocen z hlediska procesu tréninku na veřejně dostupném datasetu a zapojení do architektury aplikace.

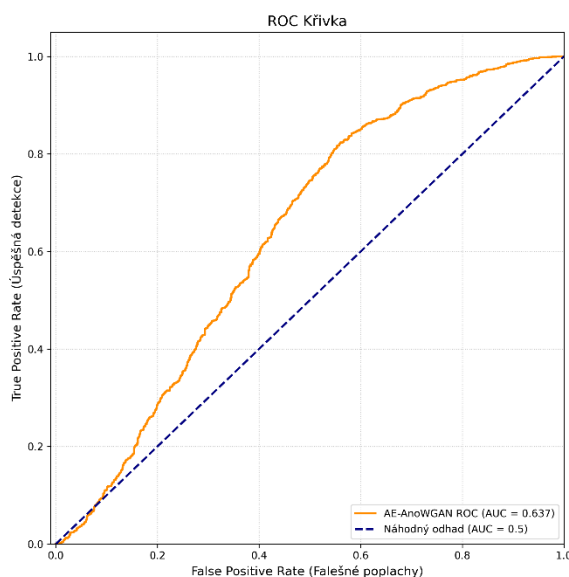
9.1.1 Model AE-AnoWGAN

V první fázi evaluace byl model podroben validaci na standardizovaném datasetu (PU), přičemž se podařilo úspěšně replikovat výsledky uváděné v literatuře. Jak dokládá obrázek 36, model dosáhl plochy pod křivkou (AUC) na úrovni 0,890, což indikuje velmi vysokou separabilitu mezi zdravým stavem a poškozením. Tato metrika AUC (Area Under Curve) kvantifikuje schopnost modelu odlišit anomální vzorky od nominálních; hodnota 0,890 přitom vyjadřuje, že model s 89% pravděpodobností správně identifikuje anomálii při zachování nízké míry falešně pozitivních výsledků. Tato úspěšná verifikace potvrzuje, že implementovaná architektura generátoru a diskriminátoru je principiálně správná a schopná učit se distribuci vibračních dat.



Obr. 36: ROC křivka modelu při tréninku na uvedeném datasetu.

S cílem adaptovat model na specifika našeho testovacího hardwaru byl následně proveden experiment, při kterém byla upravena vstupní data procesu: byl aplikován down-sampling a došlo k úpravě délky časových oken pro zachycení širšího kontextu vibrací. Výsledky tohoto experimentu, jsou zachycené na obrázku 37. Výsledky však ukazují na významnou degradaci separability modelu, kdy hodnota AUC klesla na 0,637, což lze považovat za zhroucení modelu.

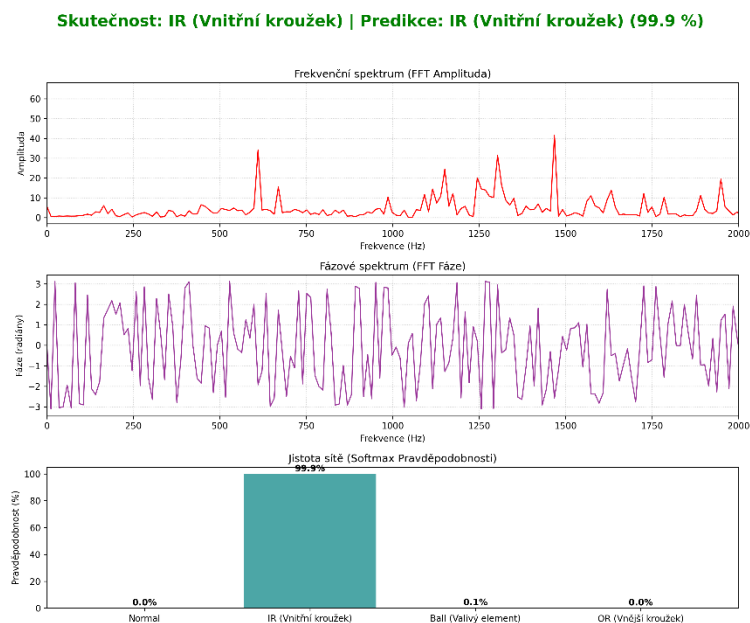


Obr. 37: ROC křivka po down-samplingu.

Příčinou tohoto kolapsu je pravděpodobně ztráta vysokofrekvenčních informací v důsledku down-samplingu. Tyto informace jsou pro diagnostiku valivých ložisek klíčové, neboť charakteristické frekvence závad (BPFO, BPFI) se často nacházejí ve vyšších pásmech spektra. Zvětšení časových oken, ačkoliv teoreticky výhodné pro zachycení dlouhodobých trendů, vedlo v kombinaci s nižším vzorkováním k „rozmělnění“ příznaků anomálie v rámci jednoho vstupního tenzoru. Výsledky experimentu tak jasně ukazují, že pro úspěšnou detekci anomálií metodou AE-AnoWGAN je naprosto kritické správné nastavení časového okna, které musí být exaktně dimenzováno podle fyzikálních parametrů konkrétního hardwaru. Časové okno není pouze hyperparametrem modelu, ale musí reflektovat mechanickou povahu stroje, zejména frekvenci rotace a očekávaný charakter degračních projevů. Jakákoliv odchylka od této fyzikální korespondence mezi vzorkovací frekvencí a délkou okna vede k degradaci citlivosti modelu na minoritní strukturální odchylky a následnému překryvu anomálních skóre zdravých a poškozených vzorků. Tato zkušenost potvrzuje, že efektivní nasazení generativních modelů pro detekci anomálií vyžaduje nejen pokročilé metody strojového učení, ale především hluboké porozumění fyzikální podstatě monitorovaného procesu.

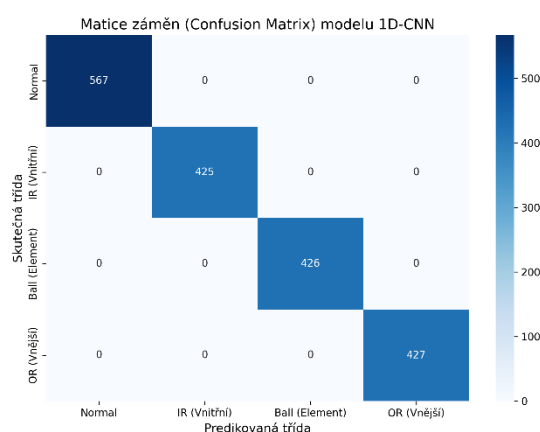
9.1.2 Model 1DCNN

Pro klasifikaci konkrétního typu poruchy byla zvolena architektura 1D-CNN, která jako vstupní příznaky využívá amplitudové a fázové spektrum získané z FFT. Hlavní předností tohoto přístupu je vysoká míra interpretovatelnosti, neboť diagnostik může přímo vizualizovat frekvenční příznaky, které vedly k rozhodnutí sítě viz obrázek 38.



Obr. 38: Predikce 1DCNN podle frekvenčního a fázového spektra FFT.

Evaluace klasifikačního modelu proběhla na vyčleněné testovací množině dat, přičemž dosažená celková přesnost (Accuracy) činila téměř 100 %. Detailní rozložení klasifikace je vizualizováno prostřednictvím matice záměn (Confusion Matrix) na obrázku 39. Z diagonály této matice je evidentní, že model dokázal přesně oddělit všechny testovací vzorky do příslušných čtyř kategorií (Normal, IR, Ball, OR) bez jediné falešně pozitivní či falešně negativní predikce. Tento výsledek potvrzuje, že zvolená reprezentace signálu poskytuje konvoluční síti maximálně diskriminativní příznaky pro spolehlivou diagnostiku na daném datasetu.



Obr. 39: Matice záměn 1DCNN.

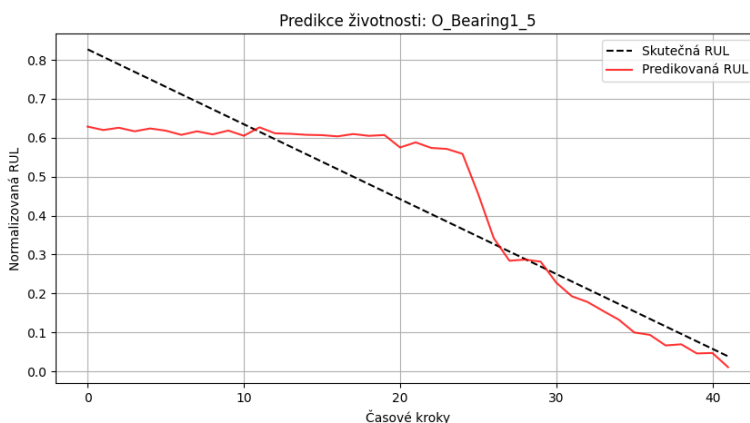
Z hlediska závislosti na parametrech měření je nutné zdůraznit, že 1D-CNN s FFT transformací sdílí principiálně stejnou citlivost na parametry sběru dat jako model AE-AnoWGAN. Experiment s přímou adaptací trénovacích dat na parametry cílového hardwaru (jako v předchozím případě) nebyl z časových důvodů u tohoto modelu proveden. Lze však

s technickou jistotou předpokládat, že by nasazení původního modelu na reálná data vedlo k jeho selhání. Ačkoliv je vzorkovací frekvence testovacího datasetu (12 kHz) relativně blízká vzorkovací frekvenci použitého hardwaru (12,8 kHz), propastným rozdílem je odlišná délka časového okna. Rozlišení frekvenčního spektra je těmito parametry přímo determinováno, a jejich změna v produkčním prostředí by nevyhnutelně vedla k posunu charakteristických frekvenčních špiček závad vůči naučeným vahám konvolučních filtrů.

9.1.3 Model Bi-LSTM

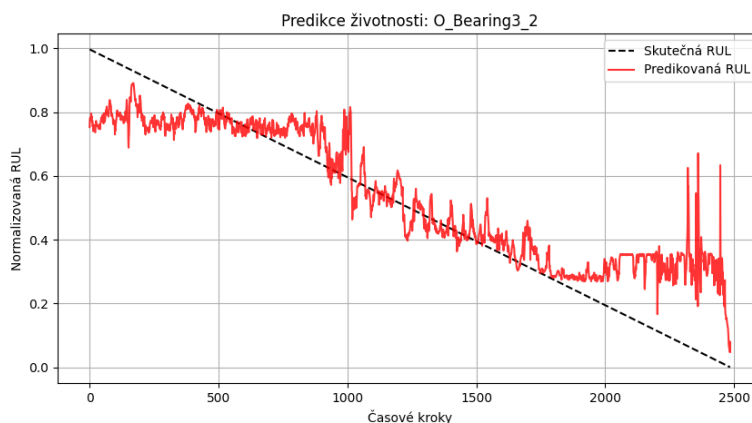
Závěrečnou fází diagnostického řetězce je predikce zbývající užitečné životnosti (RUL) ložiska, pro niž byla implementována rekurentní architektura Bi-LSTM. Během experimentální fáze a procesu ladění hyperparametrů se potvrdilo, že se jedná o suverénně nejnáročnější úlohu z celé předkládané pipeline. K této komplexitě významně přispělo i architektonické rozhodnutí nevyužít identické vstupní příznaky jako v původní referenční studii, nýbrž adaptovat vstupní tenzory na formu korespondující s celkovou koncepcí vyvíjené aplikace.

S cílem snížit složitost problému a stabilizovat tréninkový proces byla zvolena strategie dekompozice úlohy, kdy byly namísto jednoho univerzálního modelu trénovány tři separátní modely dedikované pro konkrétní typy poruch (vnitřní kroužek, vnější kroužek, valivý element). Navzdory tomuto zjednodušení však evaluace na testovacích sadách referenčního datasetu XJTU-SY prokázala značnou variabilitu ve schopnosti modelu generalizovat napříč různými provozními podmínkami.



Obr. 40: Predikce životnosti u ložiska 1_5 z XJTU-SY datasetu.

Tuto nestabilitu názorně demonstrují přiložené grafy pro poruchu vnějšího kroužku (OR). Na obrázku 40 je zachycena predikce pro ložisko pod specifickým zatížením. Model zde vykazuje funkční, byť mírně nelineární chování: v iniciální fázi životnosti, kdy je progresse poškození minimální, drží predikci na konstantní, vysoké úrovni. Následně, po krátkém období latence, model velmi přesně a plynule kopíruje strmou křivku reálné degradace v kritické závěrečné fázi.



Obr. 41: Predikce životnosti pro ložisko 3_2 z XJTU-SY datasetu.

Naproti tomu obrázek 41 zobrazuje výstup totožného modelu na ložisko se stejným typem poruchy, avšak vystavené odlišnému provoznímu zatížení. V tomto scénáři je predikce zatížena silnými oscilacemi napříč celým měřením. Zásadním nedostatkem je však především chování v terminální fázi životnosti (okolo časového kroku 2000), kdy model ztrácí schopnost sledovat strmý pokles skutečné RUL a vykazuje stagnaci na úrovni přibližně 30 %, než prudce propadne až v samotném závěru.

Dynamika šíření únavového poškození je natolik závislá na fyzikálním zatížení ložiska, že modely trénované za určitých podmínek jen obtížně generalizují na podmínky odlišné. Tento závěr potvrzuje trend zjištěný u předchozích modelů, a sice že efektivního nasazení predikce RUL lze dosáhnout výhradně tréninkem a specifikací modelu na datech odpovídajících exaktním fyzickým charakteristikám cílového stroje.

9.1.4 Celkové zhodnocení modulu strojového učení

Evaluace všech tří implementovaných architektur (AE-AnoWGAN, 1D-CNN, Bi-LSTM) napříč celým diagnostickým řetězcem jasně prokázala stěžejní inženýrský fakt: úspěšnost a robustnost modelů strojového učení v oblasti vibrodiagnostiky je absolutně závislá na rozsáhlém spektru faktorů reálného provozu. Přesnost predikcí není determinována pouze architekturou neuronové sítě, ale primárně fyzikální realitou monitorovaného uzlu, zahrnující typ ložiska, charakter jeho zatížení, dynamiku šíření únavy a v neposlední řadě také exaktními parametry digitálního zpracování signálu, jakými jsou vzorkovací frekvence a délka časového okna.

Vzhledem k tomu, že zadání této práce nebylo od počátku vázáno na dedikovaný průmyslový hardware a nebyly k dispozici rozsáhlé historické záznamy z konkrétního reálného provozu, probíhal trénink a primární evaluace na standardizovaných datasetech (PU, CWRU, XJTU-SY). Cílem této práce nicméně nebylo vytvořit jeden univerzální model odolný vůči všem myslitelným průmyslovým scénářům – takový cíl je z fyzikálního hlediska v podstatě nedosažitelný. Hlavním cílem bylo prokázat principiální použitelnost a adekvátnost vybraných metod hlubokého učení pro jednotlivé diagnostické fáze (detekce, klasifikace, predikce RUL). Tento cíl byl beze zbytku naplněn.

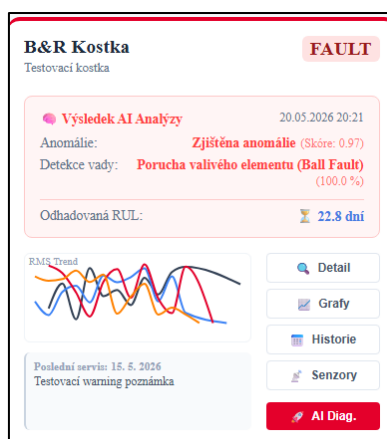
Zásadním přínosem práce je skutečnost, že tyto modely nezůstaly pouze ve stadiu teoretických experimentů či izolovaných výpočetních skriptů, nýbrž byly plně zaintegrovány do ucelené a nasaditelné softwarové pipeline.

Pro systematické řešení inherentního problému – tedy nesouladu mezi laboratorními daty a variabilní fyzikální realitou – byl v rámci aplikace navržen a implementován MLOps modul pro průběžné dotrénovávání (fine-tuning). Tento koncept představuje klíčový most mezi obecným modelem a specifickým strojem, neboť umožňuje operátorům adaptovat prediktivní kapacity systému na základě aktuálních dat zprostředkovaných lokálními senzory. Ačkoliv z objektivních důvodů (absence dlouhodobého provozu na reálném stroji a časová i výpočetní náročnost) nemohla být efektivita tohoto modulu provedena na skutečném hardwaru, celá softwarová i databázová infrastruktura je pro tento proces plně připravena. Navržené řešení tak představuje vysoce komplexní, architektonicky robustní a v praxi reálně využitelné jádro pro moderní systémy prediktivní údržby v éře Průmyslu 4.0.

9.2 Nasazení aplikace na APC

Pro finální ověření funkčnosti celého navrženého systému byla kompletní mikroslužbová architektura nasazena na cílové průmyslové PC (APC100). K orchestraci a plynulému spuštění všech kontejnerizovaných modulů (frontend, backend, databáze a ML servis) byl využit orchestrační nástroj Docker Compose. Pro potřeby tohoto nasazení byla vytvořena konfigurační sada souborů sestávající z definičního souboru `docker-compose.yaml`, inicializačního databázového skriptu `init.sql` a souboru pro správu proměnných prostředí `.env` (úplné zdrojové texty těchto konfiguračních souborů jsou k dispozici v Příloze D).

Ověření a verifikace funkčnosti celé datové a diagnostické pipeline proběhly v rámci laboratorního testovacího zapojení v rámci B&R pobočky v Brně. Vzhledem k prostorovým a materiálovým možnostem byly měřicí senzory umístěny na testovací desce (nikoliv na reálném rotujícím stroji v plném provozním zatížení). Fyzický diagnostický řetězec zahrnoval kartu pro měření vibrací (X20CM4810), příslušný modul sběrnice kontroléru (B&R X20BC0083), průmyslové PLC (X20CP3586) a samotné APC910. Všechny tyto hardwarové komponenty byly vzájemně propojeny a integrovány do jednotné podnikové sítě, v níž probíhala komunikace s PLC.



Obr. 42: Vizualizace karty stroje po analýze modelů strojového učení.

Byla úspěšně ověřena proveditelnost obou navržených postupů pro sběr dat. V obou provozních módech data korektně protekla skrze backendové API až do dedikované ML služby, kde proběhla úspěšná inference všech nasazených modelů. Výsledný stav uživatelského rozhraní po spuštění kompletní analytické pipeline a vyhodnocení technického stavu je zachycen na obrázku 42, který detailně zobrazuje klientskou komponentu machineCard. Výsledky analýzy jsou pouze ilustrační. Tato komponenta přehledně konsoliduje výstupy z detektoru anomálií, klasifikátoru závad i prediktoru zbytkové životnosti do jednotného operátorského dashboardu.

V souladu s předchozími závěry je nutné v rámci celkového vyhodnocení nasazení zmínit, že integrovaný modul pro průběžný fine-tuning modelů nebyl v reálných podmínkách APC otestován. Tento fakt byl způsoben neexistencí dat generovaných testovací sadou na desce, která ze své podstaty nemohla simulovat dlouhodobý vývoj mechanického opotřebení ložisek. Avšak návrh softwarových vazeb je připraven pro testování.

9.3 Průmyslová využitelnost

Z hlediska softwarové architektury byl celý systém od počátku vyvíjen s maximálním důrazem na modularitu a snadnou přenositelnost, čehož bylo dosaženo plnou kontejnerizací pomocí technologie Docker. Toto řešení teoreticky umožňuje nasazení plug-and-play na jakémkoliv průmyslovém PC (APC) splňujícím minimální hardwarové požadavky. Přesto je nutné objektivně konstatovat, že předkládaná aplikace v současném stadiu reprezentuje funkční prototyp (Proof of Concept), nikoliv finální komerční produkt. Vzhledem k absenci dlouhodobého testování na reálném rotačním stroji v továrním prostředí nebyl systém vystaven reálným provozním extrémům, jako je nestacionární zatížení, proměnlivé rušení či výpadky sítě. Před plným průmyslovým nasazením by tedy musela proběhnout rozsáhlá validace a trénink modelů strojového učení v reálných provozních podmínkách.

Při úvahách o průmyslovém nasazení se jako největší architektonické dilema ukazuje právě přítomnost modulu pro fine-tuning modelů. Tento koncept s sebou nese zásadní nevýhodu v podobě obrovské datové náročnosti samotné aplikace. Vzhledem k nutnosti

integrovat kompletní trénovací ekosystém knihovny PyTorch přesahuje velikost kontejneru mikroslužby strojového učení 10 GB. V podmínkách Edge computingu (výpočty přímo na stroji), kde jsou paměťové a síťové zdroje často omezené, se jedná o nezanedbatelnou zátěž. Pokud by byla aplikace osekána čistě na inferenci (např. pomocí formátů ONNX nebo TensorRT), velikost modulu by klesla o více než 90 %, avšak systém by ztratil schopnost adaptace.

Na druhou stranu je nutné zdůraznit, že nedostatek kvalitních, anotovaných a balancovaných dat představuje v současnosti tu vůbec největší bariéru pro plošné nasazování strojového učení do vibrodiagnostiky. Právě navržený koncept fine-tuningu se vůči tomuto problému staví čelem a představuje strategickou cestu s obrovským potenciálem. Umožňuje totiž vyhnout se nutnosti trénovat univerzální modely a místo toho efektivně využívá specifická data, která stroj produkuje během svého reálného života.

Závěrem lze tedy konstatovat, že vytvořená aplikace představuje komplexní a velmi nadějný návrh řešení end-to-end vibrodiagnostické pipeline. Nabízí ucelený pohled na to, jak lze moderní webové technologie, průmyslové PLC a pokročilou umělou inteligenci spojit do jednoho funkčního celku pro prediktivní údržbu v éře Průmyslu 4.0.

10 ZÁVĚR

Cílem této diplomové práce bylo navrhnout komplexní vibrodiagnostický systém využívající metody strojového učení pro účely prediktivní údržby, a to s přímou vazbou na průmyslový hardware. Na základě dosažených výsledků a úspěšného nasazení celého softwarového řetězce na aplikační PC lze konstatovat, že všechny zadané cíle práce byly úspěšně naplněny. Byla vytvořena ucelená architektura, která pokrývá celý životní cyklus dat – od jejich vyčítání z programovatelného logického automatu (PLC) a sběrnicových modulů, přes komplexní databázové operace, až po pokročilou analytiku a vizualizaci v moderním webovém uživatelském rozhraní.

V oblasti strojového učení byla úspěšně realizována třístupňová diagnostická pipeline. Generativní model AE-AnoWGAN potvrdil svou schopnost detekovat minoritní odchylky od nominálního stavu, model 1D-CNN pracující s FFT transformací dosáhl na testovacích datech bezchybné klasifikace typu poruchy a rekurentní síť Bi-LSTM byla aplikována na komplexní úlohu odhadu zbytkové životnosti (RUL). Během evaluace těchto modelů se však naplno projevila stěžejní výzva současné vibrodiagnostiky – extrémní citlivost modelů na fyzikální parametry měření a variabilitu provozních podmínek. Bylo prokázáno, že modely trénované na standardizovaných datasetech (např. CWRU či XJTU-SY) nelze bez dalších úprav přímo aplikovat na data s odlišnou vzorkovací frekvencí či délkou časového okna získaná za jiných provozních podmínek než zmiňované datasety.

Jako odpověď na tento problém je představen a softwarově implementován dedikovaný modul pro fine-tuning. Tento koncept představuje klíčový přínos práce, neboť posouvá systém od statického vyhodnocování k dynamické adaptaci. Ačkoliv modul nemohl být z důvodu absence dlouhodobých degradačních dat z reálného stroje plně otestován, představuje architektonicky robustní řešení, jak bojovat s nedostatkem kvalitních trénovacích dat přímo v místě nasazení (Edge computing). Celá aplikace byla navíc plně kontejnerizována pomocí technologie Docker, což umožňuje její snadné nasazení na průmyslových počítačích a otevírá cestu k další škálovatelnosti.

Další vývoj navrhovaného řešení architektury vibrodiagnostického systému by měl směřovat k dlouhodobému testování na reálném rotačním stroji, což by umožnilo fyzicky validovat proces fine-tuningu na vlastních provozních datech.

Na úplný závěr bych chtěl vyjádřit vděčnost za příležitost postavit se výzvě v podobě takto komplexního inženýrského problému, který propojil hardwarovou infrastrukturu, softwarový vývoj a umělou inteligenci do jednoho funkčního celku. Zkušenosti získané při návrhu, implementaci a oživování celého systému mi poskytly neocenitelný technický rozhled a upřímně se těším, až tyto znalosti naplno uplatním ve svém budoucím profesním životě.

SEZNAM POUŽITÉ LITERATURY

- [1] RANDALL, ROBERT BOND. *Vibration-based condition monitoring: industrial, aerospace and automotive applications* [online]. Chichester: John Wiley & Sons, 2011. ISBN 978-0-470-97766-8. Dostupné z: doi:10.1002/9780470977668
- [2] LIU, Ruonan, Dong XIAO, Di LIN a Weidong ZHANG. Intelligent Bearing Anomaly Detection for Industrial Internet of Things Based on Auto-Encoder Wasserstein Generative Adversarial Network. *IEEE Internet of Things Journal* [online]. 2024, **11**(13), 22869–22879. ISSN 2327-4662, 2372-2541. Dostupné z: doi:10.1109/JIOT.2024.3358871
- [3] FIDALI, Marek, Damian AUGUSTYN, Jakub OCHMANN a Wojciech UCHMAN. Evaluation of the Diagnostic Sensitivity of Digital Vibration Sensors Based on Capacitive MEMS Accelerometers. *Sensors* [online]. 2024, **24**(14), 4463. ISSN 1424-8220. Dostupné z: doi:10.3390/s24144463
- [4] B&R INDUSTRIAL AUTOMATION GMBH. *X20CM4810 Data sheet and user's manual* [online]. Eggelsberg: B&R Industrial Automation GmbH. 2025. Dostupné z: <https://www.br-automation.com/en/products/io-systems/x20-system/other-functions/x20cm4810/>
- [5] YANG, Hongyu, Joseph MATHEW a Lin MA. Vibration Feature Extraction Techniques for Fault Diagnosis of Rotating Machinery -A Literature Survey. nedatováno.
- [6] PROAKIS, John G. a Dimitris G. MANOLAKIS. *Digital signal processing*. 4th ed. Upper Saddle River, N.J: Pearson Prentice Hall, 2007. ISBN 978-0-13-187374-2.
- [7] MALLAT, S. G. *A wavelet tour of signal processing: the sparse way*. 3rd ed. Amsterdam ; Boston: Elsevier/Academic Press, 2009. ISBN 978-0-12-374370-1.
- [8] PENG, Z.K. a F.L. CHU. Application of the wavelet transform in machine condition monitoring and fault diagnostics: a review with bibliography. *Mechanical Systems and Signal Processing* [online]. 2004, **18**(2), 199–221. ISSN 08883270. Dostupné z: doi:10.1016/S0888-3270(03)00075-X
- [9] BOUAOUICHE, Karim, Yamima MENASRIA a Dalila KHALFA. Detection of defects in a bearing by analysis of vibration signals. *Diagnostyka* [online]. 2023, **24**(2), 1–7. ISSN 2449-5220. Dostupné z: doi:10.29354/diag/162230
- [10] JASON MAIS. *Vibration Diagnostic Guide* [online]. CM5118 EN. B.m.: SKF. 2002. Dostupné z: https://cdn.skfmediahub.skf.com/api/public/0901d1968024acef/pdf_preview_medium/0901d1968024acef_pdf_preview_medium.pdf
- [11] JARDINE, Andrew K.S., Daming LIN a Dragan BANJEVIC. A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing* [online]. 2006, **20**(7), 1483–1510. ISSN 08883270. Dostupné z: doi:10.1016/j.ymssp.2005.09.012

- [12] GOODFELLOW, Ian, Yoshua BENGIO a Aaron COURVILLE. *Deep learning*. Cambridge, Mass: The MIT press, 2016. Adaptive computation and machine learning. ISBN 978-0-262-03561-3.
- [13] CHANDOLA, Varun, Arindam BANERJEE a Vipin KUMAR. Anomaly detection: A survey. *ACM Computing Surveys* [online]. 2009, **41**(3), 1–58. ISSN 0360-0300, 1557-7341. Dostupné z: doi:10.1145/1541880.1541882
- [14] LIU, Fei Tony, Kai Ming TING a Zhi-Hua ZHOU. Isolation Forest. In: *2008 Eighth IEEE International Conference on Data Mining (ICDM): 2008 Eighth IEEE International Conference on Data Mining* [online]. Pisa, Italy: IEEE, 2008, s. 413–422 [vid. 2026-04-17]. ISBN 978-0-7695-3502-9. Dostupné z: doi:10.1109/ICDM.2008.17
- [15] YANG, Zheng, Binbin XU, Wei LUO a Fei CHEN. Autoencoder-based representation learning and its application in intelligent fault diagnosis: A review. *Measurement* [online]. 2022, **189**, 110460. ISSN 02632241. Dostupné z: doi:10.1016/j.measurement.2021.110460
- [16] WIDODO, Achmad a Bo-Suk YANG. Support vector machine in machine condition monitoring and fault diagnosis. *Mechanical Systems and Signal Processing* [online]. 2007, **21**(6), 2560–2574. ISSN 08883270. Dostupné z: doi:10.1016/j.ymssp.2006.12.007
- [17] HAN, Te, Dongxiang JIANG, Qi ZHAO, Lei WANG a Kai YIN. Comparison of random forest, artificial neural networks and support vector machine for intelligent diagnosis of rotating machinery. *Transactions of the Institute of Measurement and Control* [online]. 2018, **40**(8), 2681–2693. ISSN 0142-3312, 1477-0369. Dostupné z: doi:10.1177/0142331217708242
- [18] HAKIM, Mohammed, Abdoulhadi A. Borhana OMRAN, Jawaid I. INAYAT-HUSSAIN, Ali Najah AHMED, Hamdan ABDELLATEF, Abdallah ABDELLATIF a Hassan Muwafaq GHENI. Bearing Fault Diagnosis Using Lightweight and Robust One-Dimensional Convolution Neural Network in the Frequency Domain. *Sensors* [online]. 2022, **22**(15), 5793. ISSN 1424-8220. Dostupné z: doi:10.3390/s22155793
- [19] YAO, Xuejian, Junjun ZHU, Quansheng JIANG, Qin YAO, Yehu SHEN a Qixin ZHU. RUL prediction method for rolling bearing using convolutional denoising autoencoder and bidirectional LSTM. *Measurement Science and Technology* [online]. 2024, **35**(3), 035111. ISSN 0957-0233, 1361-6501. Dostupné z: doi:10.1088/1361-6501/ad123c
- [20] B&R INDUSTRIAL AUTOMATION GMBH. *X20(c)CP168x(X) and 368x(x) Data sheet* [online]. Eggelsberg, Rakousko: B&R Industrial Automation GmbH. leden 2026. Dostupné z: <https://www.br-automation.com/en/downloads/control-and-io-systems/x20-system/cpus/x20cp3687x/data-sheet-x20cpx68xx/>
- [21] QIU, Tie, Jiancheng CHI, Xiaobo ZHOU, Zhaolong NING, Mohammed ATIQUEZZAMAN a Dapeng Oliver WU. Edge Computing in Industrial Internet of Things: Architecture, Advances and Challenges. *IEEE Communications Surveys & Tutorials* [online]. 2020, **22**(4), 2462–2488. ISSN 1553-877X, 2373-745X. Dostupné z: doi:10.1109/COMST.2020.3009103

- [22] DOCKER INC. Home. *Docker Documentation* [online]. [vid. 2026-04-25]. Dostupné z: <https://docs.docker.com/>
- [23] GOLDSCHMIDT, Thomas, Stefan HAUCK-STATTELMANN, Somayeh MALAKUTI a Sten GRÜNER. Container-based architecture for flexible industrial control applications. *Journal of Systems Architecture* [online]. 2018, **84**, 28–36. ISSN 13837621. Dostupné z: doi:10.1016/j.sysarc.2018.03.002
- [24] NEWMAN, Sam. *Building microservices: designing fine-grained systems*. First Edition. Beijing Sebastopol, CA: O'Reilly Media, 2015. ISBN 978-1-4919-5035-7.
- [25] *pandas documentation — pandas 3.0.3 documentation* [online]. [vid. 2026-05-19]. Dostupné z: <https://pandas.pydata.org/docs/>
- [26] *SciPy documentation — SciPy v1.17.0 Manual* [online]. [vid. 2026-05-19]. Dostupné z: <https://docs.scipy.org/doc/scipy/>
- [27] *NumPy documentation — NumPy v2.4 Manual* [online]. [vid. 2026-05-19]. Dostupné z: <https://numpy.org/doc/stable/>
- [28] GREGORY LEE, a kol. *PyWavelets/pywt: v1.9.0* [online]. B.m.: Zenodo. 4. srpen 2025 [vid. 2026-05-19]. Dostupné z: doi:10.5281/ZENODO.1407171
- [29] *Matplotlib documentation — Matplotlib 3.10.9 documentation* [online]. [vid. 2026-05-19]. Dostupné z: <https://matplotlib.org/stable/index.html>
- [30] *FastAPI* [online]. [vid. 2026-04-25]. Dostupné z: <https://fastapi.tiangolo.com/>
- [31] FOWLER, Matthew. *Python concurrency with asyncio*. Shelter Island: Manning Publications, 2021. ISBN 978-1-61729-866-0.
- [32] *SQLAlchemy Documentation — SQLAlchemy 2.0 Documentation* [online]. [vid. 2026-05-21]. Dostupné z: <https://docs.sqlalchemy.org/en/20/>
- [33] Welcome to Pydantic. *Pydantic Docs* [online]. [vid. 2026-05-21]. Dostupné z: <https://pydantic.dev/docs/validation/latest/get-started/>
- [34] Django documentation | Django documentation. *Django Project* [online]. [vid. 2026-05-21]. Dostupné z: <https://docs.djangoproject.com/en/6.0/>
- [35] *Welcome to Flask — Flask Documentation (3.1.x)* [online]. [vid. 2026-05-21]. Dostupné z: <https://flask.palletsprojects.com/en/stable/>
- [36] *Uvicorn* [online]. [vid. 2026-04-25]. Dostupné z: <https://uvicorn.dev/>
- [37] Technical documentation. *Grafana Labs* [online]. [vid. 2026-05-21]. Dostupné z: <https://grafana.com/docs/>
- [38] *Quick Start – React* [online]. [vid. 2026-05-19]. Dostupné z: <https://react.dev/learn>
- [39] *nginx documentation* [online]. [vid. 2026-05-20]. Dostupné z: <https://nginx.org/en/docs/>

- [40] PUNTAMBEKAR, Rohan, Pratyaksh VYAS, Ankit THAKKAR a Dhaval PATEL. A survey of machine learning and deep learning methods for vibration-based Bearing fault diagnosis: The need, challenges, and potential future research directions. *Neurocomputing* [online]. 2026, **659**, 131628. ISSN 09252312. Dostupné z: doi:10.1016/j.neucom.2025.131628

SEZNAM PŘÍLOH

Příloha A – Prohlášení o využití AI v rámci práce

Příloha B – Zdrojové kódy jednotlivých služeb

Příloha C – Zdrojové kódy využité při tréninku modelů strojového učení

Příloha D – Konfigurační soubory pro orchestrátor docker compose

Příloha E – Důležité odkazy